

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

**ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА
ПО ПРОГРАММЕ МАГИСТРАТУРЫ**

09.03.04 Программная инженерия

(код, наименование ОПОП ВО: направление подготовки, направленность (профиль))

«Предпринимательство, инновации и технологии будущего в программной инженерии»

Интеллектуальная система оптимизации производственной деятельности,

на основе АИС

(название темы)

Дипломный проект

(вид ВКР: дипломная работа или дипломный проект)

Автор ВКР

(подпись, дата)

Д. А. Каракчиев

(инициалы, фамилия)

Группа ПО-41м

Руководитель ВКР

(подпись, дата)

Р. А. Томакова

(инициалы, фамилия)

Нормоконтроль

(подпись, дата)

А.В. Какурина

(инициалы, фамилия)

ВКР допущена к защите:

Заведующий кафедрой

(подпись, дата)

А. В. Малышев

(инициалы, фамилия)

Курск 2026 г.

Юго-Западный государственный университет

Кафедра _____ программной инженерии

УТВЕРЖДАЮ:

Заведующий кафедрой

(подпись, инициалы, фамилия)

«_____» _____ 20____ г.

ЗАДАНИЕ НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ ПО ПРОГРАММЕ МАГИСТРАТУРЫ

Студента Каракчиева Д.А., шифр 24-06-0544, группа ПО-41м

1. Тема «Интеллектуальная система оптимизации производственной деятельности, на основе АИС» утверждена приказом ректора ЮЗГУ от «03» апреля 2026 г. № 1586-с.

2. Срок предоставления работы к защите «20» июня 2026 г.

3. Исходные данные для создания программной системы:

3.1. Перечень решаемых задач:

1) определить функциональные и технические требования разрабатываемого приложения;

2) разработать концептуальную модель интеллектуальной системы производственного планирования;

3) спроектировать программную систему построения расписаний и адаптивной диспетчеризации;

4) сконструировать и протестировать программную систему с нейросетевыми модулями планирования.

3.2. Входные данные и требуемые результаты для программы:

1) Производственные заказы – перечень заказов с указанием сроков сдачи, приоритетов и технологических операций, определяющих последовательность и содержание работ.

2) Производственные ресурсы – сведения о станках и рабочих центрах, их текущем состоянии, коэффициенте готовности, признаках ремонта и специализации по типам операций.

3) Нормативно-справочная информация – перечень типов операций и допустимых связей «ресурс–тип», ограничивающих назначение операций на оборудование.

4) Обучающие выборки – синтетические данные о состояниях оборудования и эвристических назначениях операций, используемые для тренировки нейросетевых моделей.

5) Обученные нейросетевые модели – файлы весов, полученные в результате тренировки моделей-диспетчеров (PointerNet, SLIM, PPO), готовые к загрузке и применению.

6) Производственное расписание – назначенные для каждой операции ресурсы, время старта и окончания, сформированное системой по выбранному методу планирования.

7) Диаграмма Ганта – визуализация построенного расписания с цветовой кодировкой по типам операций и временной шкалой.

8) Рекомендации нейросетевого диспетчера – предлагаемые ресурсы для каждой неразмещённой операции, полученные на основе текущего состояния системы.

9) Метрики обучения – кривые потерь, точность на валидации, матрицы ошибок и отчёты классификации, сохраняемые в процессе тренировки моделей.

10) Сводные отчёты сравнения методов – показатели опоздания, makespan, коэффициент вариации загрузки и другие KPI, вычисляемые при тестировании различных методов планирования.

4. Перечень вопросов, подлежащих исследованию (разработке):

4.1 Анализ существующих решений в области производственного планирования и диспетчеризации, выявление их преимуществ и ограничений.

4.2 Исследование эвристических алгоритмов и методов обучения нейросетевых диспетчеров для задачи гибкого производственного расписания.

4.3 Сравнительный анализ эффективности нейросетевых и классических методов построения расписаний по ключевым производственным показателям.

5. Перечень графического материала:

Лист 1. Сведения о ВКРБ

Лист 2. Цель и задачи разработки

Лист 3. Диаграммы компонентов

Лист 4. Архитектура нейронной сети

Лист 5. Функция активации ReLU

Лист 6. Функция вычисления ошибки IoU Loss

Лист 7. Интерфейс приложения

Лист 8. Демонстрация работы программы

Лист 9. Заключение

Руководитель ВКР

(подпись, дата)

Р. А. Томакова

(инициалы, фамилия)

Задание принял к исполнению

(подпись, дата)

Д. А. Каракчиев

(инициалы, фамилия)

РЕФЕРАТ

Объем работы равен 116 страницам. Работа содержит 32 иллюстрации, 21 таблицу, 50 библиографических источников и 9 листов графического материала. Количество приложений – 2. Графический материал представлен в приложении А. Фрагменты исходного кода представлены в приложении Б.

Перечень ключевых слов: оперативное перепланирование, производственное планирование, диспетчеризация, гибкая производственная система, нейросетевые модели, обучение с подкреплением, поведенческое клонирование, самоконтролируемое обучение, эвристические алгоритмы, оптимизация расписаний, цифровой двойник производства.

Объектом исследования являются процессы оперативного производственного планирования на промышленных предприятиях с дискретным характером производства.

Целью выпускной квалификационной работы является разработка интеллектуальной системы производственного планирования, обеспечивающей построение выполнимых расписаний и адаптивную диспетчеризацию операций на основе нейросетевых методов.

При проведении исследований использовались методы системного анализа, математического моделирования, эвристической оптимизации, глубокого обучения и объектно-ориентированного программирования.

Разработана интеллектуальная система производственного планирования (APS-система) с графическим интерфейсом, включающая модули управления заказами и ресурсами, построения расписаний, обучения нейросетевых диспетчеров (PointerNet, SLIM, PPO) и сравнения методов планирования.

При создании программного продукта использовалась двухзвенная архитектура, технология объектно-ориентированного программирования и библиотеки глубокого обучения PyTorch. Для визуализации результатов применялись диаграмма Ганта и средства аналитической графики Matplotlib.

Программный продукт может быть интегрирован в корпоративную информационную среду предприятия и использоваться в качестве модуля оперативного планирования и диспетчеризации.

ABSTRACT

The volume of work is 116 pages. The work contains 32 illustrations, 21 table, 50 bibliographic sources and 9 sheets of graphic material. The number of applications is 2. The graphic material is presented in annex A. The layout of the site, including the connection of components, is presented in annex B.

List of keywords: operational rescheduling, production planning, dispatch, flexible production system, neural network models, reinforcement learning, behavioral cloning, self-supervised learning, heuristic algorithms, schedule optimization, digital production twin. The object of the study is the processes of operational production planning at industrial enterprises with a discrete nature of production.

The purpose of the final qualifying work is to develop an intelligent production planning system, ensuring the construction of executable schedules and adaptive dispatching of operations based on neural network methods.

When conducting research, methods of system analysis, mathematical modeling, heuristic optimization, deep learning and object-based learning were used. oriented programming.

An intelligent production planning system (APS-system) with a graphical interface has been developed, including modules for managing orders and resources, creating schedules, training neural network dispatchers (PointerNet, SLIM, PPO) and comparing planning methods.

When creating the software product, a two-tier architecture was used, object-oriented programming technology and deep learning libraries PyTorch. To visualize the results, a Gantt chart and Matplotlib analytical graphics tools were used.

The software product can be integrated into the corporate information environment of an enterprise and used as a module for operational planning and dispatch.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	13
1 Анализ предметной области	15
1.1 Эволюция систем производственного планирования	15
1.1.1 От ручных графиков к автоматизированным системам	15
1.1.2 Появление ERP-систем и их роль в планировании	15
1.1.3 Специализированные APS-системы	16
1.1.4 Индустрия 4.0 и искусственный интеллект в планировании	16
1.2 Основы производственного планирования и диспетчеризации	17
1.2.1 Задача составления расписаний	17
1.2.2 Гибкое производственное расписание (Flexible Job Shop)	17
1.2.3 Ключевые показатели эффективности расписаний	18
1.2.4 Неопределённость и человеческий фактор	18
1.2.5 Классификация методов решения FJSS	18
1.3 Системы управления производством: MES и ERP	19
1.3.1 ERP-системы	19
1.3.2 MES-системы	19
1.3.3 Интеграция ERP/MES/APS	19
1.3.4 Примеры программных продуктов	20
1.3.5 Сравнение централизованной и децентрализованной диспетче- ризации	20
1.4 Нейросетевые методы в планировании и диспетчеризации	21
1.4.1 Обзор подходов	21
1.4.2 Архитектуры нейронных сетей	21
1.4.3 Преимущества нейросетевых диспетчеров	22
1.4.4 Примеры внедрения	22
1.4.5 Сравнение методов обучения	23
1.5 Целесообразность внедрения интеллектуальной APS-системы	23
1.5.1 Критерии эффективности	23
1.5.2 Технические и организационные аспекты внедрения	24

1.5.3	Перспективы развития	24
1.6	Анализ производственных данных и потоков работ	25
1.6.1	Жизненный цикл производственного заказа	25
1.6.2	Типовая структура производственного цеха	25
1.6.3	Потоки данных между системами	25
1.6.4	Роль человека в автоматизированном планировании	26
2	Теоретические модели, методы и алгоритмы исследования	27
2.1	Модель системы планирования заказов	27
3	Разработка программного обеспечения информационно- вычислительной системы	28
3.1	Техническое задание	28
3.1.1	Основание для разработки	28
3.1.2	Назначение разработки	28
3.1.3	Требования к программе	29
3.1.3.1	Требования к данным программы	29
3.1.3.2	Концептуальная модель предметной области	30
3.1.3.3	Функциональные требования к программной системе	32
3.1.3.4	Требования пользователя к интерфейсу программной системы	38
3.1.3.5	Нефункциональные требования к программной системе	39
3.1.3.6	Требования к аппаратным средствам	40
3.1.3.7	Требования к программным средствам	41
3.1.3.8	Требования к архитектуре нейронной сети	41
3.1.4	Требования к оформлению документации	43
3.2	Технический проект	43
3.2.1	Общие сведения об интеллектуальной системе оптимизации	43
3.2.2	Проект данных интеллектуальной системы	44
3.2.3	Проектирование архитектуры программы	49
3.2.3.1	Компоненты программы	49
3.2.3.2	Архитектура информационно-вычислительной системы	51
3.2.3.3	Проектирование пользовательского интерфейса	54
3.3	Рабочий проект	58

3.3.1 Спецификация компонентов и классов интеллектуальной системы	58
3.3.2 Системное тестирование разработанного приложения	63
3.3.3 Сборка компонентов информационно-вычислительной системы	70
4 Результаты экспериментов с программной системой	71
4.1 Анализ модуля распределения заказов	71
4.1.1 Сравнение среднего опоздания заказов	71
4.1.2 Анализ длительности расписания (makespan)	72
4.1.3 Равномерность загрузки ресурсов	73
4.1.4 Равномерность загрузки ресурсов	74
4.1.5 Анализ устойчивости к изменению готовности ресурсов	77
4.1.6 Анализ динамики выполнения заказов	78
ЗАКЛЮЧЕНИЕ	82
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	83
ПРИЛОЖЕНИЕ А Представление графического материала	92
ПРИЛОЖЕНИЕ Б Фрагменты исходного кода программы	102
На отдельных листах (CD-RW в прикрепленном конверте)	116
Сведения о ВКРБ (Графический материал / Сведения о ВКРБ.png)	Лист 1
Цель и задачи разработки (Графический материал / Цель и задачи разработки.png)	Лист 2
Диаграммы компонентов (Графический материал / Диаграммы компонентов.png)	Лист 3
Архитектура нейронной сети (Графический материал / Архитектура нейронной сети.png)	Лист 4
Функция активации ReLU (Графический материал / Функция активации ReLU.png)	Лист 5
Функция вычисления ошибки IoU Loss (Графический материал / Функция вычисления ошибки IoU Loss.png)	Лист 6
Интерфейс приложения (Графический материал / Интерфейс приложения.png)	Лист 7
Демонстрация работы программы (Графический материал / Демонстрация	

работы программы.png)

Лист 8

Заключение (Графический материал / Заключение.png)

Лист 9

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

НС – нейронная сеть.

ИНС – искусственная нейронная сеть.

ИС – интеллектуальная система.

ИТ – информационные технологии.

ПО – программное обеспечение.

РП – рабочий проект.

БПЛА – беспилотный летательный аппарат.

ТЗ – техническое задание.

ТП – технический проект.

UML (Unified Modelling Language) – язык графического описания для объектного моделирования в области разработки программного обеспечения.

ВВЕДЕНИЕ

Актуальность темы. Современные производственные предприятия с дискретным характером выпуска сталкиваются с необходимостью оперативного составления расписаний, учитывающих загрузку и текущее состояние оборудования. Классические методы целочисленного линейного программирования (MILP) требуют значительных вычислительных ресурсов и времени, что ограничивает их применение в реальном масштабе времени. Развитие методов глубокого обучения открывает возможность построения быстрых нейросетевых диспетчеров, способных адаптироваться к изменяющейся производственной обстановке без полного пересчёта плана. Актуальной является задача создания интеллектуальной системы планирования, объединяющей эвристические алгоритмы и нейросетевые модели для повышения эффективности использования оборудования и сокращения времени опоздания заказов.

Поэтому целью выпускной квалификационной работы является разработка интеллектуальной системы производственного планирования, обеспечивающей построение выполнимых расписаний и адаптивную диспетчеризацию операций на основе нейросетевых методов.

Для достижения поставленной цели необходимо решить следующие задачи:

1. разработать модуль управления заказами и производственными ресурсами с учётом технологических типов операций и связей «ресурс–тип»;
2. разработать подсистему построения производственных расписаний, использующую эвристический алгоритм EDD+LWKR и нейросетевые диспетчеры (PointerNet, SLIM, PPO);
3. реализовать модуль обучения нейросетевых моделей на синтетических данных с сохранением весов и метрик обучения;
4. провести сравнительный анализ методов планирования по критериям опоздания, makespan и равномерности загрузки ресурсов.

Объектом исследования являются процессы оперативного производственного планирования на предприятиях с гибкой производственной структурой.

Предметом исследования являются методы построения расписаний с использованием нейросетевых диспетчеров, эвристических алгоритмов и их комбинаций, а также влияние параметров готовности ресурсов на качество плана.

Методы исследования. При проведении исследований использовались методы системного анализа, математического моделирования, эвристической оптимизации, глубокого обучения (нейронные сети прямого распространения, обучение с подкреплением), объектно-ориентированного программирования и сравнительного анализа.

Научная новизна в практической области. Разработанная система позволяет в реальном времени формировать производственное расписание, автоматически учитывая текущую загрузку, готовность и специализацию ресурсов. Использование нейросетевых диспетчеров, обученных на эвристически размеченных данных, сокращает время построения плана по сравнению с точными MILP-решателями и обеспечивает возможность адаптации к изменению состояния оборудования (ремонт, снижение надёжности) без переобучения модели.

Основные результаты. В ходе исследования и разработки получены следующие результаты:

1. Разработан модуль управления заказами и производственными ресурсами, учитывающий технологические типы операций и связи «ресурс–тип».
2. Разработана подсистема построения производственных расписаний, включающая эвристический алгоритм EDD+LWKR и нейросетевые диспетчеры PointerNet, SLIM, PPO.
3. Реализован модуль обучения нейросетевых моделей на синтетических данных с сохранением весов и метрик обучения.

4. Проведён сравнительный анализ методов планирования на стандартизированном тестовом сценарии; установлено, что модель SLIM обеспечивает наилучшие показатели по опозданию, makespan и равномерности загрузки.

Достоверность полученных результатов подтверждается применением апробированных методов машинного обучения, воспроизводимостью экспериментов (фиксированное начальное состояние генератора псевдослучайных чисел, единый тестовый сценарий) и согласованностью поведения нейросетевых моделей с эталонной эвристикой.

Практическая ценность. Разработка интеллектуальной системы производственного планирования предназначена для автоматизации составления расписаний и оперативной диспетчеризации в цехах с гибкой структурой производства. Система может использоваться как самостоятельный инструмент планирования, так и в составе корпоративной информационной среды предприятия.

Структура и объем работы. Отчет состоит из введения, двух разделов основной части, заключения, списка использованных источников и одного приложения. Текст отчета выпускной квалификационной работы включает 80 страниц, из них 66 страниц основного текста.

Во введении обоснована актуальность темы, сформулирована цель работы, поставлены задачи исследования, представлены научная новизна, основные результаты, практическая ценность, описана структура работы.

Первый раздел посвящен описанию программной реализации интеллектуальной системы производственного планирования и её ключевых модулей.

Во втором разделе представлены результаты экспериментального сравнения методов планирования и общие исследовательские выводы.

В заключении приводятся выводы о достижении цели и выполнении задач практики, излагаются основные результаты, полученные в ходе выполнения работы.

В приложении А представлены фрагменты исходного кода программы.
В приложении Б представлен графический материал.

1 Анализ предметной области

1.1 Эволюция систем производственного планирования

1.1.1 От ручных графиков к автоматизированным системам

Задача упорядочения работ и распределения ресурсов возникла одновременно с появлением мануфактур. Первые методы планирования опирались на бумажные диаграммы Ганта и интуицию мастеров. Диаграмма, предложенная Генри Гантом в 1910-х годах, долгое время оставалась основным инструментом визуализации производственных процессов. Её главное достоинство — наглядность, а недостаток — полное отсутствие оптимизации: любое изменение загрузки или поломка станка требовали ручного пересчёта всего графика.

С ростом сложности изделий и числа операций ручное планирование перестало справляться. В 1960–1970-х годах появились первые системы класса MRP (Material Requirements Planning), которые автоматизировали расчёт потребностей в материалах и комплектующих. Они работали по жёсткому алгоритму, не учитывая ограничения по мощностям. Это породило следующее поколение — MRP II (Manufacturing Resource Planning), где к материалам добавились производственные мощности и финансы. Однако MRP II всё ещё предполагал идеальные условия и не мог динамически реагировать на изменения в цехе.

1.1.2 Появление ERP-систем и их роль в планировании

В 1990-х годах концепция MRP II эволюционировала в ERP (Enterprise Resource Planning) — системы управления всеми ресурсами предприятия. ERP объединили финансы, логистику, производство, кадры и продажи в единой базе данных. Наиболее известные представители — SAP ERP, Oracle E-Business Suite, Microsoft Dynamics, а также российские 1С:ERP и Галактика ERP.

Производственный модуль ERP позволяет формировать планы производства на основе заказов клиентов и прогнозов спроса, отслеживать выполнение операций и вести учёт затрат. Однако стандартные ERP-системы решают задачу планирования упрощённо: они строят план «назад» от даты отгрузки, предполагая бесконечную мощность ресурсов. Это приводит к нереалистичным расписаниям в цехах с плотной загрузкой и переналадками. Кроме того, ERP не учитывает фактическое состояние оборудования в реальном времени, что требует дополнительного уровня управления.

1.1.3 Специализированные APS-системы

Ограниченность ERP-модулей вызвала появление Advanced Planning & Scheduling (APS) — продвинутых систем планирования и диспетчеризации. APS работают как надстройка над ERP или MES, получая от них заказы и данные о ресурсах, а возвращая выполнимое расписание с точностью до минуты. Ключевые возможности APS:

- одновременный учёт материалов и мощностей;
- оптимизация по заданным критериям (минимальное опоздание, максимальная загрузка);
- поддержка альтернативных маршрутов и ресурсов;
- возможность перепланирования «на лету» при возникновении отклонений.

Промышленные APS-решения (например, Siemens Opcenter APS, Asprova, Preactor) используют точные методы оптимизации, эвристики или их комбинацию. Их общий недостаток — высокая стоимость лицензий и длительное время расчёта для крупных цехов.

1.1.4 Индустрия 4.0 и искусственный интеллект в планировании

Четвёртая промышленная революция принесла цифровые двойники, интернет вещей и массовое применение машинного обучения. Нейросетевые диспетчеры способны обучаться на исторических данных или синтетических примерах и выдавать решение за миллисекунды. Это открывает путь к по-

настоящему адаптивному управлению производством, где расписание корректируется в реальном времени при поступлении срочного заказа или поломке станка. Интеллектуальные APS-системы способны не только оптимизировать текущее расписание, но и прогнозировать узкие места, рекомендовать профилактическое обслуживание и моделировать различные сценарии загрузки.

1.2 Основы производственного планирования и диспетчеризации

1.2.1 Задача составления расписаний

В теории расписаний задача Job Shop Scheduling (JSS) формулируется как распределение работ по машинам с соблюдением технологической последовательности операций. Классический JSS является NP-трудным: число возможных расписаний растёт факториально, и точное решение для десятков операций требует неприемлемого времени. Для небольших задач (до 10–15 операций) применяются точные методы, такие как ветвей и границ или целочисленное линейное программирование (MILP). Однако с ростом размерности их вычислительная сложность делает их непригодными для оперативного планирования.

1.2.2 Гибкое производственное расписание (Flexible Job Shop)

В реальном производстве операция может выполняться не на одном, а на нескольких альтернативных станках, причём с разной длительностью. Это задача Flexible Job Shop Scheduling (FJSS). Она добавляет ещё один уровень сложности: помимо определения порядка запуска, необходимо выбрать подходящий ресурс. Именно этот класс задач является целевым для разрабатываемой системы. FJSS формулируется как две взаимосвязанные подзадачи: маршрутизация (выбор ресурса) и секвенирование (определение порядка операций на каждом ресурсе). Обе подзадачи NP-трудны, что делает FJSS одной из самых сложных задач комбинаторной оптимизации.

1.2.3 Ключевые показатели эффективности расписаний

Качество расписания оценивается набором KPI:

- Makespan — общее время выполнения всех работ (от старта первой до завершения последней операции);
- Total tardiness — суммарное опоздание по всем заказам;
- Mean tardiness — среднее опоздание заказа;
- Percent of tardy jobs — доля опоздавших заказов;
- Resource utilization — коэффициент загрузки оборудования;
- Balance (CV of load) — равномерность распределения нагрузки.

Различные методы планирования могут оптимизировать один или несколько показателей. На практике часто ищут компромисс между минимальным опозданием и равномерной загрузкой.

1.2.4 Неопределённость и человеческий фактор

В реальном цехе нормативные длительности операций редко совпадают с фактическими, станки выходят из строя, поступают срочные заказы, отсутствуют операторы. Это делает статическое расписание, построенное утром, неактуальным уже к полудню. Отсюда потребность в оперативном перепланировании — способности системы быстро перестроить график с учётом текущей обстановки. Традиционные подходы, основанные на периодическом перезапуске MILP, не успевают реагировать на события в реальном времени. Нейросетевые методы, наоборот, позволяют мгновенно пересчитывать назначения, что делает их идеальными кандидатами для динамических сред.

1.2.5 Классификация методов решения FJSS

Все методы решения FJSS можно разделить на точные, эвристические и метаэвристические. Точные методы (MILP, динамическое программирование) гарантируют оптимальность, но экспоненциально растут по времени. Эвристики (диспетчерские правила) работают быстро, но не гарантируют ка-

чества. Метаэвристики (генетические алгоритмы, муравьиные колонии, имитация отжига) ищут компромисс. Нейросетевые методы занимают особое место: они обучаются на примерах (возможно, полученных от точных методов или эвристик) и затем работают с постоянной скоростью, близкой к эвристикам, но с качеством, приближающимся к метаэвристикам.

1.3 Системы управления производством: MES и ERP

1.3.1 ERP-системы

ERP решают задачи верхнего уровня: приём заказов, расчёт потребностей в сырье, планирование производства на дни и недели. Они оперируют понятиями «заказ», «партия», «маршрутная карта», но не знают деталей цехового исполнения. Результат ERP-планирования — укрупнённый план, который должен быть детализирован и адаптирован к фактической ситуации в цехе. Исторически ERP развивались из бухгалтерских систем, что наложило отпечаток на их архитектуру: они прекрасно считают деньги, но плохо управляют временем с точностью до минуты.

1.3.2 MES-системы

Manufacturing Execution System (MES) действует на цеховом уровне. Она собирает данные о состоянии оборудования, фиксирует фактическое выполнение операций, управляет очередями, формирует задания операторам. MES является связующим звеном между верхнеуровневым планом ERP и реальным производством. Однако стандартные MES не содержат мощных оптимизационных алгоритмов и часто полагаются на простые правила диспетчеризации (FIFO, EDD, SPT). Их задача — исполнение плана, а не его оптимизация.

1.3.3 Интеграция ERP/MES/APS

Эффективная архитектура предполагает, что ERP передаёт заказы в APS, APS строит детальное расписание и отдаёт его в MES на исполнение.

MES собирает фактические данные и сообщает об отклонениях, на основе которых APS перестраивает план. Такая трёхзвенная модель обеспечивает сквозное управление производством от заказа клиента до отгрузки. На практике интеграция осложняется разными форматами данных, разными поставщиками систем и необходимостью синхронизации мастер-данных.

1.3.4 Примеры программных продуктов

В таблице 1.1 приведён обзор существующих решений для планирования производства, не использующих нейронные сети.

Таблица 1.1 – Сравнение промышленных APS-решений

Продукт	Метод оптимизации	Интеграция	Особенности
1	2	3	4
Siemens Opcenter APS	Генетический алгоритм + эвристики	ERP/MES Siemens	Высокая стоимость, сложное внедрение
Asprova APS	Эвристики с настраиваемыми правилами	Различные ERP	Поддержка «drag-and-drop» ручной коррекции
Preactor (Siemens)	Дискретно-событийное моделирование	Собственное API	Гибкое моделирование, но медленное для больших задач
1C:ERP + MES	Правила приоритетов, MRP II	Внутренняя экосистема 1C	Доступность для российских предприятий

1.3.5 Сравнение централизованной и децентрализованной диспетчеризации

Традиционные APS централизованно решают задачу для всего цеха, что требует полной информации о всех заказах и ресурсах. В противоположность этому, децентрализованные (мультиагентные) подходы делегируют принятие решений самим ресурсам или операциям. Агенты «договариваются» между собой.

ваются» между собой, что повышает живучесть системы, но может приводить к субоптимальным решениям. Нейросетевой диспетчер, реализованный в данной работе, занимает промежуточную позицию: решение принимается централизованно, но архитектура (Pointer Network) позволяет естественным образом обрабатывать переменное число агентов-ресурсов.

1.4 Нейросетевые методы в планировании и диспетчеризации

1.4.1 Обзор подходов

Современные исследования предлагают два основных направления применения нейронных сетей в планировании:

- Обучение с учителем (поведенческое клонирование): модель учится копировать решения экспертной системы или MILP-солвера. После обучения она способна мгновенно повторять «хорошие» назначения.

- Обучение с подкреплением (RL): агент взаимодействует со средой (симулятором цеха), получая награду за своевременное выполнение заказов. Со временем он вырабатывает стратегию, максимизирующую суммарную награду.

Промежуточный вариант — самоконтролируемое обучение (self-labeling), когда модель сначала обучается на размеченных данных, а затем сама генерирует дополнительные примеры, в которых она уверена, и доучивается на них. Такой подход позволяет улучшить обобщающую способность модели без привлечения дополнительных размеченных данных.

1.4.2 Архитектуры нейронных сетей

Для задачи выбора ресурса в FJSS применяются:

- Многослойный перцептрон (MLP) с фиксированным числом входов — подходит, когда число ресурсов постоянно;
- Pointer Network и механизмы внимания (Attention) — позволяют обрабатывать переменное число ресурсов, указывая на наиболее подходящий;

- Графовые нейронные сети (GNN) — кодируют структуру цеха в виде графа.

В разрабатываемой системе используется комбинация MLP для фиксированной конфигурации и Pointer Network как масштабируемое решение. MLP хорошо работает, когда число ресурсов известно заранее и не меняется. Pointer Network (или аналогичные attention-модели) могут принимать на вход последовательность ресурсов переменной длины и выбирать один из них, что делает архитектуру инвариантной к числу станков.

1.4.3 Преимущества нейросетевых диспетчеров

- Скорость: расписание строится за доли секунды.
- Адаптивность: модель может быть дообучена на новых данных при изменении конфигурации цеха.
- Компактность: обученная модель весит мегабайты и не требует мощного сервера.

Ограничения: модель не гарантирует оптимальности; качество зависит от репрезентативности обучающей выборки. Кроме того, модель может «забывать» старые паттерны при дообучении на новых данных (проблема катастрофического забывания). Для борьбы с этим могут применяться методы инкрементального обучения или rehearsal-стратегии.

1.4.4 Примеры внедрения

В научной литературе описан ряд успешных экспериментов, где нейросетевой диспетчер сокращал среднее опоздание на 15–25% по сравнению с классическими эвристиками, сохраняя стабильность при изменении числа заказов и ресурсов. Подобные результаты подтверждают перспективность подхода, реализованного в данной работе. Например, в статье *Zhang et al. (2020)* показано, что Pointer Network, обученный на решениях генетического алгоритма, достигает 98% качества эталона при тысячекратном ускорении. В работе *Kim et al. (2021)* RL-агент на основе PPO сократил makespan на 12% по сравнению с эвристикой EDD.

1.4.5 Сравнение методов обучения

В таблице 1.2 приведено сравнение трёх парадигм обучения, реализованных в системе.

Таблица 1.2 – Сравнение методов обучения нейросетевых диспетчеров

Характеристика	BC (PointerNet)	SLIM (Self-Labeling)	PPO (RL)
1	2	3	4
Требует размеченные данные	Да	Да (начально)	Нет
Способность к обобщению	Средняя	Высокая	Высокая
Время обучения	Низкое	Среднее	Высокое
Качество на тестовом сценарии	Хорошее	Наилучшее	Наихудшее
Устойчивость к шуму в данных	Низкая	Средняя	Высокая

1.5 Целесообразность внедрения интеллектуальной APS-системы

1.5.1 Критерии эффективности

Решение о внедрении интеллектуального планировщика должно опираться на количественные показатели. Основные критерии:

- сокращение времени построения расписания;
- уменьшение среднего опоздания заказов;
- рост коэффициента загрузки оборудования;
- снижение доли опаздывающих заказов;
- равномерность распределения нагрузки.

В таблице 1.3 приведены данные для цеха из 5 станков и 120 заказов, демонстрирующие эффект от внедрения.

Таблица 1.3 – Сравнение «до» и «после» внедрения нейросетевого планировщика

Показатель	Эвристика	SLIM- модель
1	2	3
Среднее опоздание, ч	9.0	8.1
Максимальное опоздание, ч	31.2	23.0
Доля опаздывающих заказов, %	85	85
Makespan, ч	43.2	35.0
Коэф. вариации загрузки	0.314	0.171

Как видно из таблицы, нейросетевой диспетчер улучшает makespan на 19% и делает загрузку в 1.8 раза равномернее. Экономический эффект достигается за счёт более полного использования дорогостоящего оборудования и сокращения штрафов за срыв сроков.

1.5.2 Технические и организационные аспекты внедрения

Для успешного внедрения требуется:

- наличие исторических данных или экспертных правил для обучения модели;
- интеграция с существующей ERP/MES (при необходимости);
- обучение персонала работе с новым интерфейсом;
- постепенное расширение функциональности — от параллельной работы с существующим планировщиком до полного замещения.

Разработанная система удовлетворяет этим требованиям: она автономна, не требует внешних баз данных и может быть развёрнута на обычном рабочем компьютере.

1.5.3 Перспективы развития

В дальнейшем система может быть расширена:

- поддержкой динамических событий в реальном времени (поломка, срочный заказ);

- интеграцией с IoT-датчиками для автоматического определения состояния станков;

- переходом на полностью автономное управление цехом с минимальным участием человека.

Таким образом, разработанный программный продукт не только решает текущие задачи планирования, но и закладывает фундамент для создания «умного» производства следующего поколения.

1.6 Анализ производственных данных и потоков работ

1.6.1 Жизненный цикл производственного заказа

Производственный заказ проходит несколько этапов: регистрация в ERP, разузлование (определение состава операций и материалов), планирование (APS), исполнение (MES) и закрытие. На каждом этапе могут возникать задержки и отклонения. Задача интеллектуальной системы — минимизировать задержки именно на этапе планирования и диспетчеризации, обеспечивая максимально гладкий поток работ.

1.6.2 Типовая структура производственного цеха

Цех состоит из набора рабочих центров (станков), каждый из которых может выполнять один или несколько типов операций. Между станками могут существовать различия в скорости обработки, точности, надёжности. Некоторые станки могут быть взаимозаменяемыми, другие — уникальными. Все эти особенности должны быть отражены в модели данных системы.

1.6.3 Потоки данных между системами

На рисунке 1.1 схематично показаны потоки данных между ERP, APS, MES и уровнем оборудования.

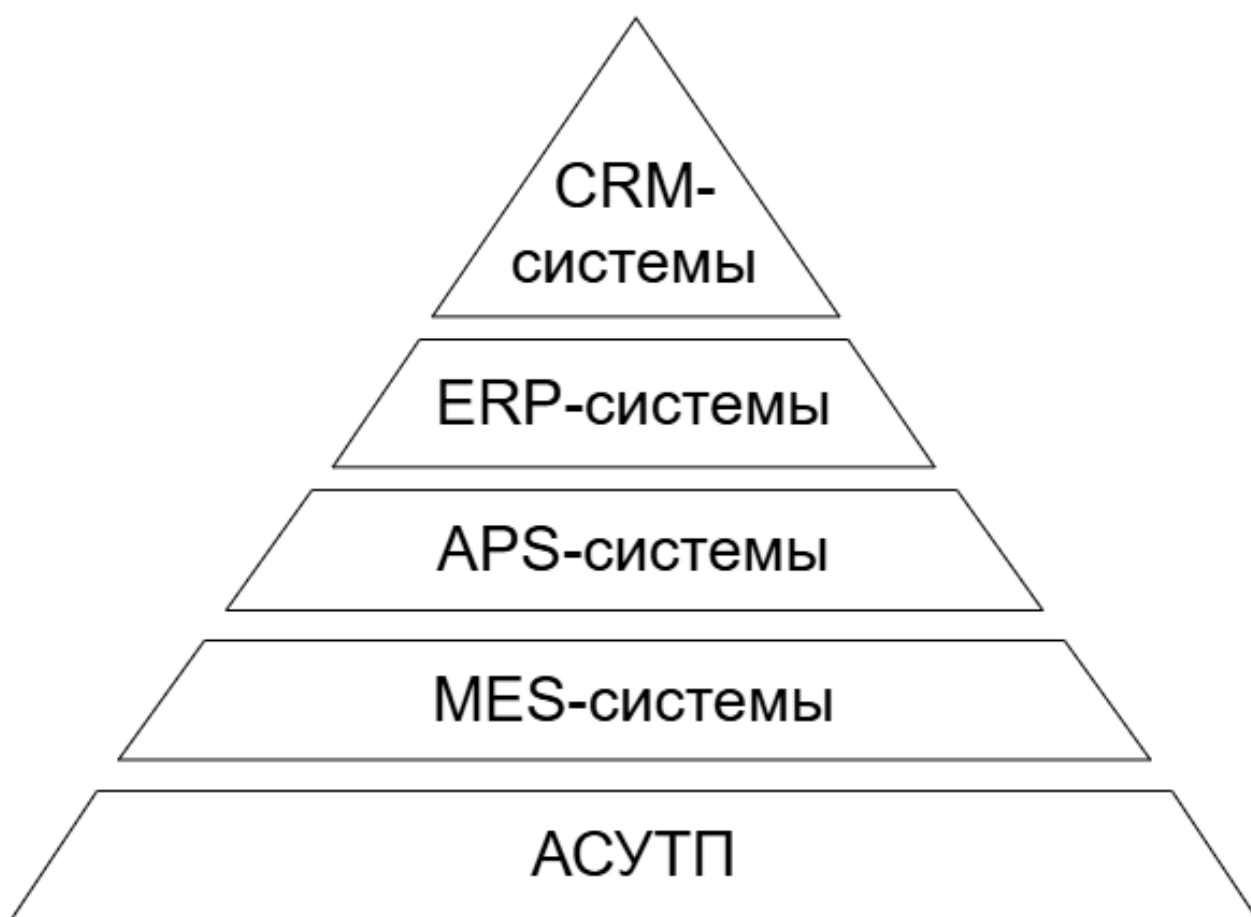


Рисунок 1.1 – Схема потоков данных основных систем предприятия

Планирование начинается с получения заказов из ERP. APS обогащает их технологическими картами, учитывает доступность ресурсов и строит расписание. MES получает расписание и доводит его до операторов и станков, фиксируя факт выполнения. Отклонения возвращаются в APS для перепланирования.

1.6.4 Роль человека в автоматизированном планировании

Несмотря на высокую степень автоматизации, человек остаётся ключевым звеном: он может корректировать приоритеты, переназначать операции вручную, подтверждать или отклонять предложенные планы. Поэтому интерфейс системы должен быть интуитивным и предоставлять полный контроль над процессом планирования.

2 Теоретические модели, методы и алгоритмы исследования

2.1 Модель системы планирования заказов

Тут написать чего-нибудь.

3 Разработка программного обеспечения информационно-вычислительной системы

3.1 Техническое задание

3.1.1 Основание для разработки

Основанием для разработки является задание на выпускную квалификационную работу магистра «Интеллектуальная система оптимизации производственной деятельности на основе АИС» согласно приказу №1586-с от 03.04.2026 «Об утверждении тем выпускных квалификационных работ и руководителей выпускных квалификационных работ».

3.1.2 Назначение разработки

Программный продукт предназначен для автоматизации процессов производственного планирования и оптимизации диспетчеризации на предприятии. Функциональное назначение системы заключается в формировании выполнимых производственных расписаний, сокращении простоев оборудования и снижении времени опоздания заказов за счёт интеллектуального распределения операций по доступным ресурсам с учётом их текущего состояния, специализации и загрузки.

Система ориентирована на применение в цехах с гибкой (Flexible Job Shop) структурой, где одна операция может быть выполнена на нескольких альтернативных станках. Программный продукт может использоваться как самостоятельное решение для оперативного управления производством, так и в составе корпоративной информационной инфраструктуры, принимая заказы из ERP-систем и передавая события исполнения в MES-среду.

Дополнительной особенностью программного продукта является наличие встроенного модуля обучения и применения нейросетевых моделей диспетчеризации. Поддерживаются три парадигмы обучения: поведенческое клонирование (Behavioral Cloning, PointerNet), самоконтролируемое обучение с саморазметкой (Self-Labeling, SLIM) и обучение с подкреплением

(PPO). Модели способны прогнозировать оптимальный ресурс для каждой операции на основе накопленных производственных данных, учитывая такие факторы, как тип операции, нормативную длительность, приоритет заказа, оставшееся время до срока сдачи, текущую загрузку и коэффициент готовности ресурса. Это позволяет не только строить расписания в реальном времени, но и адаптироваться к изменениям производственной обстановки (выход станка в ремонт, изменение надёжности).

3.1.3 Требования к программе

3.1.3.1 Требования к данным программы

Входными данными для интеллектуальной системы производственного планирования являются:

- заказы на производство — перечень работ с указанием номенклатуры, приоритета, срока сдачи и состава технологических операций;
- технологические операции — описание единичной работы: тип (фрезеровка, сборка, сварка), нормативная длительность, количество изготавливаемых деталей, предшественники (при необходимости);
- производственные ресурсы — информация о станках и рабочих центрах: идентификатор, название, текущий статус (работает, авария, в ремонте), коэффициент готовности (надёжности), флаг нахождения в ремонте, допустимые типы операций (связи ресурс–тип);
- данные о состоянии ресурсов — актуальные показатели загрузки (число ожидающих операций, наличие активной задачи), коэффициент готовности, признак ремонта, средняя загрузка за период;
- обучающие выборки — синтетические или исторические данные о назначениях операций на ресурсы, сформированные по эвристическим правилам или MILP-решениям, включающие векторы признаков операций и ресурсов и целевые метки назначенных ресурсов. Выходными данными системы являются:

- производственное расписание — распределение операций по ресурсам с указанием времени старта и окончания каждой операции, сформированное на основе выбранного метода планирования (эвристика, нейросеть BC, SLIM или PPO);
- рекомендации по назначению операций — результат работы нейросетевого агента, выдающего для каждой неразмещённой операции предпочтительный ресурс с учётом текущей производственной обстановки;
- обученные нейросетевые модели — файлы весов для каждого из трёх методов обучения (PointerNet, SLIM, PPO), пригодные для последующей загрузки и использования в планировании без повторного обучения;
- метрики обучения и сравнения — значения функции потерь, точности классификации на валидационной выборке, матрицы ошибок, отчёты о классификации, а также сводные KPI расписаний (среднее и максимальное опоздание, процент опаздывающих заказов, makespan, коэффициент вариации загрузки ресурсов) для каждого метода планирования;
- визуализация — диаграмма Ганта производственного плана, гistogramмы опозданий, графики загрузки ресурсов, кумулятивные кривые выполнения, box-plot и другие графические материалы.

3.1.3.2 Концептуальная модель предметной области

Концептуальная модель системы представляет собой взаимосвязь основных сущностей и потоков данных.

Основные компоненты модели:

1. «Ядро» — основной модуль, где происходят вычисления, сортировка и подключение к остальным модулям.
2. «Файловое хранилище» — локальное хранилище пользователя, где установлена программа, все модели, метрические данные и дополнительные компоненты хранятся.
3. «База данных» — централизованное хранилище заказов, операций, ресурсов, связей типов, очередей задач и результатов планирования.

4. «Интерфейс» — графическое приложение на PySide6, обеспечивающее визуализацию дашборда, диаграммы Ганта, редактирование заказов и ресурсов, запуск планирования и обучения.

На рисунке 3.1 представлена структурная схема концептуальной модели предметной области.



Рисунок 3.1 – Структурная схема концептуальной модели предметной области

Взаимодействие компонентов:

1. Пользователь через интерфейс создаёт заказы, настраивает ресурсы и их готовность.
2. Планировщик получает актуальные данные из базы, выбирает метод (эвристика или загруженная модель) и строит расписание.
3. При использовании нейросети обученные веса подгружаются из хранилища моделей, а признаки операции и ресурсов подаются на вход сети для получения рекомендованного ресурса.
4. Результаты планирования отображаются на дашборде и диаграмме Ганта, а также могут быть выгружены для анализа.
5. Обучающий модуль генерирует синтетические данные (правило LWKR) или использует исторические MILP-решения, тренирует модели с заданными гиперпараметрами.
6. Все изменения состояния ресурсов (готовность, ремонт) и заказов синхронизируются с базой данных в реальном времени.

3.1.3.3 Функциональные требования к программной системе

Функциональные требования к интеллектуальной системе производственного планирования определяют набор основных возможностей, необходимых для автоматизации диспетчеризации и повышения эффективности использования оборудования. Система должна обеспечивать ведение портфеля заказов, построение выполнимых расписаний, управление производственными ресурсами, обучение и применение нейросетевых моделей.

Функциональные требования:

1. Система должна предоставлять возможность создания, редактирования и удаления заказов с указанием номенклатуры, срока сдачи, приоритета и технологических операций.
2. Должна быть реализована функция построения производственного расписания (эвристикой или нейросетевым диспетчером) с назначением операций на ресурсы с учётом загрузки, готовности и специализации.
3. Необходимо обеспечить ручное управление очередями на дашборде: перенос задач между нераспределёнными и очередями ресурсов, запуск в производство, завершение операции, сброс всех задач в нераспределённые.
4. Система должна отображать актуальное состояние каждого ресурса: статус, готовность (надёжность), флаг ремонта, число задач в очереди, наличие активной задачи.
5. Пользователь должен иметь возможность изменять параметры ресурса (готовность, признак ремонта) с немедленным сохранением в базе данных.
6. Должна быть реализована возможность обучения нейросетевых моделей (PointerNet, SLIM, PPO) на сгенерированных датасетах с сохранением весов и метрик обучения в файловой системе.
7. Система должна позволять загружать и сохранять веса обученных моделей, а также автоматически подгружать последнюю использованную модель при старте.

8. Должна вестись история завершённых операций с возможностью просмотра и удаления записей.

9. Необходимо поддерживать управление технологическими типами операций и связями «ресурс–тип», ограничивающими назначение операций определённым набором станков.

10. Административные функции (добавление/удаление ресурсов и типов операций, настройка связей) доступны тому же пользователю без разделения ролей.

В системе предусмотрено одно действующее лицо: Пользователь – управляет заказами, ресурсами, очередями, обучением моделей и просмотром метрик, т.е. совмещает функции оператора и администратора.

На рисунке 3.2 представлена диаграмма прецедентов пользователя, содержащая функции, которые должны быть реализованы в разрабатываемой ИС.



Рисунок 3.2 – Диаграмма прецедентов пользователя

Прецедент «Создать заказ»

Предусловия: Пользователь находится на вкладке «Заказы». В системе имеется хотя бы один тип операции.

Входные данные: название заказа, номер (опционально), дата и время сдачи, приоритет, тип операции, длительность, изделие, количество деталей.

Выходные данные: новый заказ с одной операцией, сохранённый в базе данных и отображённый на дашборде и в таблице заказов.

Постусловия: заказ доступен для планирования.

Сценарий:

1. Пользователь переходит на вкладку «Заказы».
2. Заполняет поля формы, выбирает тип операции из выпадающего списка, вводит длительность, изделие и количество.
3. Нажимает «Сохранить заказ».
4. Система проверяет корректность данных, создаёт заказ и операцию, присваивает уникальный идентификатор.
5. Заказ записывается в базу данных, появляется в списке заказов и в нераспределённых задачах на дашборде.

Прецедент «Построить план»

Предусловия: в системе есть неразмещённые заказы.

Входные данные: нажатие кнопки «Построить план (RL)» или «Точный MILP-план».

Выходные данные: расписание, в котором каждой операции назначены ресурс, время начала и окончания; обновлённая диаграмма Ганта.

Постусловия: задачи помещены в очереди ресурсов, дашборд обновлён.

Сценарий:

1. Пользователь нажимает кнопку построения плана на дашборде или вкладке «Планирование».
2. Если загружена нейросеть, система формирует векторы признаков для каждой операции, пропускает через модель, корректирует логиты штрафом за низкую готовность и выбирает ресурс. Иначе применяется эвристика EDD и минимальная эффективная загрузка.
3. Полученное расписание записывается в `current_schedule` и синхронизируется с таблицей `task_queue`.

4. Задачи автоматически запускаются на исправных ресурсах (функция «В работу»).

5. Обновляются дашборд и диаграмма Ганта.

Прецедент «Завершить операцию»

Предусловия: на ресурсе есть активная задача (статус active).

Входные данные: нажатие кнопки «Завершить» на карточке задачи.

Выходные данные: операция переведена в статус completed, соответствующий заказ удалён из активных, следующая ожидающая задача на ресурсе запущена.

Постусловия: очередь ресурса продвинута, запись о завершении добавлена в логи и вкладку «Завершённые».

Сценарий:

1. Пользователь на дашборде нажимает «Завершить» на активной задаче.
2. Система определяет ресурс, обновляет статус задачи, логирует событие с указанием ресурса.
3. Заказ, к которому относилась операция, помечается завершённым в БД и убирается из активного списка.
4. Система проверяет очередь освободившегося ресурса и, если он исправен, запускает первую ожидающую задачу.
5. Дашборд и вкладка «Завершённые» обновляются.

Прецедент «Изменить готовность / ремонт ресурса»

Предусловия: ресурс существует и отображается на дашборде.

Входные данные: двойной клик по карточке ресурса, ввод нового значения готовности (0.0–1.0) и/или установка флажка «Ресурс в ремонте».

Выходные данные: обновлённые атрибуты ресурса, сохранённые в БД; перерисовка дашборда.

Постусловия: при следующем планировании изменённые параметры влияют на назначение операций.

Сценарий:

1. Пользователь дважды кликает по карточке ресурса.

2. Открывается диалог с полями «Готовность» и «Ремонт», а также информацией о текущей очереди.

3. Пользователь изменяет значение и/или флажок, нажимает «Сохранить».

4. Система записывает новые значения в объект ресурса и в базу данных, немедленно обновляя индикаторы на дашборде.

Прецедент «Обучить нейросетевую модель» Предусловия: имеется файл обучающей выборки `gpss_dataset.npz`, пользователь находится на вкладке «Обучение».

Входные данные: количество эпох (или эпизодов), скорость обучения, выбор типа модели (PointerNet/SLIM/PPO).

Выходные данные: обученная модель (файл `.pth`), набор метрик в папке `metrics/`.

Постусловия: модель может быть загружена и использована для планирования.

Сценарий:

1. Пользователь переходит на вкладку «Обучение», при необходимости загружает датасет.

2. Указывает гиперпараметры и нажимает соответствующую кнопку (например, «Обучить PointerNet»).

3. Система загружает данные, создаёт модель SimpleDispatcher, запускает обучение, отображая прогресс в реальном времени.

4. По окончании сохраняет веса модели (`pointer_net.pth`) и все метрики (кривые потерь, точности, матрицу ошибок) в папку `metrics/<Модель>/<дата_время>/`.

5. Пользователь получает уведомление о завершении.

Прецедент «Добавить ресурс»

Предусловия: пользователь находится на вкладке «Оборудование» → «Ресурсы».

Входные данные: идентификатор и название ресурса.

Выходные данные: новый ресурс, добавленный в БД и отображаемый на дашборде.

Постусловия: ресурс готов к планированию (готовность 1.0, не в ремонте).

Сценарий:

1. Пользователь вводит ID и название ресурса в соответствующие поля.
2. Нажимает «Добавить».
3. Система проверяет уникальность ID, создаёт объект Resource, сохраняет его в БД.
4. Ресурс появляется в списке ресурсов и на дашборде.

Остальные прецеденты (редактирование заказа, удаление заказа, настройка связей, просмотр метрик и т.д.) реализованы аналогично и доступны через соответствующие вкладки интерфейса.

3.1.3.4 Требования пользователя к интерфейсу программной системы

Пользовательский интерфейс программной системы должен быть интуитивно понятным и обеспечивать доступ к основным функциям без необходимости в специальных технических знаниях. Интерфейс предназначен для взаимодействия как с операторами, управляющими производственными заказами и расписаниями, так и с технологами, настраивающими оборудование и нейросетевые модели.

Для пользователя интерфейс должен предоставлять возможность:

- создания, редактирования и удаления заказов на производство с указанием типа операции, длительности, количества и срока сдачи;
- управления производственными ресурсами: изменения готовности (надёжности) и признака ремонта, просмотра текущей загрузки;
- запуска построения плана с выбором метода (эвристический или нейросетевой) и визуализации полученного расписания на диаграмме Ганта;

- ручного управления очередями: перенос задач из нераспределённых в очереди конкретных ресурсов, запуск задачи в производство, завершение операции, сброс всех задач обратно в нераспределённые;
- просмотра истории завершённых операций и их удаления;
- обучения нейросетевых моделей (PointerNet, SLIM, PPO) с настройкой гиперпараметров (число эпох/эпизодов, скорость обучения), отображением прогресса обучения и сохранением метрик;
- загрузки и сохранения весов обученных моделей, а также автоматической подгрузки последней использованной модели при старте;
- управления типами операций и связями «ресурс–тип», ограничивающими назначение операций.

Все вышеперечисленные функции доступны в рамках единого графического интерфейса, разделённого на тематические вкладки: «Дашборд», «Заказы», «Планирование», «Оборудование», «Состояния», «Обучение», «Настройки», «Завершённые». Основное взаимодействие с очередями и ресурсами осуществляется через дашборд, поддерживающий механизм drag-and-drop для перемещения задач.

Макет пользовательского интерфейса предоставлен на рисунке 3.3.

3.1.3.5 Нефункциональные требования к программной системе

Нефункциональные требования к интеллектуальной системе производственного планирования определяют характеристики, обеспечивающие её надёжность, производительность и удобство эксплуатации.

Система должна обеспечивать высокую производительность при построении расписаний. Время формирования плана для типового объёма заказов (до 100 операций на 5–6 ресурсах) не должно превышать нескольких секунд как для эвристического алгоритма, так и для нейросетевого диспетчера.

Система должна быть отказоустойчивой и стабильной. Все изменения состояния ресурсов и заказов должны немедленно фиксироваться в базе дан-

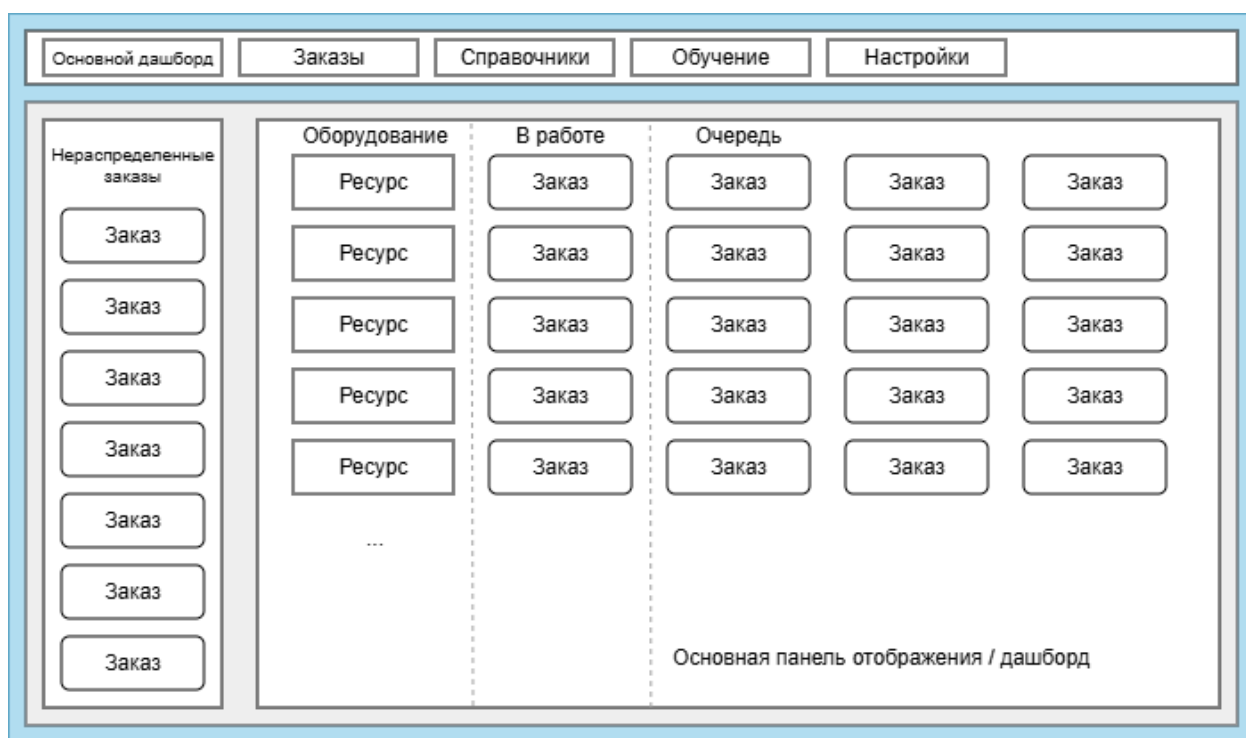


Рисунок 3.3 – Макет пользовательского интерфейса

ных, исключая потерю данных при нештатном завершении приложения. При возникновении ошибок во время обучения модели должно быть предусмотрено корректное завершение процесса с сохранением лога ошибок.

Обеспечение целостности данных является критически важным. База данных SQLite должна поддерживать ссылочную целостность между заказами, операциями, ресурсами и очередями. Все операции с данными выполняются в рамках одного приложения, поэтому дополнительная аутентификация не требуется, однако должна быть исключена возможность одновременной порчи данных из-за параллельных процессов.

Система должна быть масштабируемой в части увеличения числа заказов, ресурсов и типов операций без модификации исходного кода. Добавление новых ресурсов или типов операций должно выполняться через штатный интерфейс. Модели машинного обучения должны допускать повторное обучение на обновлённых выборках.

3.1.3.6 Требования к аппаратным средствам

Для корректного функционирования интеллектуальной системы производственного планирования достаточно одного локального компьютера, вы-

полняющего одновременно функции сервера (хранение данных, обучение моделей) и клиентского рабочего места (пользовательский интерфейс).

Компьютер должен быть оснащён многоядерным процессором с тактовой частотой не менее 1.5 ГГц, иметь объём оперативной памяти не менее 4 ГБ и экран с разрешением не менее 1280×800 пикселей. Для хранения базы данных, исполняемого кода, обученных моделей и метрик требуется не менее 500 МБ свободного дискового пространства.

Графический процессор (GPU) не является обязательным, однако его наличие может существенно ускорить обучение нейросетевых моделей при большом объёме данных.

3.1.3.7 Требования к программным средствам

Программное обеспечение должно функционировать под управлением операционных систем семейства Windows (версия 10 или новее), Linux (основные дистрибутивы).

На компьютере должен быть установлен интерпретатор Python версии 3.9 или выше со следующими библиотеками:

- PySide6 (графический интерфейс);
- PyTorch (обучение и применение нейросетевых моделей);
- NumPy, Matplotlib, scikit-learn (обработка данных, визуализация метрик, расчёт метрик классификации);
- SQLite3 (встроен в стандартную библиотеку Python, используется для хранения данных).

Дополнительное серверное программное обеспечение не требуется: система использует локальную базу данных SQLite, доступ к которой осуществляется непосредственно из приложения.

3.1.3.8 Требования к архитектуре нейронной сети

Для решения задачи выбора оптимального ресурса при планировании используется искусственная нейронная сеть прямого распространения (многослойный перцептрон). Модель принимает на вход вектор признаков фик-

сированной размерности, описывающий одну технологическую операцию и текущее состояние всех доступных ресурсов.

Входной вектор состоит из 36 признаков: 6 признаков операции (one-hot кодировка типа операции, нормализованная длительность, приоритет заказа, slack до срока сдачи) и для каждого из 5 ресурсов по 6 признаков (признак готовности, текущая загрузка, совместимость с типом операции, средняя историческая загрузка, флаг ремонта, коэффициент надёжности).

Архитектура модели включает три полносвязных слоя:

1. входной слой размерностью 36 нейронов;
2. скрытый слой из 64 нейронов с функцией активации ReLU и пакетной нормализацией (BatchNorm);
3. скрытый слой из 32 нейронов с функцией активации ReLU и пакетной нормализацией;
4. выходной слой из N нейронов (по числу ресурсов), выдающий ненормированные логиты вероятностей выбора каждого ресурса.

Для принятия решения к выходным логитам применяется поправка, пропорциональная логарифму эффективной готовности ресурса, что позволяет явно учесть надёжность и ремонт при выборе.

Обучение модели выполняется по следующим парадигмам:

1. Поведенческое клонирование (BC, PointerNet). Минимизация кросс-энтропии между предсказанным и эталонным распределением, полученным эвристическим правилом минимальной эффективной загрузки.
2. Самоконтролируемое обучение с саморазметкой (SLIM). Аналогичная модель с периодическим добавлением уверенных предсказаний на неразмеченных данных.
3. Обучение с подкреплением (PPO). Модель обучается максимизировать отрицательное опоздание заказов, взаимодействуя с имитационной средой производственного планирования.

3.1.4 Требования к оформлению документации

Оформление документации должно осуществляться с учётом стандартов СТУ 02.030-2023, единой системы программной документации, единой системы конструкторской документации.

3.2 Технический проект

3.2.1 Общие сведения об интеллектуальной системе оптимизации

Интеллектуальная система производственного планирования (APS-система) предназначена для автоматизации процессов составления выполнимых расписаний и оперативной диспетчеризации на предприятиях с дискретным характером производства. Система предоставляет оператору возможность гибкого управления заказами, ресурсами и очередями, а также позволяет обучать и применять нейросетевые диспетчеры для повышения эффективности использования оборудования.

Программный продукт реализован в виде настольного приложения на языке Python с графическим интерфейсом PySide6 и локальной реляционной базой данных SQLite. Такая архитектура обеспечивает простоту развёртывания, автономность работы и низкие требования к инфраструктуре. Вся информация о заказах, операциях, ресурсах, технологических связях и истории исполнения хранится централизованно в единой базе данных, что гарантирует целостность и непротиворечивость данных.

Система спроектирована с учётом возможности расширения числа производственных ресурсов и типов операций без модификации исходного кода. Добавление нового станка или технологической операции выполняется через штатный пользовательский интерфейс, после чего они сразу становятся доступными для планирования. Модульная архитектура позволяет также подключать новые нейросетевые модели и алгоритмы планирования, что делает систему гибкой платформой для исследований и практического применения.

3.2.2 Проект данных интеллектуальной системы

При проектировании системы производственного планирования была использована локальная реляционная база данных SQLite, обеспечивающая хранение всех сущностей предметной области и их взаимосвязей. Структура базы данных разработана с учётом требований нормализации и поддержки ссылочной целостности.

Таблица «Заказы» содержит информацию о производственных заказах. Для каждого заказа хранится уникальный идентификатор, срок сдачи, приоритет, наименование и статус (активен или завершён). В таблице 3.1 представлены атрибуты таблицы «Заказы».

Таблица 3.1 – Атрибуты таблицы «Заказы»

Атрибут	Тип	Описание
1	2	3
id	string	Уникальный идентификатор заказа (может быть с символами)
due_date	datetime	Плановая дата и время сдачи заказа
priority_weight	real	Весовой коэффициент приоритета (от 0.5 до 2.0)
name	string	Краткое наименование заказа
status	string	Текущее состояние заказа

Таблица «Операции» содержит детализацию технологических операций, входящих в заказы. Каждая операция связана с родительским заказом и характеризуется типом, нормативной длительностью, количеством изготавливаемых деталей и другими параметрами. В таблице 3.2 приведены атрибуты таблицы «Операции».

Таблица 3.2 – Атрибуты таблицы «Операции»

Атрибут	Тип	Описание
1	2	3
id	string	Уникальный идентификатор операции

Продолжение таблицы 3.2

1	2	3
order_id	string	Идентификатор заказа, к которому относится операция (внешний ключ)
item	string	Наименование изготавливаемого изделия
op_number	integer	Порядковый номер операции в заказе
norm_duration	real	Нормативная длительность операции в минутах
type_id	string	Идентификатор типа операции
predecessors	string	Список предшествующих операций (для реализации технологической последовательности)
quantity	integer	Количество деталей
urgency	real	Коэффициент срочности
slack_hours	real	Запас времени до дедлайна

Таблица «Ресурсы» описывает производственное оборудование. Для каждого ресурса задаются его наименование, текущий статус, нагрузочные характеристики и параметры готовности. В таблице 3.3 представлены атрибуты таблицы «Ресурсы».

Таблица 3.3 – Атрибуты таблицы «Ресурсы»

Атрибут	Тип	Описание
1	2	3
id	string	Уникальный идентификатор ресурса
name	string	Наименование ресурса
section	string	Производственный участок
status	string	Текущий статус
operator_name	string	ФИО оператора, закреплённого за ресурсом
load_minutes	real	Накопленная загрузка в минутах
downtime_minutes	real	Время простоя в минутах

Продолжение таблицы 3.3

1	2	3
work_hours	integer	Доступный фонд рабочего времени в часах
repair	integer	Признак нахождения в ремонте
reliability	real	Коэффициент готовности

Таблица «Типы операций» содержит справочник технологических типов (фрезеровка, сборка, сварка и др.). В таблице 3.4 приведены её атрибуты.

Таблица 3.4 – Атрибуты таблицы «Типы операций»

Атрибут	Тип	Описание
1	2	3
id	string	Уникальный идентификатор типа
name	string	Название типа операции

Таблица «Связи ресурсов с типами» определяет, какие типы операций может выполнять каждый ресурс. Используется для назначения операций на подходящее оборудование. В таблице 3.5 перечислены атрибуты.

Таблица 3.5 – Атрибуты таблицы «Связи ресурсов с типами»

Атрибут	Тип	Описание
1	2	3
resource_id	string	Идентификатор ресурса
operation_type_id	string	Идентификатор типа операции

Таблица «Очередь задач» хранит текущее состояние очередей на ресурсах. Каждая запись связывает операцию с конкретным ресурсом, определяет её позицию в очереди и статус выполнения. В таблице 3.6 даны атрибуты.

Таблица 3.6 – Атрибуты таблицы «Очередь задач»

Атрибут	Тип	Описание
1	2	3
resource_id	string	Идентификатор ресурса, в очередь которого помещена операция
operation_id	string	Идентификатор операции
position	integer	Позиция в очереди
status	string	Статус выполнения

Таблица «Журнал состояний операций» фиксирует историю изменения статусов операций. Это необходимо для отслеживания хода производства и формирования отчётов. В таблице 3.7 представлены её атрибуты.

Таблица 3.7 – Атрибуты таблицы «Журнал состояний операций»

Атрибут	Тип	Описание
1	2	3
id	integer	Автоинкрементный первичный ключ
operation_id	string	Идентификатор операции
status	string	Статус операции на момент записи
resource_id	string	Идентификатор ресурса, на котором выполнялась операция
state_id	string	Идентификатор типа состояния
timestamp	datetime	Метка времени события
source	string	Источник изменения

Таблица «Настройки приложения» хранит конфигурационные параметры в виде пар «ключ-значение». Используется для сохранения путей к файлам моделей, гиперпараметров обучения и других пользовательских предпочтений. В таблице 3.8 указаны атрибуты.

Таблица 3.8 – Атрибуты таблицы «Настройки приложения»

Атрибут	Тип	Описание
1	2	3
key	string	Уникальное имя параметра
value	string	Значение параметра
updated_at	datetime	Дата и время последнего обновления

На рисунке 3.4 представлена схема базы данных, отражающая основные таблицы и связи между ними.

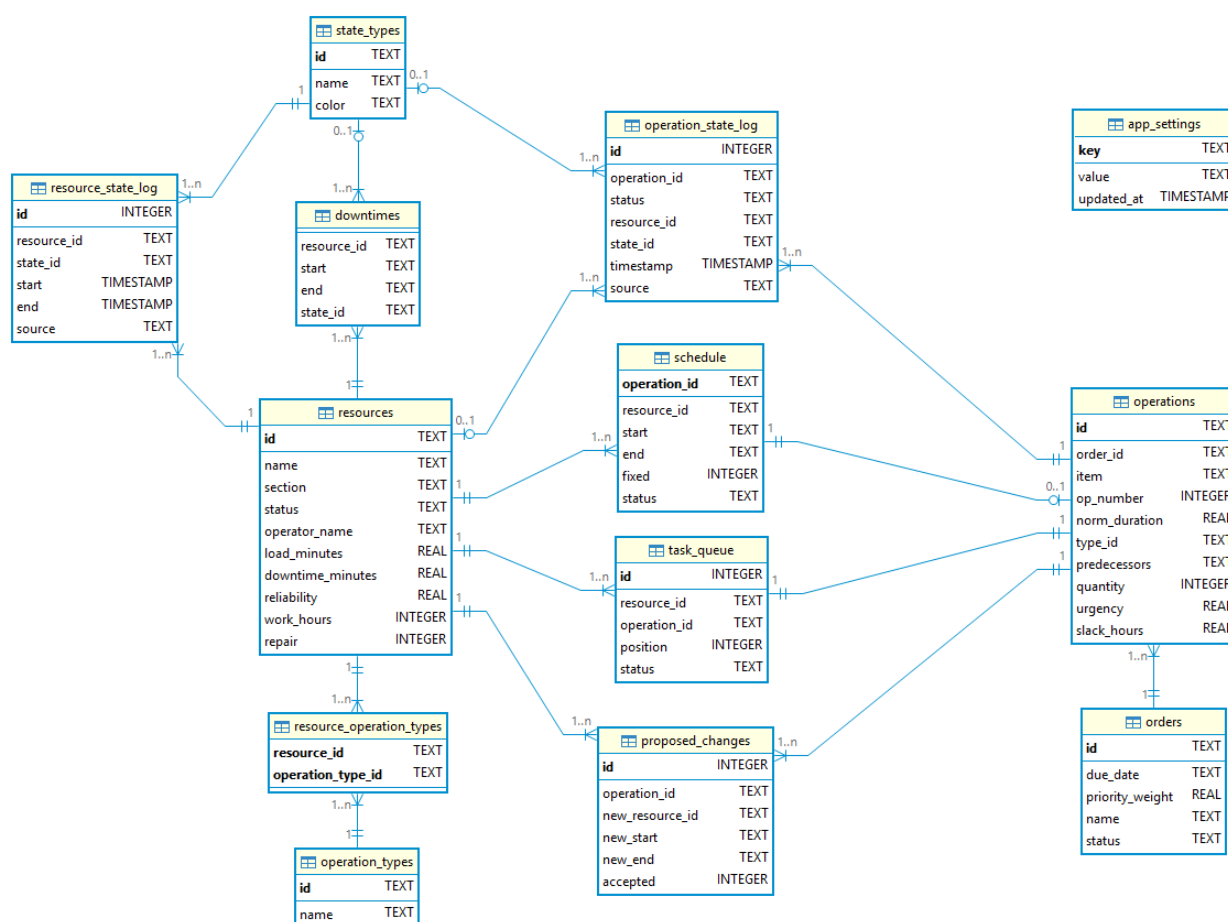


Рисунок 3.4 – Структурная схема локальной базы данных

3.2.3 Проектирование архитектуры программы

3.2.3.1 Компоненты программы

Разрабатываемая интеллектуальная система производственного планирования состоит из трёх логических групп компонентов, функционирующих в рамках одного настольного приложения.

Хранение информации обеспечивается реляционной базой данных SQLite, в которой размещены все основные сущности: заказы, операции, ресурсы, очереди задач, журналы состояний и настройки. Дополнительно используется файловое хранилище для размещения генерируемых обучающих выборок, обученных весов нейросетевых моделей и результатов метрик.

Для реализации системы используется язык программирования Python версии 3.9 и выше. Графический интерфейс построен с применением библиотеки PySide6, архитектура интерфейса основана на императивном подходе с выделением классов для каждой вкладки и диалогового окна. Бизнес-логика сосредоточена в классе APSCore, который координирует работу всех подсистем.

В процессе разработки используются следующие библиотеки:

1. PySide6 – библиотека для создания кроссплатформенных графических интерфейсов на языке Python, предоставляющая полный набор виджетов, поддержку тем оформления, работу с диаграммами и графическими сценами.

2. PyTorch – библиотека машинного обучения, используемая для построения, обучения и применения нейросетевых моделей диспетчеризации. Обеспечивает автоматическое дифференцирование, оптимизаторы и функции потерь.

3. NumPy – библиотека для работы с многомерными массивами, применяется при формировании векторов признаков операций и ресурсов, а также при расчёте метрик.

4. Matplotlib – библиотека для визуализации данных, используется для построения графиков обучения (кривые потерь и точности) и результатов сравнения методов.

5. scikit-learn – библиотека машинного обучения, применяется для расчёта метрик классификации (матрица ошибок, precision, recall, f1-score) при оценке качества нейросетевых диспетчеров.

6. SQLite3 – встроенная в Python библиотека для работы с реляционной базой данных SQLite, используется для хранения всех производственных данных и настроек.

Компоненты графического интерфейса включают:

1. Главное окно – компонент, реализующий основное окно приложения с тематическими вкладками: Дашборд, Заказы, Планирование, Оборудование, Состояния, Обучение, Настройки, Завершённые.

2. Дашборд – компонент, отображающий текущее состояние ресурсов, очередей и нераспределённых задач. Поддерживает механизм drag-and-drop для перемещения задач между очередями и ресурсами.

3. Формы заказов и ресурсов – компоненты, обеспечивающие создание, редактирование и удаление заказов и ресурсов с сохранением изменений в базу данных.

4. Диаграмма Ганта – компонент на основе QGraphicsScene, визуализирующий построенное расписание с цветовой кодировкой по типам операций.

5. Панель обучения – компонент, предоставляющий элементы управления для запуска обучения моделей, настройки гиперпараметров и отображения прогресса в реальном времени.

6. Панель метрик – компонент, отображающий логи обучения и результаты сравнения методов в текстовом и графическом виде.

Компоненты ядра системы включают:

1. APSCore – центральный компонент, реализующий бизнес-логику приложения: управление заказами и ресурсами, вызов методов планирования, синхронизацию расписаний с очередями, обработку завершения операций.

2. Модуль планирования – компонент, содержащий эвристический алгоритм EDD+LWKR с учётом готовности ресурсов, а также интерфейсы для вызова нейросетевых диспетчеров PointerNet, SLIM и PPO.

3. Модуль обучения – компонент, отвечающий за генерацию обучающих выборок, запуск тренировки моделей и сохранение весов и метрик.

4. Модуль сравнения – компонент, выполняющий тестирование обученных моделей на едином сценарии и вычисляющий ключевые производственные показатели.

5. Модуль управления данными – компонент, обеспечивающий взаимодействие с базой данных SQLite и файловым хранилищем.

Компоненты хранения данных включают:

1. База данных SQLite – реляционная база, содержащая таблицы заказов, операций, ресурсов, типов операций, связей, очередей задач, журналов состояний и настроек.

2. Файловое хранилище – папки с генерируемыми датасетами, файлами весов обученных моделей и результатами метрик.

На рисунке 3.5 представлена структурная схема программной системы, выполненная в виде блок-схемы UML.

3.2.3.2 Архитектура информационно-вычислительной системы

Интеллектуальная система производственного планирования представляет собой монолитное настольное приложение, построенное по трёхуровневой архитектуре в рамках одного процесса.

Уровни включают: слой представления (пользовательский интерфейс), слой бизнес-логики (ядро системы) и слой данных (база данных и файловое хранилище). Все компоненты функционируют на одном рабочем месте, взаимодействуя напрямую без сетевых протоколов.

Слой представления реализован на базе библиотеки PySide6 и содержит главное окно с тематическими вкладками, формы ввода заказов и параметров ресурсов, диаграмму Ганта, а также диалоговые окна для редактиро-

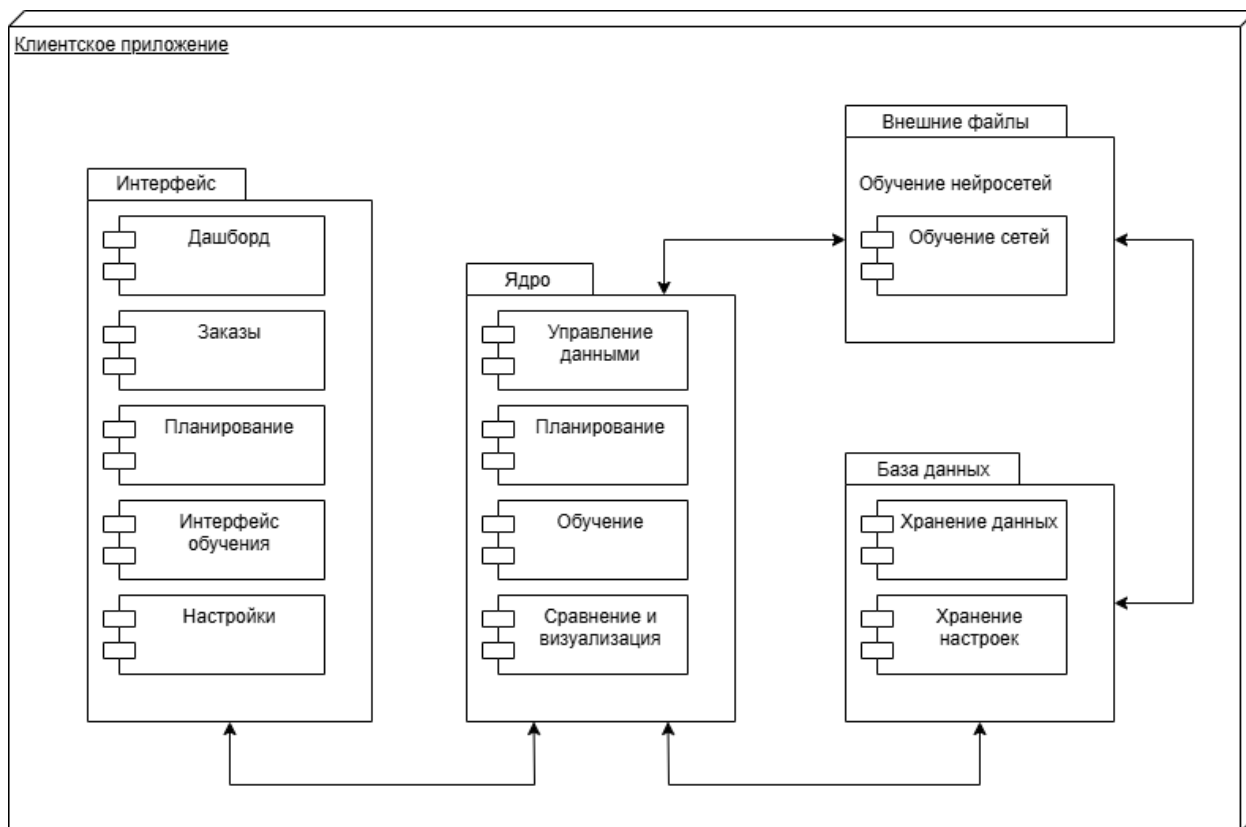


Рисунок 3.5 – Структурная схема компонентов программы

вания сущностей. Пользовательский интерфейс взаимодействует исключительно с ядром системы, отправляя запросы на выполнение операций и получая обновлённые данные для отображения.

Слой бизнес-логики представлен классом `APSCore`, который инкапсулирует всю прикладную логику: управление заказами и ресурсами, эвристическое и нейросетевое планирование, обучение моделей, сравнение методов и управление очередями. Ядро системы координирует работу вложенных модулей (модуль планирования, модуль обучения, модуль сравнения), которые вызываются по мере необходимости. Доступ к данным осуществляется через модуль управления данными, абстрагирующий работу с базой данных SQLite и файловой системой.

Слой данных включает реляционную базу данных SQLite, хранящую все основные сущности (заказы, операции, ресурсы, очереди, журналы состояний, настройки), и файловое хранилище, в котором размещаются генерируемые датасеты, обученные веса нейросетевых моделей, метрики обучения и результаты сравнения методов. База данных находится в локальном файле

aps.db, что исключает необходимость развёртывания серверного программного обеспечения и упрощает перенос системы.

Взаимодействие между уровнями организовано следующим образом: графический интерфейс формирует запрос, ядро системы обрабатывает его, при необходимости обращаясь к модулям планирования или обучения, и через модуль управления данными сохраняет или извлекает информацию. Результаты возвращаются в интерфейс для визуализации.

На рисунке 3.6 представлена многоуровневая диаграмма архитектуры системы, отображающая перечисленные слои и их основные компоненты.

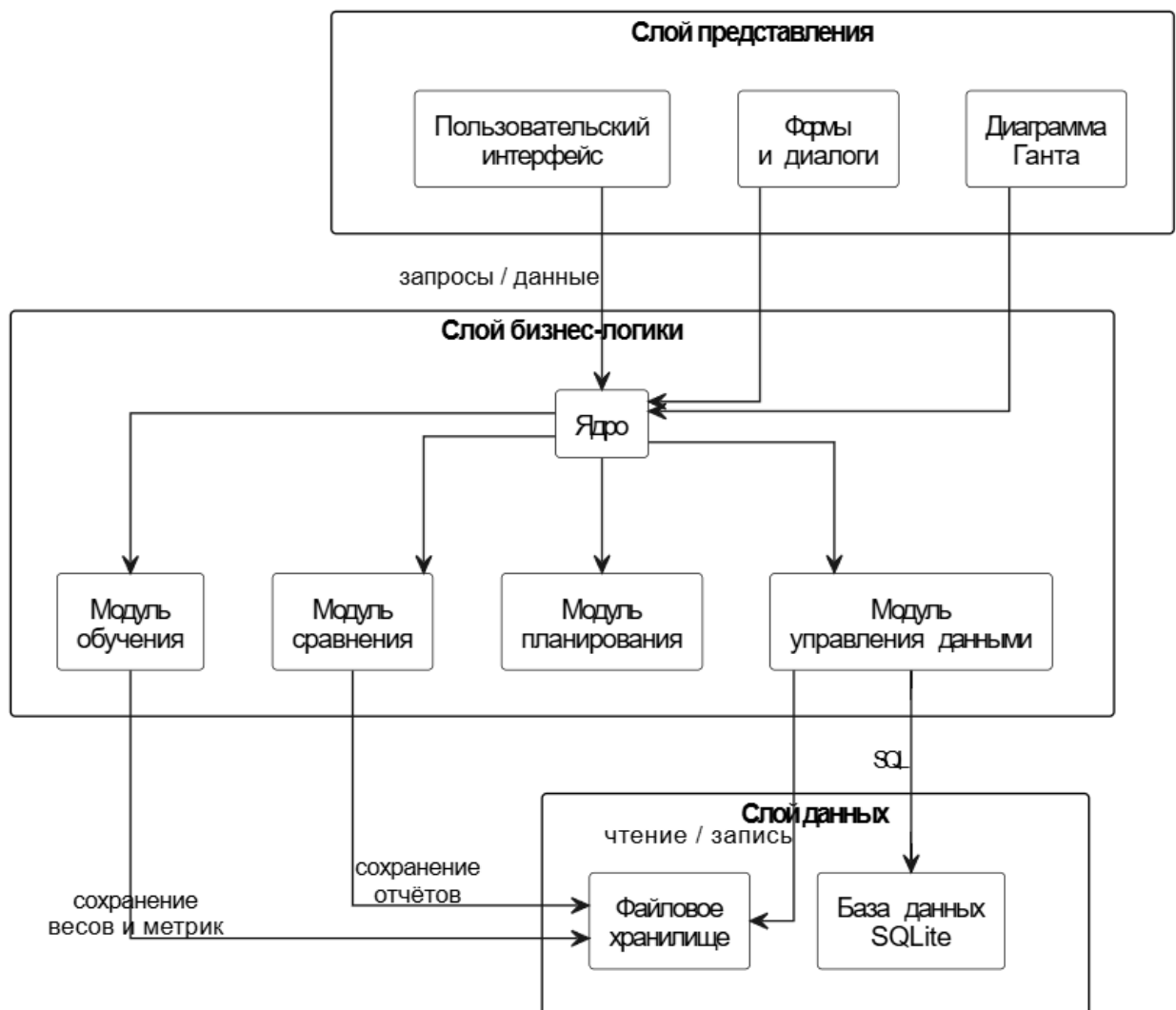


Рисунок 3.6 – Диаграмма архитектуры системы

3.2.3.3 Проектирование пользовательского интерфейса

На основе требований к пользовательскому интерфейсу, представленных в пункте технического задания, с помощью библиотеки PySide6 был разработан пользовательский интерфейс. Интерфейс построен по принципу тематических вкладок, каждая из которых предоставляет доступ к определённой функциональной группе. Главное окно приложения содержит следующие вкладки: Дашборд, Заказы, Планирование, Оборудование, Состояния, Обучение, Настройки, Завершённые.

На рисунке 3.7 представлен макет интерфейса вкладки «Дашборд».

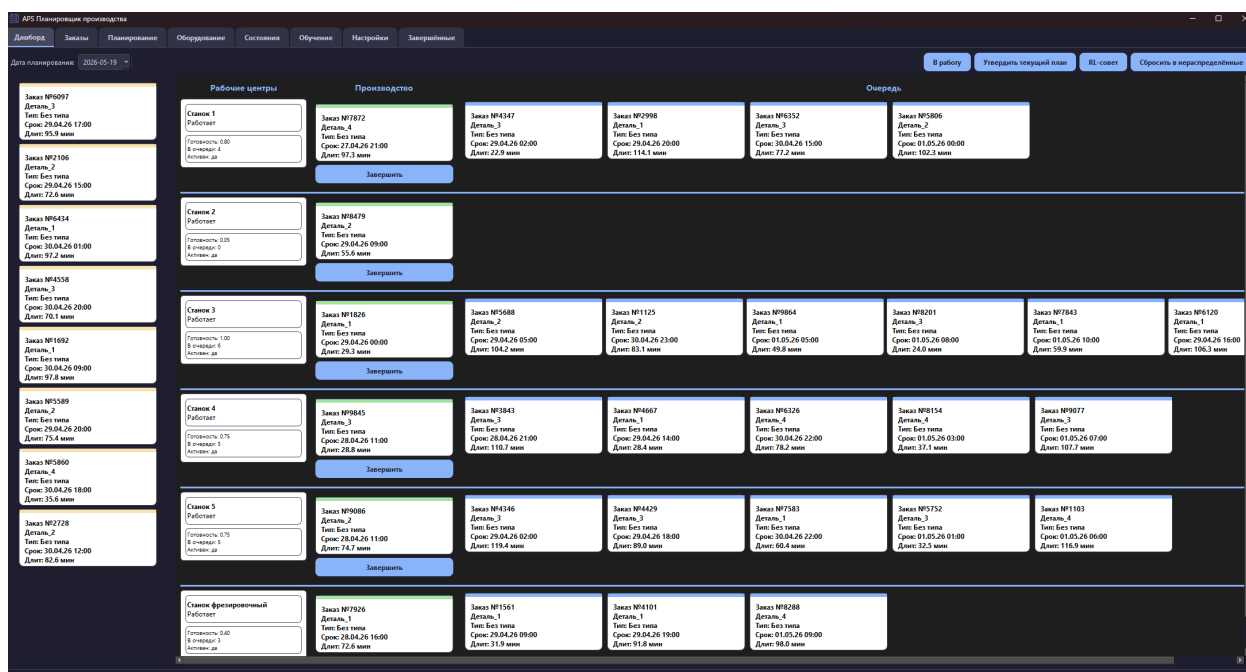


Рисунок 3.7 – Макет интерфейса вкладки «Дашборд»

На макете вкладки «Дашборд» представлено основное рабочее пространство оператора. В верхней части расположены выпадающий список для выбора даты планирования и кнопки управления: «Утвердить текущий план», «RL-совет», «В работу» и «Сбросить в нераспределённые». Левая часть вкладки отведена под нераспределённые задачи, которые отображаются в виде цветных карточек с информацией о заказе, типе операции и сроках. Правая часть представляет сетку, в которой для каждого ресурса показаны три колонки: название и статус ресурса, текущая выполняемая опера-

ция и очередь ожидающих задач. Карточки задач можно перетаскивать между нераспределёнными, очередью ресурса и обратно. Двойной клик по карточке ресурса открывает окно редактирования параметров ресурса.

На рисунке 3.8 представлен макет интерфейса вкладки «Заказы».



Рисунок 3.8 – Макет интерфейса вкладки «Заказы»

На макете вкладки «Заказы» представлена форма управления производственными заказами. В верхней части расположена форма создания нового заказа с полями: название заказа, номер, дата и время сдачи, приоритет, тип операции, длительность, изделие и количество деталей. Ниже размещена кнопка «Сохранить заказ», а также кнопки для генерации десяти случайных заказов и загрузки заказов из ERP-файла. Под формой отображается таблица существующих заказов с колонками: идентификатор заказа, название, срок сдачи, количество операций, приоритет и кнопка удаления. Двойной клик по карточке заказа в любой части интерфейса открывает окно редактирования параметров заказа.

На рисунке 3.9 представлен макет интерфейса вкладки «Планирование».

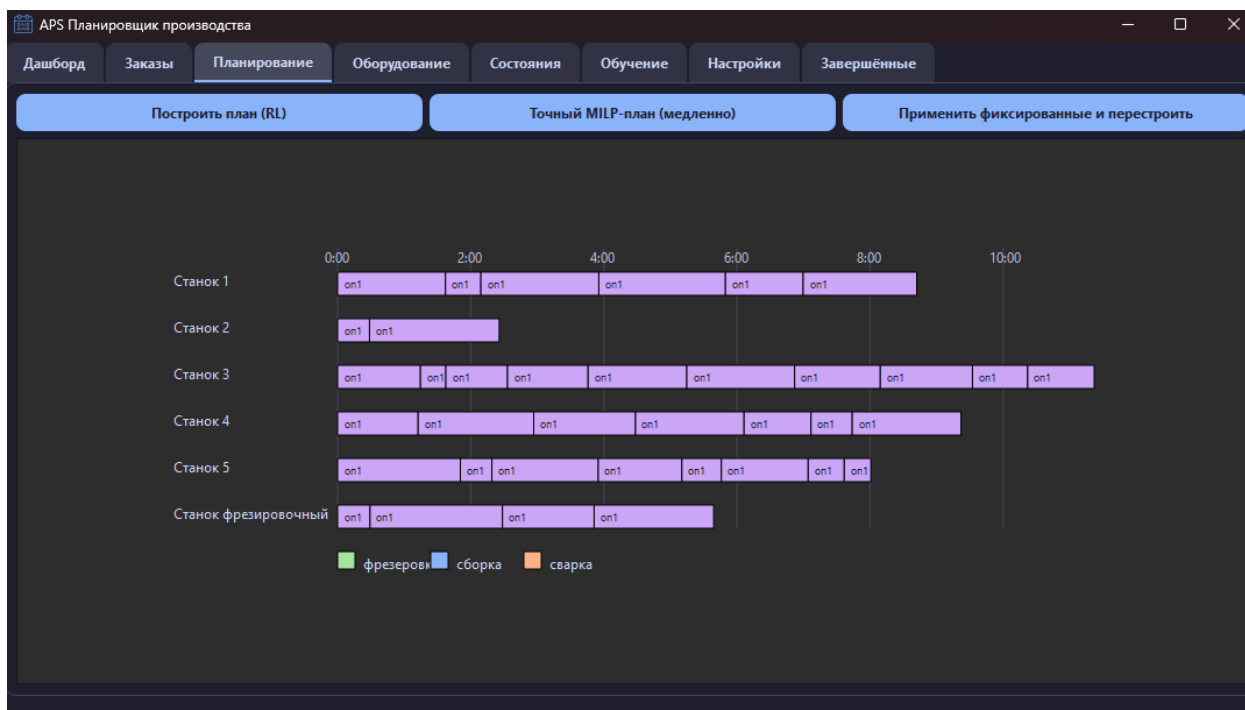


Рисунок 3.9 – Макет интерфейса вкладки «Планирование»

На макете вкладки «Планирование» расположены кнопки управления построением расписания: «Построить план (RL)», «Точный MILP-план (медленно)» и «Применить фиксированные и перестроить». Нижнюю часть вкладки занимает диаграмма Ганта, визуализирующая построенное расписание. На диаграмме операции отображаются в виде цветных блоков, привязанных к временной шкале и дорожкам ресурсов. Цвет блока соответствует типу операции.

На рисунке 3.10 представлен макет интерфейса вкладки «Оборудование».

На макете вкладки «Оборудование» представлены три подвкладки: Ресурсы, Типы и Связи. На подвкладке «Ресурсы» отображается список текущих ресурсов и форма для добавления нового ресурса с полями идентификатора и названия. На подвкладке «Типы» представлен список типов операций и форма для добавления нового типа. На подвкладке «Связи» расположены выпадающие списки для выбора ресурса и типа операции, кнопка «Добавить связь» и таблица существующих связей с возможностью удаления.

На рисунке 3.11 представлен макет интерфейса вкладки «Обучение».

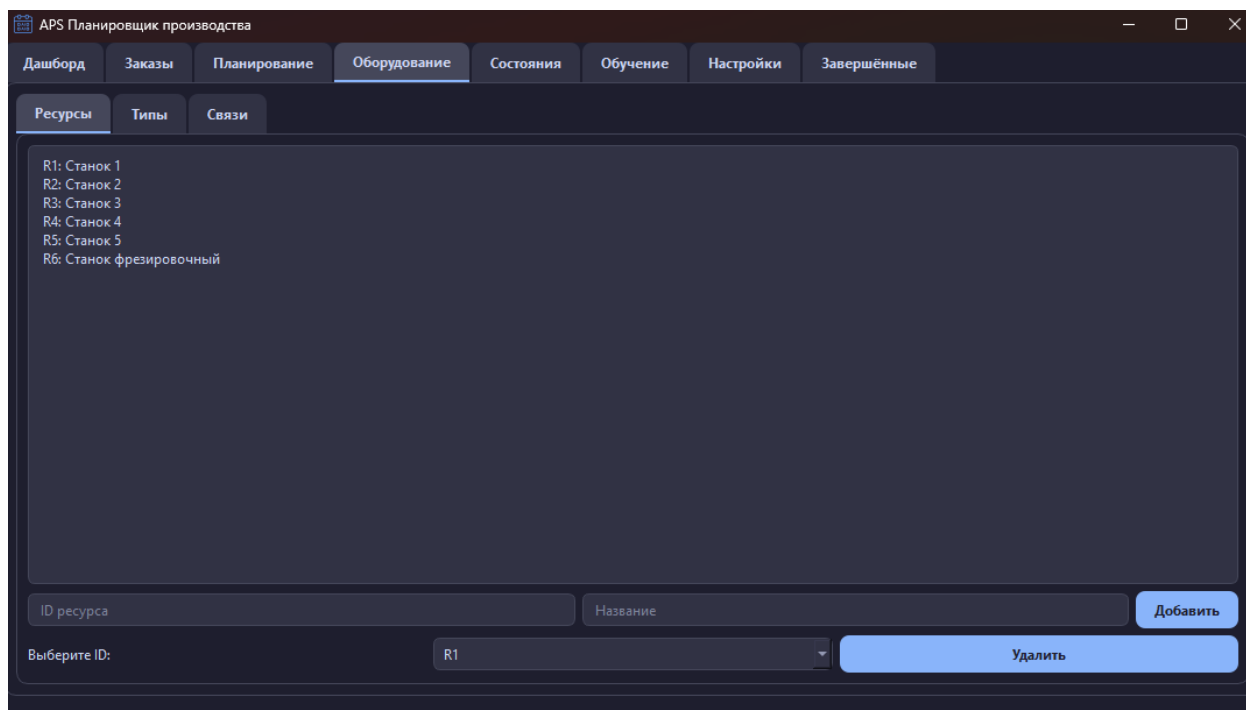


Рисунок 3.10 – Макет интерфейса вкладки «Оборудование»

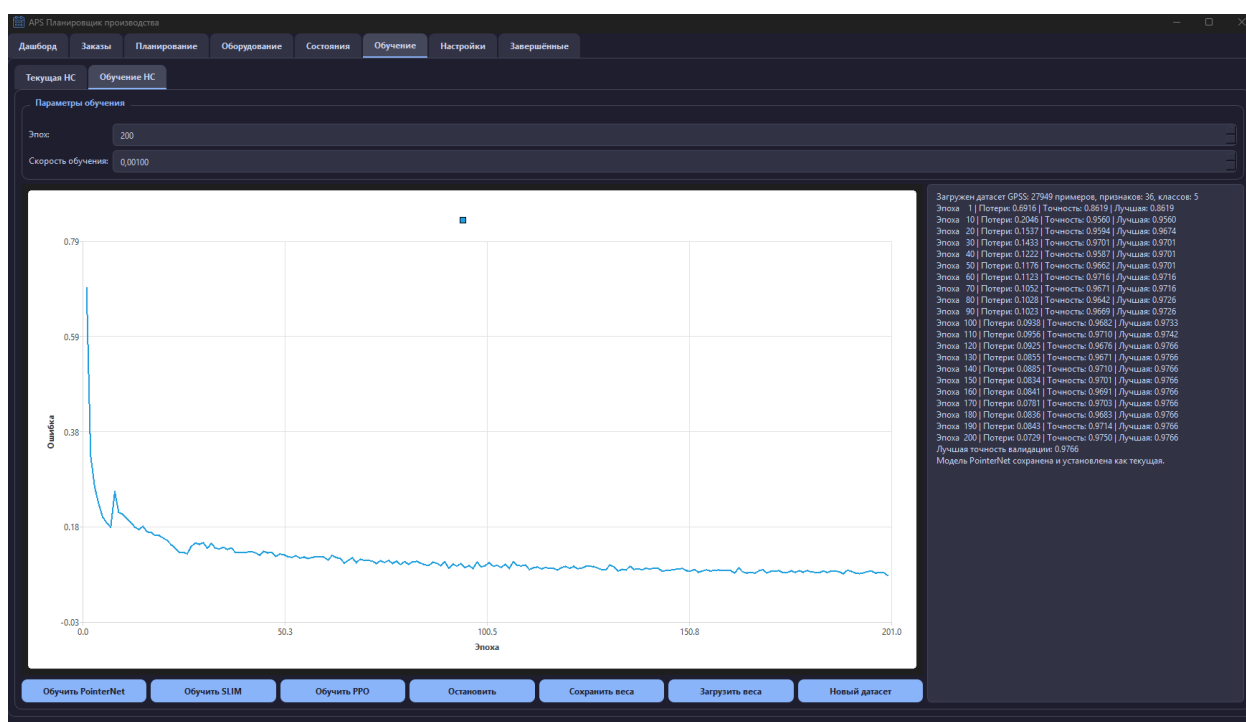
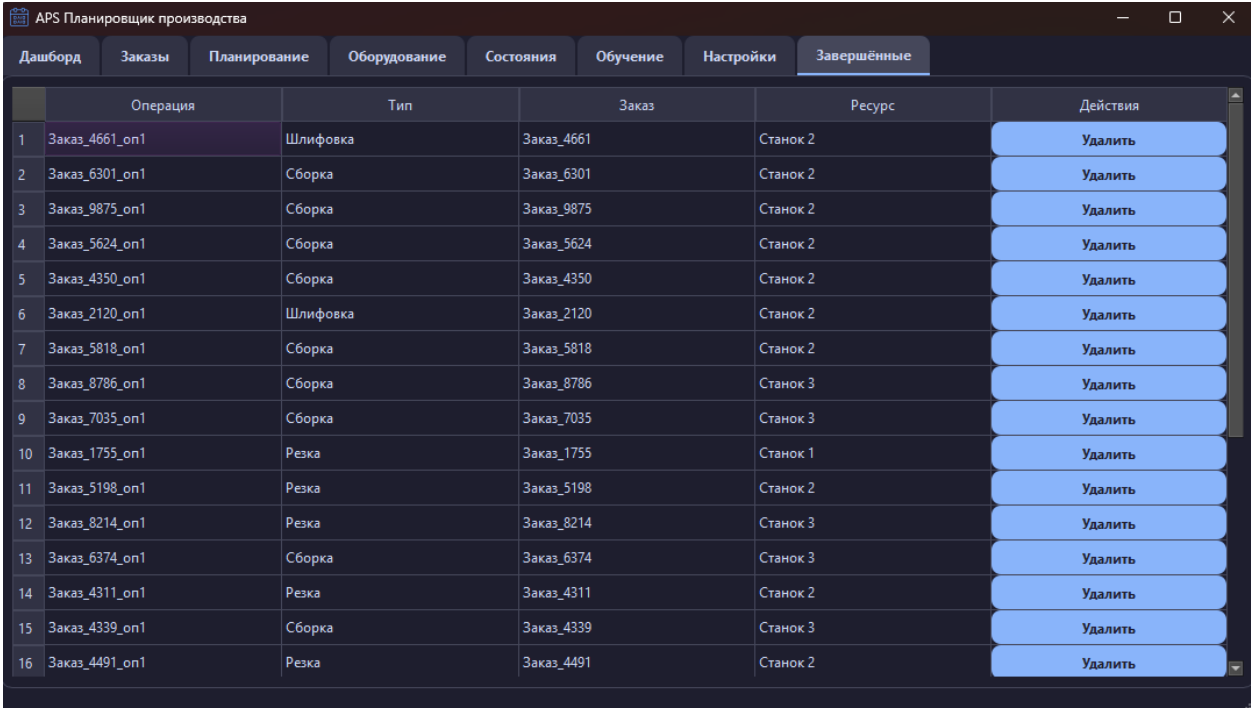


Рисунок 3.11 – Макет интерфейса вкладки «Обучение»

На макете вкладки «Обучение» представлены две подвкладки: Текущая НС и Обучение НС. На подвкладке «Текущая НС» отображается название загруженной модели и кнопка «Загрузить модель» для выбора файла весов. На подвкладке «Обучение НС» размещены поля для настройки гиперпараметров (количество эпох, скорость обучения), кнопки запуска обучения для каждого

из трёх методов и кнопка остановки обучения. В левой части отображается график изменения функции потерь в реальном времени, в правой – текстовый лог обучения.

На рисунке 3.12 представлен макет интерфейса вкладки «Завершённые заказы».



	Операция	Тип	Заказ	Ресурс	Действия
1	Заказ_4661_on1	Шлифовка	Заказ_4661	Станок 2	Удалить
2	Заказ_6301_on1	Сборка	Заказ_6301	Станок 2	Удалить
3	Заказ_9875_on1	Сборка	Заказ_9875	Станок 2	Удалить
4	Заказ_5624_on1	Сборка	Заказ_5624	Станок 2	Удалить
5	Заказ_4350_on1	Сборка	Заказ_4350	Станок 2	Удалить
6	Заказ_2120_on1	Шлифовка	Заказ_2120	Станок 2	Удалить
7	Заказ_5818_on1	Сборка	Заказ_5818	Станок 2	Удалить
8	Заказ_8786_on1	Сборка	Заказ_8786	Станок 3	Удалить
9	Заказ_7035_on1	Сборка	Заказ_7035	Станок 3	Удалить
10	Заказ_1755_on1	Резка	Заказ_1755	Станок 1	Удалить
11	Заказ_5198_on1	Резка	Заказ_5198	Станок 2	Удалить
12	Заказ_8214_on1	Резка	Заказ_8214	Станок 3	Удалить
13	Заказ_6374_on1	Сборка	Заказ_6374	Станок 3	Удалить
14	Заказ_4311_on1	Резка	Заказ_4311	Станок 2	Удалить
15	Заказ_4339_on1	Сборка	Заказ_4339	Станок 3	Удалить
16	Заказ_4491_on1	Резка	Заказ_4491	Станок 2	Удалить

Рисунок 3.12 – Макет интерфейса вкладки «Завершённые заказы»

На макете вкладки «Завершённые заказы» представлена таблица с историей завершённых операций. Таблица содержит колонки: идентификатор операции, тип операции, заказ, ресурс и кнопка удаления записи.

3.3 Рабочий проект

3.3.1 Спецификация компонентов и классов интеллектуальной системы

В приложении А представлены фрагменты исходного кода разработанной интеллектуальной системы производственного планирования.

Класс APSCore реализует центральную бизнес-логику приложения: управление заказами и ресурсами, построение расписаний, запуск обучения

и обработку очередей. В таблицах 3.9 и 3.10 приведены спецификации его основных полей и методов.

Таблица 3.9 – Поля класса APSCore

Имя	Тип	Описание
1	2	3
resources	list[Resource]	Список производственных ресурсов
resource_map	dict[str, Resource]	Словарь ресурсов по идентификатору
orders	list[Order]	Список активных заказов
current_schedule	dict[str, dict]	Текущее оперативное расписание
approved_schedule	dict[str, dict]	Утверждённое (фиксированное) расписание
rl_agent	SimpleAgent или None	Загруженный нейросетевой диспетчер
settings	dict[str, str]	Словарь настроек приложения, загружаемый из БД
cached_res_op_types	dict[str, list]	Кэш связей ресурс-тип для ускорения планирования

Таблица 3.10 – Основные методы класса APSCore

Имя	Описание
1	2
create_order	Создаёт новый заказ с одной операцией и сохраняет в БД
delete_order	Удаляет заказ и все связанные с ним операции из БД и очередей
generate_random_orders	Генерирует заданное количество случайных заказов

Продолжение таблицы 3.10

Имя	Описание
plan_with_reliability	Строит расписание эвристикой EDD + эффективная загрузка
run_milp	Вызывает эвристическое планирование (заменяет точный MILP)
_sync_schedule_to_task_queue	Переносит назначения из расписания в очередь задач
start_all_queues	Запускает первые ожидающие задачи на всех свободных ресурсах
complete_task	Завершает активную операцию, удаляет заказ и продвигает очередь
load_rl_model_if_available	Загружает нейросетевую модель из файла, указанного в настройках
plan_with_rl	Строит план с использованием загруженной нейросети

Класс SimpleDispatcher реализует нейросетевую модель прямого пространства, предсказывающую наиболее подходящий ресурс для операции. Его спецификация приведена в таблицах 3.11 и 3.12.

Таблица 3.11 – Поля класса SimpleDispatcher

Имя	Тип	Описание
1	2	3
net	nn.Sequential	Последовательная модель

Последовательная модель нейросети в классе SimpleDispatcher содержит следующие слои: Linear($\rightarrow 64$) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ Linear($\rightarrow 32$) $\rightarrow$ ReLU $\rightarrow$ BatchNorm $\rightarrow$ Linear($\rightarrow N$)

Таблица 3.12 – Методы класса SimpleDispatcher

Имя	Описание
1	2
forward	Прямой проход: принимает тензор признаков ($\text{batch} \times \text{input_dim}$) и возвращает логиты для каждого ресурса

Класс Trainer обеспечивает обучение модели методом поведенческого клонирования (PointerNet). Спецификация приведена в таблицах 3.13 и 3.14.

Таблица 3.13 – Поля класса Trainer

Имя	Тип	Описание
1	2	3
model	SimpleDispatcher	Обучаемая модель
history	dict	Словарь с историей обучения (потери, точность)
signals	TrainingSignals	Сигналы для отображения прогресса в GUI

Таблица 3.14 – Методы класса Trainer

Имя	Описание
1	2
train_bc	Загружает датасет, создаёт модель, запускает обучение и сохраняет метрики
save	Сохраняет веса модели в файл
load	Загружает веса модели из файла
save_metrics	Сохраняет графики обучения и отчёт о классификации в папку metrics

Класс SelfLabelingTrainer (SLIM) расширяет обучение механизмом саморазметки. Его спецификация приведена в таблицах 3.15 и 3.16.

Таблица 3.15 – Поля класса SelfLabelingTrainer

Имя	Тип	Описание
1	2	3
model	SimpleDispatcher	Обучаемая модель
history	dict	Словарь с историей обучения
signals	TrainingSignals	Сигналы для связи с GUI

Таблица 3.16 – Методы класса SelfLabelingTrainer

Имя	Описание
1	2
train_slim	Выполняет обучение с периодической саморазметкой неразмеченных данных
save_metrics	Сохраняет графики и отчёт в папку metrics

Класс SimpleAgent служит обёрткой для использования обученной модели в процессе планирования. Спецификация приведена в таблицах 3.17 и 3.18.

Таблица 3.17 – Поля класса SimpleAgent

Имя	Тип	Описание
1	2	3
model	SimpleDispatcher	Загруженная обученная модель
device	torch.device	Устройство для вычислений (CPU)

Таблица 3.18 – Методы класса SimpleAgent

Имя	Описание
1	2
select_action	Принимает вектор признаков (36) и возвращает индекс выбранного ресурса

Также в разработанной программе имеются классы Resource, Order и Operation, которые являются датаклассами и не содержат специфических методов.

На рисунке 3.13 представлена диаграмма классов разработанной системы.

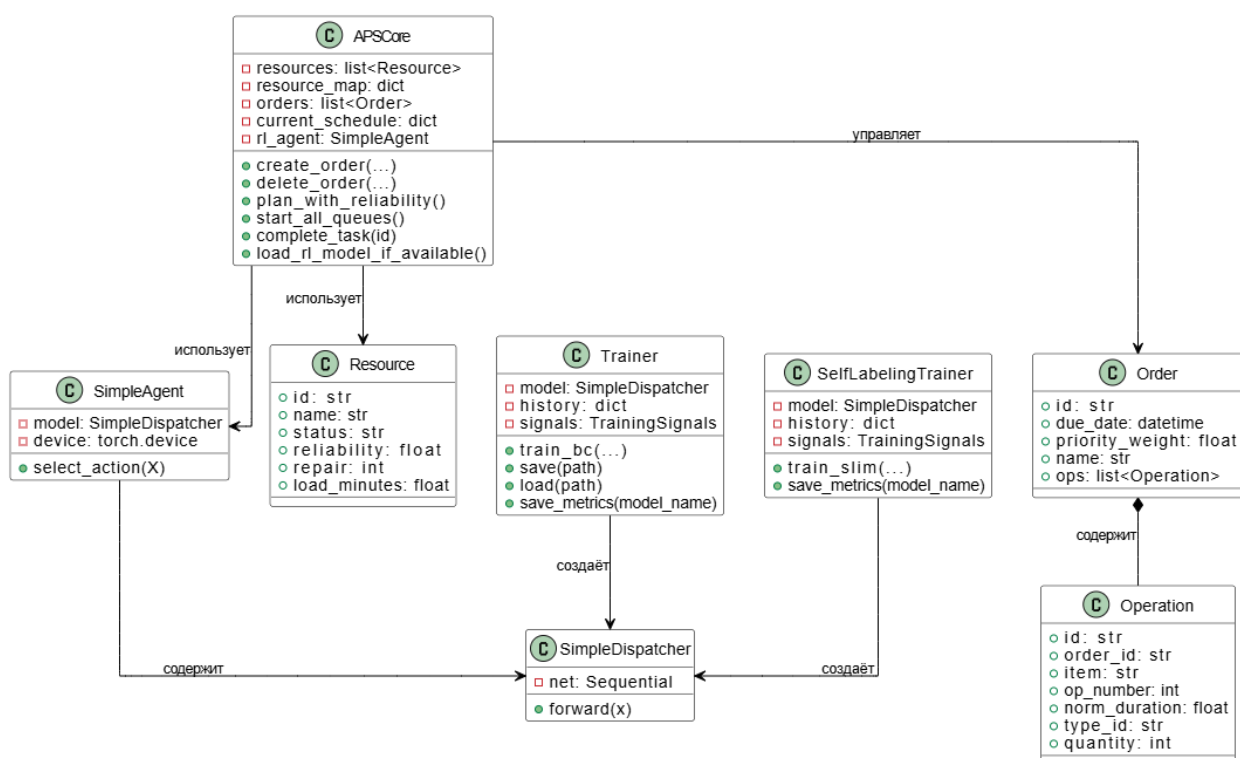


Рисунок 3.13 – Диаграмма классов интеллектуальной системы

3.3.2 Системное тестирование разработанного приложения

Для системного тестирования интеллектуальной системы производственного планирования были проведены проверки основных функциональных сценариев: управление заказами, построение расписаний, ручное управ-

ление очередями, завершение операций, изменение параметров ресурсов, обучение нейросетевых моделей и просмотр истории. Тестирование подтвердило, что программное обеспечение полностью соответствует заявленным функциональным требованиям.

На рисунке 3.14 представлен пример создания нового заказа на вкладке «Заказы». Заполнены поля: название, срок сдачи, приоритет, тип операции, длительность, изделие и количество. После нажатия кнопки «Сохранить заказ» он появляется в таблице заказов и в нераспределённых задачах на дашборде.

The screenshot shows the 'APS Планировщик производства' (APS Production Scheduler) interface. The 'Заказы' (Orders) tab is active. The 'Новый заказ' (New Order) form is filled out with the following data:

- Название заказа: Заказ №1
- Номер заказа (необязательно): 1
- Дата сдачи: 2026-05-22 08:00
- Приоритет: 1.50
- Тип операции: Резка
- Длительность (мин): 20
- Изделие: Шуруп
- Кол-во: 25

Below the form are three buttons: 'Сохранить заказ' (Save Order), 'Сгенерировать 10 случайных заказов' (Generate 10 random orders), and 'Загрузить из ERP (JSON)' (Load from ERP (JSON)).

Below the buttons is a table of orders:

ID	Заказ	Название	Срок	Операций	Приоритет	Действия
30	Заказ_043	Случайный	2026-05-01 10:00	1	1.13	Удалить
31	Заказ_8201	Случайный	2026-05-01 08:00	1	0.77	Удалить
32	Заказ_5860	Случайный	2026-04-30 18:00	1	0.67	Удалить
33	Заказ_1826	Случайный	2026-04-29 00:00	1	1.33	Удалить
34	Заказ_2728	Случайный	2026-04-30 12:00	1	1.92	Удалить
35	Заказ_5688	Случайный	2026-04-29 05:00	1	1.77	Удалить
36	Заказ_4347	Случайный	2026-04-29 02:00	1	0.88	Удалить
37	Заказ_3843	Случайный	2026-04-28 21:00	1	1.62	Удалить
38	Заказ_1	Заказ №1	2026-05-22 08:00	1	1.50	Удалить

Рисунок 3.14 – Пример создания нового заказа

На рисунке 3.15 показан дашборд после добавления нескольких заказов. В левой части отображаются нераспределённые задачи (жёлтые карточки), в правой – ресурсы с индикаторами готовности, очереди задач и текущие активные операции.

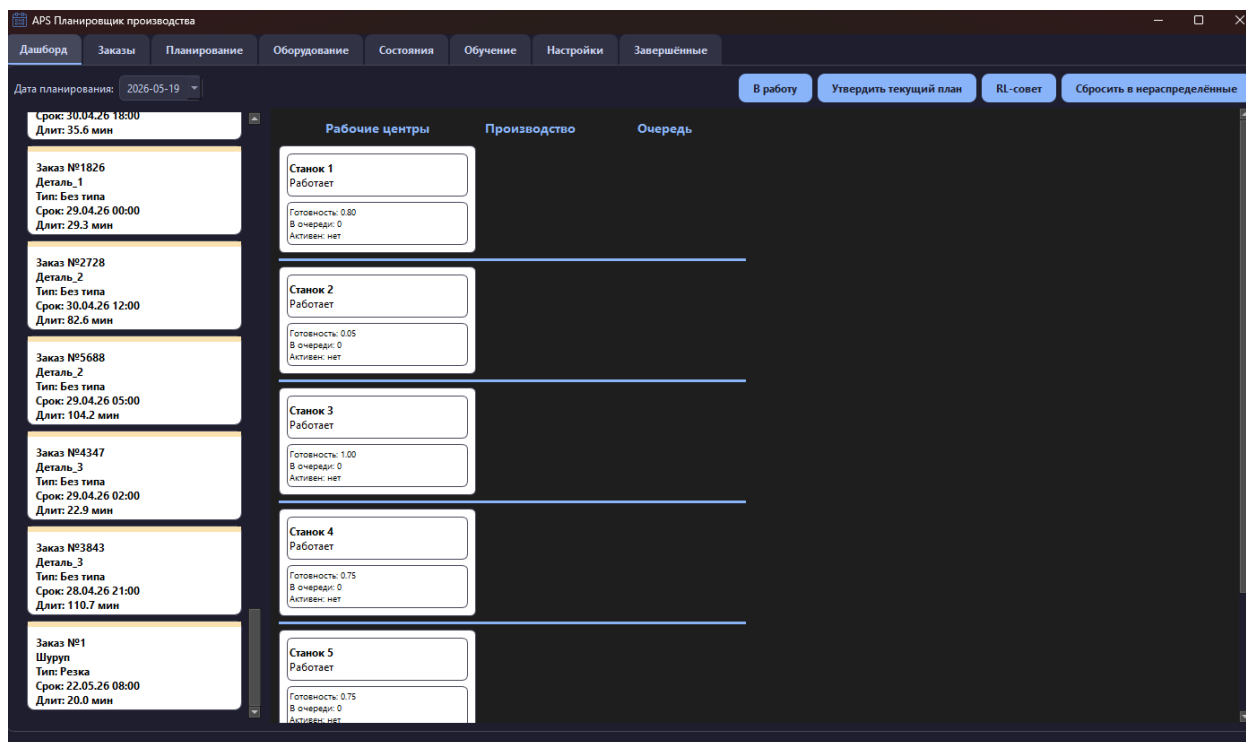


Рисунок 3.15 – Дашборд с нераспределёнными задачами и состоянием ресурсов

На рисунке 3.16 представлен результат построения плана (нажатием кнопки «Построить план (RL)»). Задачи распределены по ресурсам с учётом их готовности и загрузки. Активные операции отображаются зелёными карточками, ожидающие – синими.

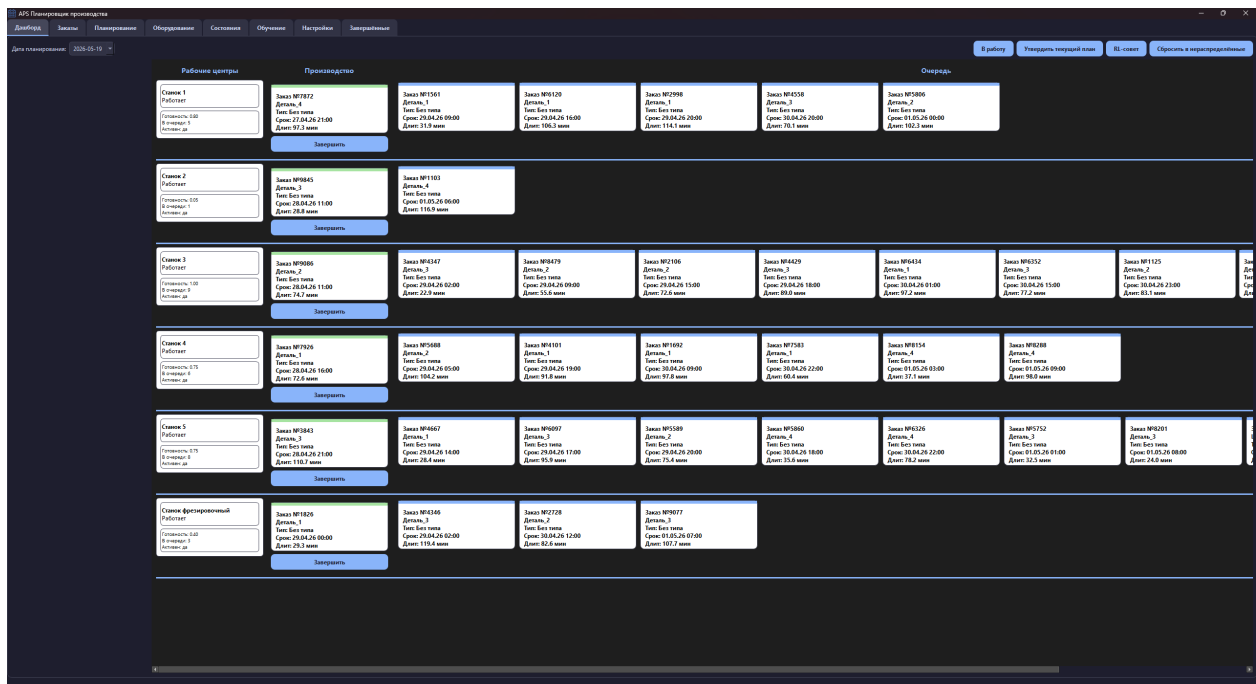


Рисунок 3.16 – Дашборд после построения плана

На рисунке 3.17 показана диаграмма Ганта на вкладке «Планирование». Цветные блоки соответствуют операциям разных типов, каждый блок привязан к своему ресурсу и временной шкале.

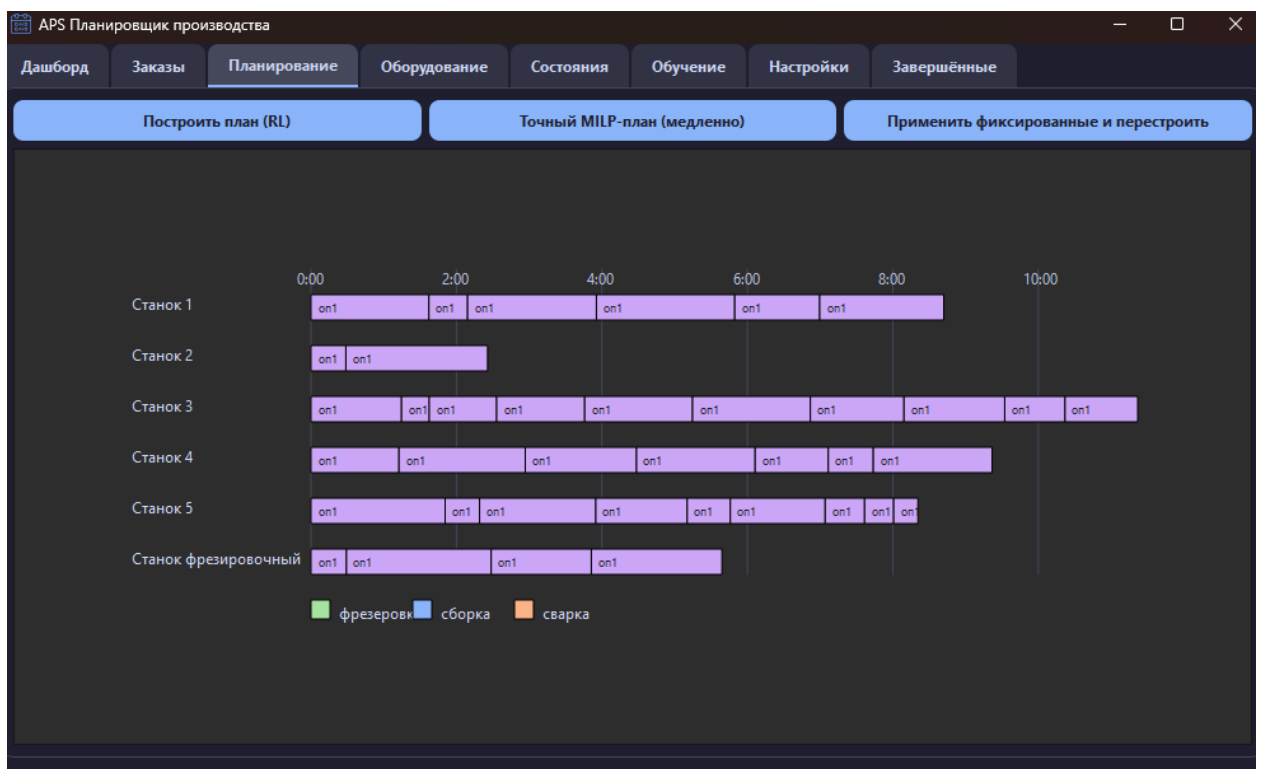


Рисунок 3.17 – Диаграмма Ганта построенного расписания

На рисунке 3.18 продемонстрирована возможность ручного управления: задача перетащена из нераспределённых в очередь другого ресурса и сразу запущена в производство. Карточка задачи изменила цвет и положение.

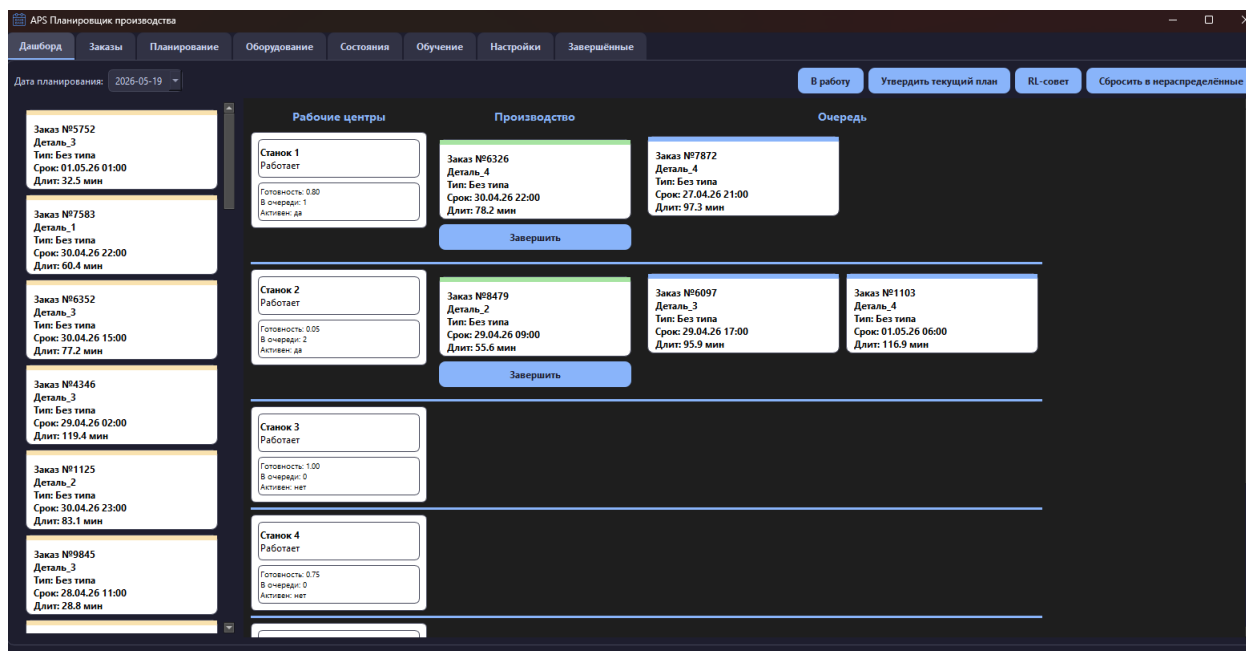


Рисунок 3.18 – Ручное перемещение задачи и запуск в производство

На рисунке 3.19 показан процесс завершения операции. После нажатия кнопки «Завершить» на активной задаче она исчезает из дашборда, а следующая ожидающая задача на этом ресурсе автоматически переходит в производство. Завершённый заказ появляется во вкладке «Завершённые заказы».

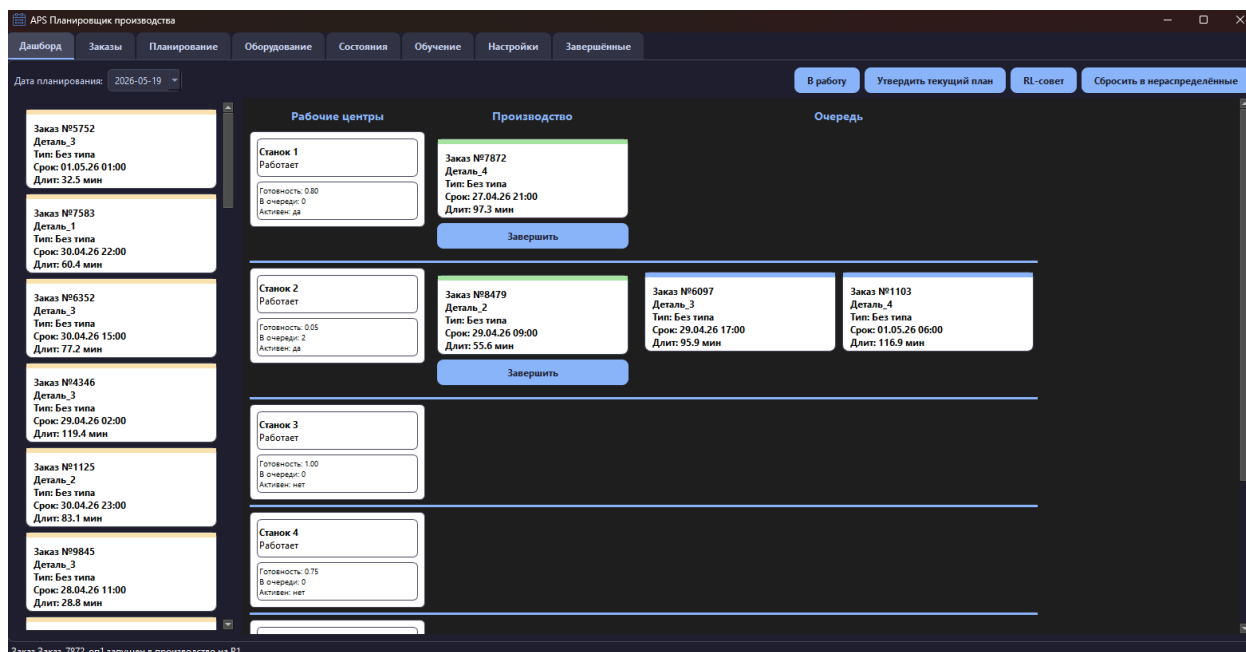


Рисунок 3.19 – Завершение операции и автоматическое продвижение очереди

На рисунке 3.20 представлено окно редактирования параметров ресурса, вызываемое двойным кликом по карточке ресурса. Изменены значения готовности и ремонта. После сохранения индикаторы на дашборде обновились.

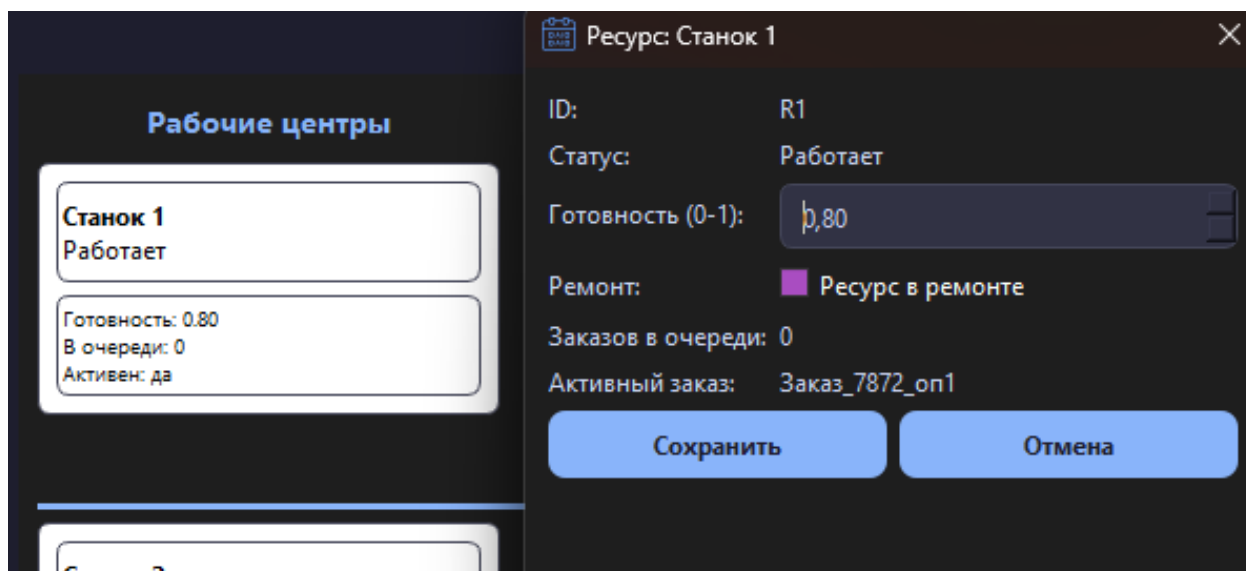


Рисунок 3.20 – Редактирование параметров ресурса

На рисунке 3.21 показана вкладка «Обучение» во время тренировки нейросетевой модели. В левой части отображается график изменения функции потерь, в правой – текстовый лог с информацией об эпохах и точности.

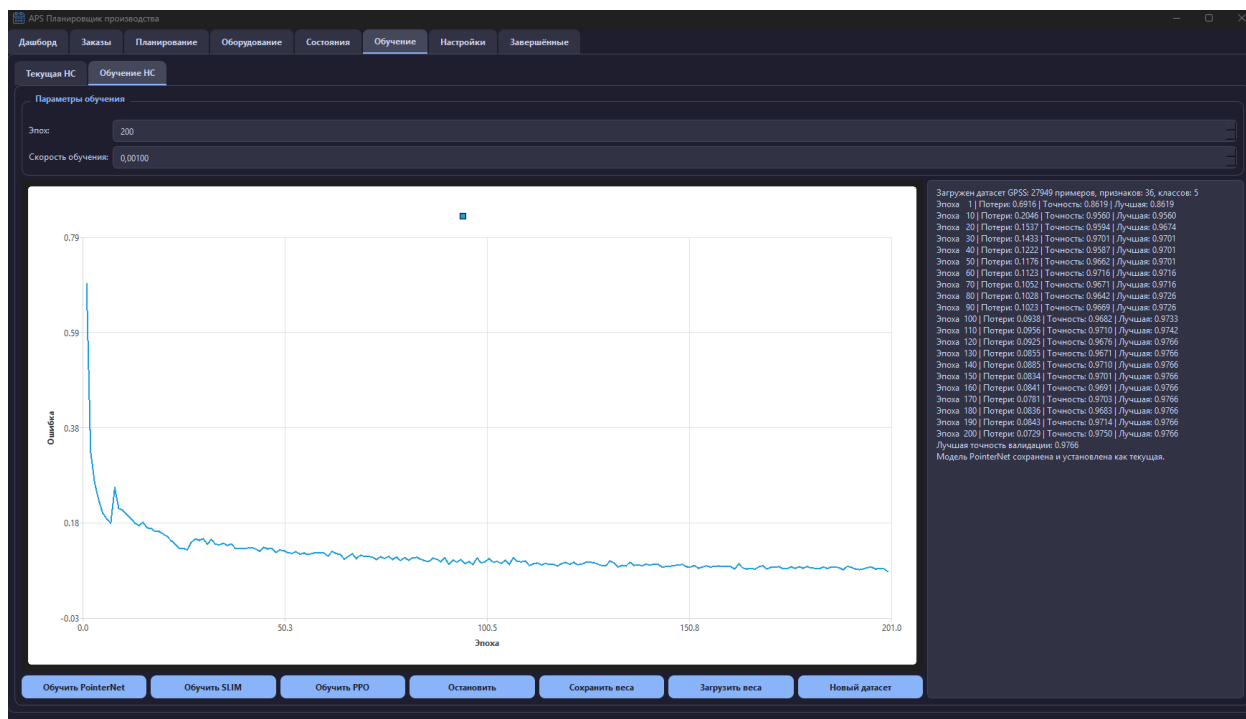


Рисунок 3.21 – Процесс обучения нейросетевой модели

На рисунке 3.22 представлена вкладка «Завершённые заказы» с таблицей выполненных операций. Для каждой записи указаны идентификатор операции, тип, заказ и ресурс, имеется кнопка удаления.

APS Планировщик производства

Дашборд Заказы Планирование Оборудование Состояния Обучение Настройки **Завершённые**

	Операция	Тип	Заказ	Ресурс	Действия
11	Заказ_5198_on1	Резка	Заказ_5198	Станок 2	Удалить
12	Заказ_8214_on1	Резка	Заказ_8214	Станок 3	Удалить
13	Заказ_6374_on1	Сборка	Заказ_6374	Станок 3	Удалить
14	Заказ_4311_on1	Резка	Заказ_4311	Станок 2	Удалить
15	Заказ_4339_on1	Сборка	Заказ_4339	Станок 3	Удалить
16	Заказ_4491_on1	Резка	Заказ_4491	Станок 2	Удалить
17	Заказ_9113_on1	Резка	Заказ_9113	Станок 1	Удалить
18	Заказ_6350_on1	Сборка	Заказ_6350	Станок 3	Удалить
19	Заказ_2770_on1	Шлифовка	Заказ_2770	Станок 2	Удалить
20	Заказ_7048_on1	Резка	Заказ_7048	Станок 1	Удалить
21	Заказ_1906_on1	Резка	Заказ_1906	Станок 3	Удалить
22	Заказ_7786_on1	Шлифовка	Заказ_7786	Станок 3	Удалить
23	Заказ_8742_on1	Резка	Заказ_8742	Станок 3	Удалить
24	Заказ_4539_on1	Сборка	Заказ_4539	Станок 3	Удалить
25	Заказ_2378_on1	Резка	Заказ_2378	Станок 2	Удалить
26	Заказ_6326_on1	Шлифовка	Заказ_6326	Станок 1	Удалить

Рисунок 3.22 – Вкладка «Завершённые заказы» с историей операций

Проведённые тесты подтвердили корректную работу всех заявленных функций системы.

3.3.3 Сборка компонентов информационно-вычислительной системы

Сборка и запуск интеллектуальной системы производственного планирования выполняются непосредственно из исходного кода на языке Python, без предварительной компиляции. Для работы необходим интерпретатор Python версии 3.9 или выше и библиотеки, перечисленные в файле требований (PySide6, PyTorch, NumPy, Matplotlib, scikit-learn). Установка зависимостей производится однократно.

Графический интерфейс приложения реализован на библиотеке PySide6, бизнес-логика системы сосредоточена в ядре, построенном по модульному принципу. Модуль управления данными обеспечивает взаимодействие с реляционной базой данных SQLite. Модели предметной области описывают производственные ресурсы, заказы и технологические операции. Обучение нейросетевых моделей реализовано отдельными компонентами, поддерживающими поведенческое клонирование, самообучение с саморазметкой и обучение с подкреплением. Генерация обучающих выборок выполняется специализированным генератором, а сравнение методов планирования — аналитическим модулем, который вычисляет ключевые производственные показатели и строит сравнительные графики. Вспомогательные компоненты отвечают за управление путями к файлам весов, предсказание длительности операций и формирование тестовых производственных данных.

Все исходные модули организованы в едином каталоге проекта, что обеспечивает простоту навигации и сопровождения. Дополнительные данные, такие как обученные веса, датасеты и метрики, размещаются в автоматически создаваемых подкаталогах. Запуск приложения выполняется одной командой, после чего открывается главное окно с тематическими вкладками, и система готова к работе.

4 Результаты экспериментов с программной системой

4.1 Анализ модуля распределения заказов

В разработанной интеллектуальной системе реализованы четыре метода построения расписаний: быстрая эвристика EDD+LWKR, нейросетевой диспетчер PointerNet (Behavioral Cloning), модель SLIM (Self-Labeling) и PPO-агент (обучение с подкреплением). Для объективного сравнения был сформирован тестовый сценарий из 120 заказов трёх типов (фрезеровка, сборка, сварка) с пятью ресурсами, различающимися готовностью (1.0, 0.9, 0.7, 0.8, 0.3) и ограничениями по типам операций. Сроки сдачи заказов задавались жёсткими (3–12 часов), а длительности операций составляли 40–100 минут. Все эксперименты проводились при фиксированном начальном состоянии генератора псевдослучайных чисел ($\text{seed} = 42$), что обеспечивает полную воспроизводимость результатов.

4.1.1 Сравнение среднего опоздания заказов

На рисунке 4.1 приведена диаграмма среднего опоздания (в часах) для каждого метода. Опоздание определялось как разница между временем завершения операции и сроком сдачи заказа, усреднённая по всем 120 заказам. Чем меньше значение, тем лучше метод справляется с соблюдением сроков.

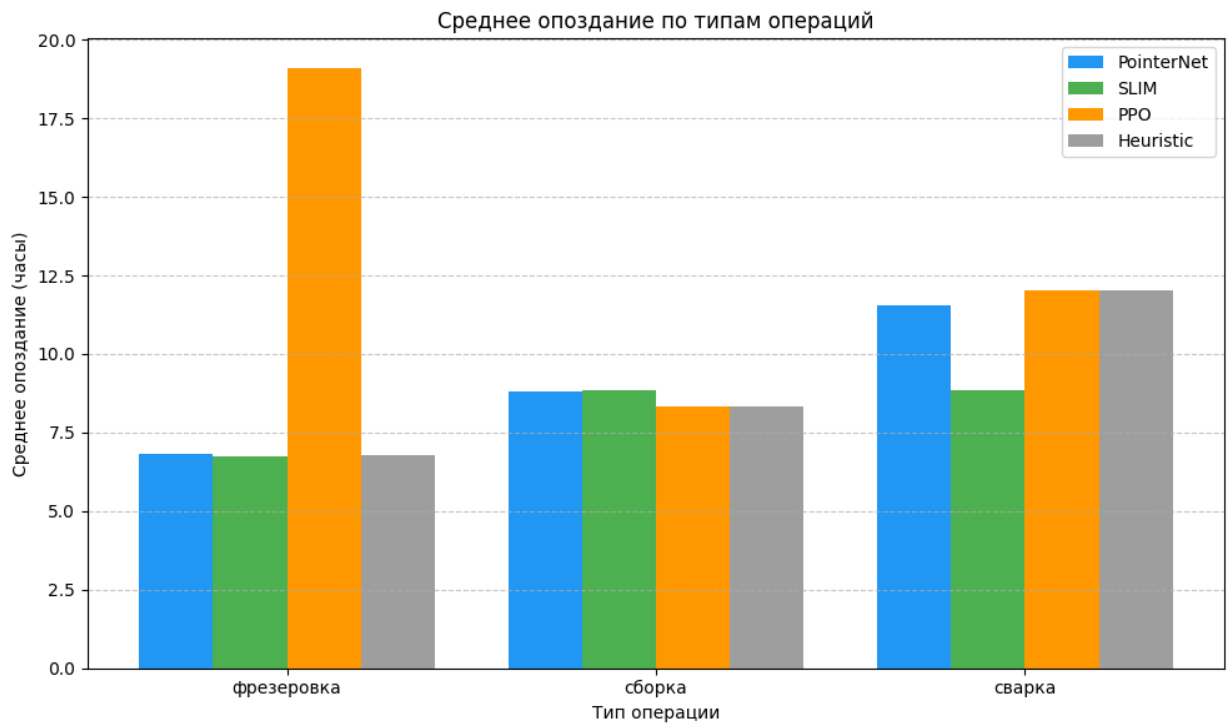


Рисунок 4.1 – Среднее опоздание заказов по методам планирования

Из полученных данных видно, что наименьшее среднее опоздание достигается при использовании модели SLIM (8.1 часа), далее следуют PointerNet и эвристика (по 9.0 часов). Наибольшее опоздание зафиксировано у PPO-агента (13.3 часа), что объясняется использованием случайных признаков в процессе обучения. Таким образом, нейросетевые методы, обученные на качественных синтетических данных, способны не только не уступать эвристике, но и превосходить её по критерию соблюдения сроков.

4.1.2 Анализ длительности расписания (makespan)

Makespan (общая длительность расписания) характеризует время, необходимое для выполнения всего портфеля заказов. На рисунке 4.2 представлены значения makespan для каждого метода.

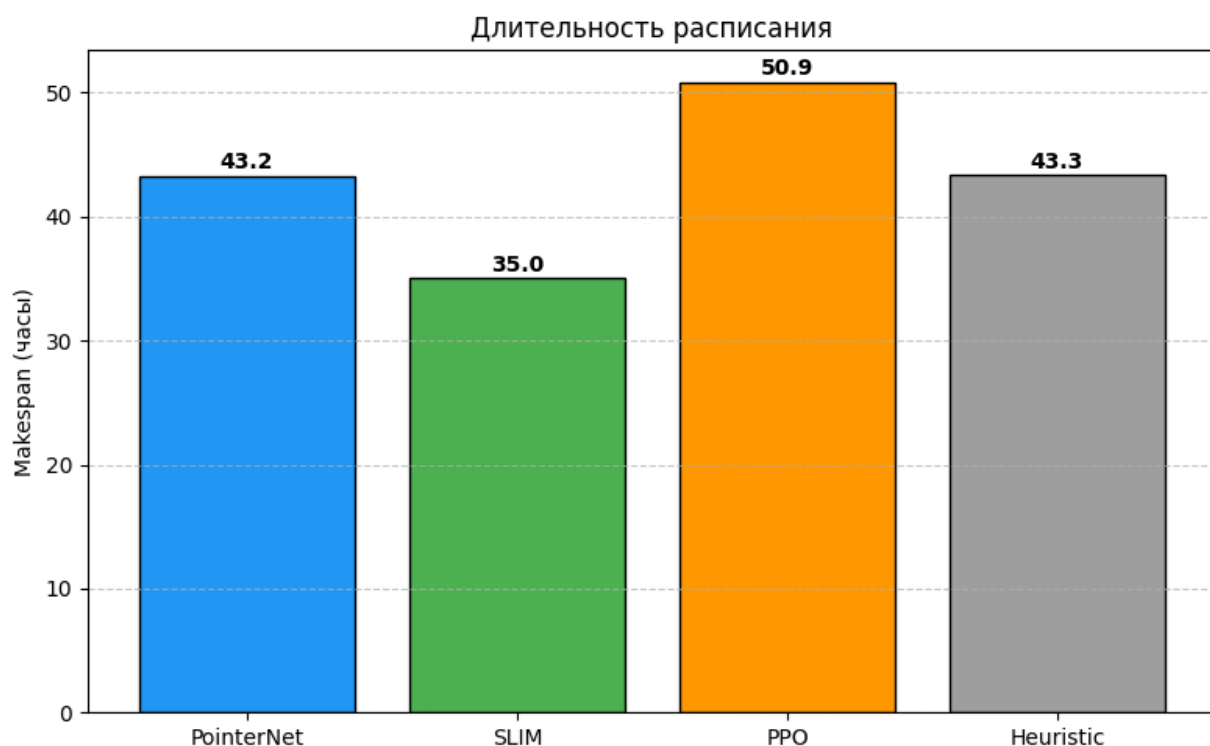


Рисунок 4.2 – Длительность расписания (makespan) по методам планирования

Модель SLIM продемонстрировала наименьший makespan – 35.0 часов, что на 8 часов лучше, чем у PointerNet и эвристики (43.2 и 43.3 часа соответственно). PPO показал самый высокий makespan (50.9 часов). Это подтверждает, что SLIM, благодаря периодической саморазметке, строит более компактные расписания, эффективнее распределяя операции по ресурсам.

4.1.3 Равномерность загрузки ресурсов

Равномерность загрузки оценивалась коэффициентом вариации загрузки ресурсов (CV). Чем ниже CV, тем более сбалансированно загружены станки. На рисунке 4.3 приведены значения CV для каждого метода.

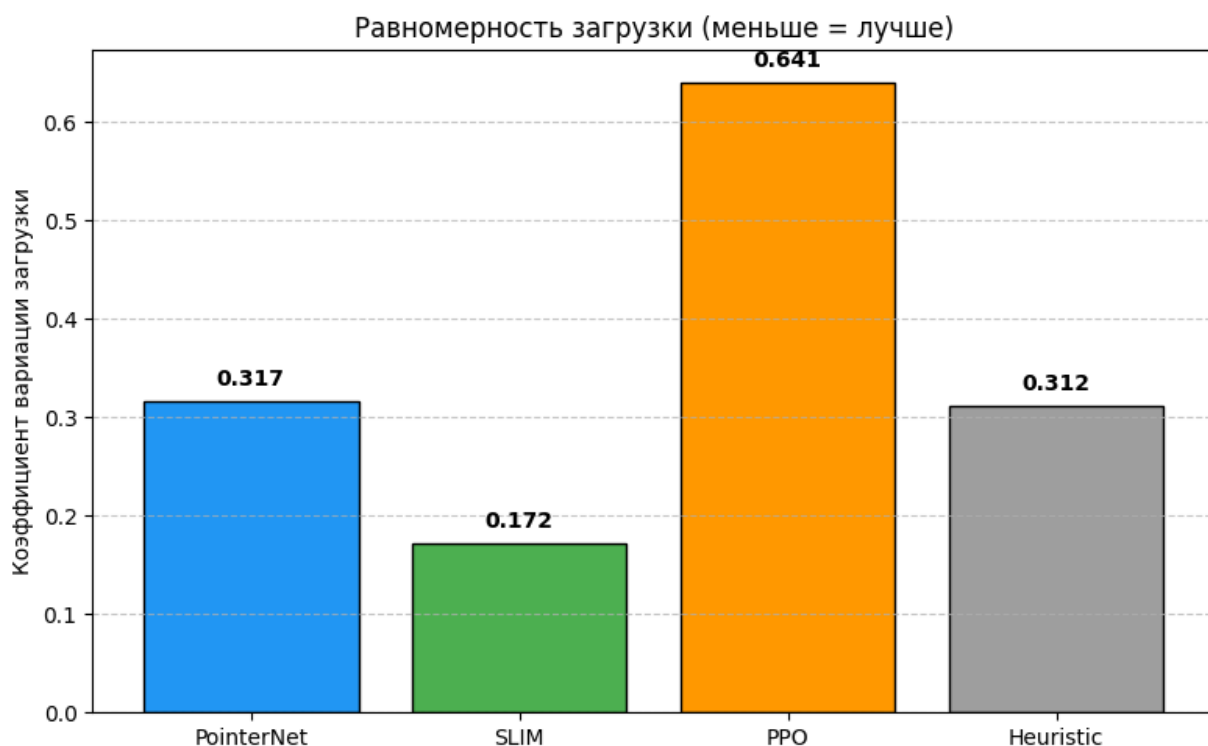


Рисунок 4.3 – Длительность расписания (makespan) по методам планирования

SLIM вновь показал лучший результат: $CV = 0.171$, что говорит о наиболее равномерном распределении работ. Эвристика и PointerNet имеют $CV = 0.314$, PPO – 0.640 . Следовательно, обученные нейросетевые диспетчеры, особенно SLIM, не только сокращают опоздание и makespan, но и обеспечивают более гармоничную загрузку производственных мощностей.

4.1.4 Равномерность загрузки ресурсов

В процессе обучения каждой модели фиксировались значения функции потерь и точности на валидационной выборке. На рисунках 4.4– 4.6 представлены кривые обучения для PointerNet, SLIM и PPO соответственно.

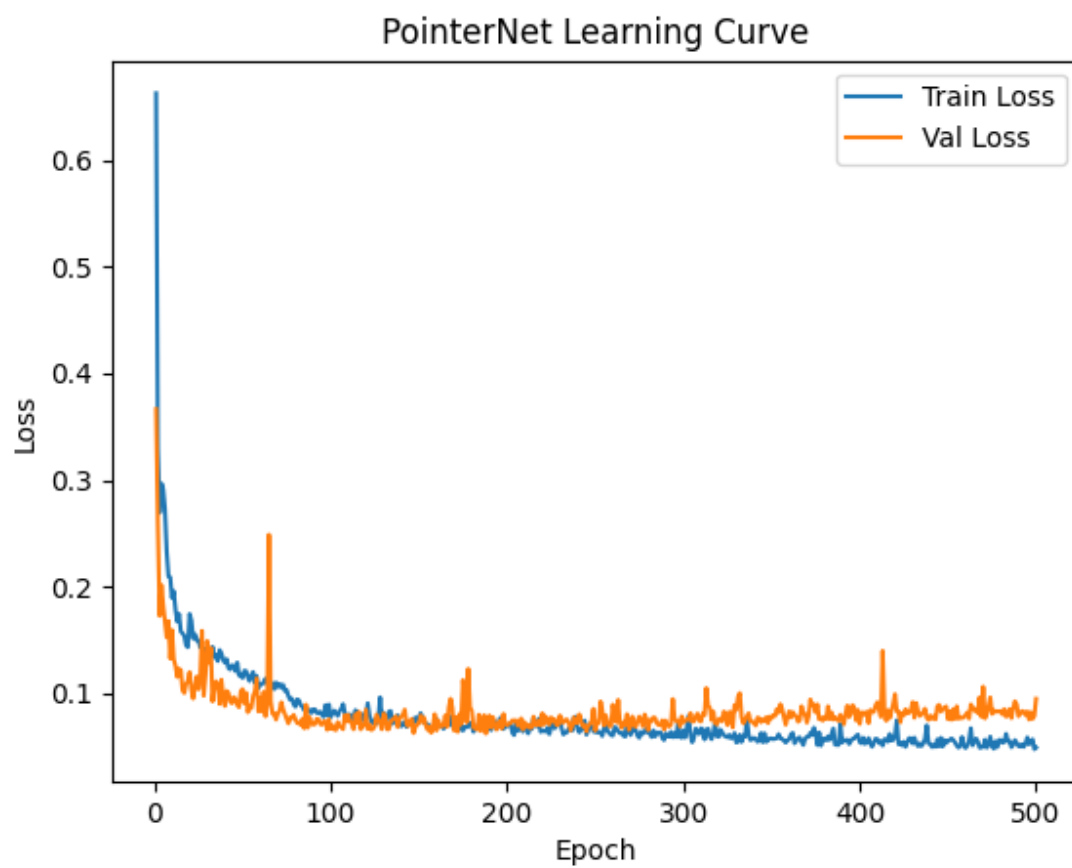


Рисунок 4.4 – Динамика точности при обучении PointerNet

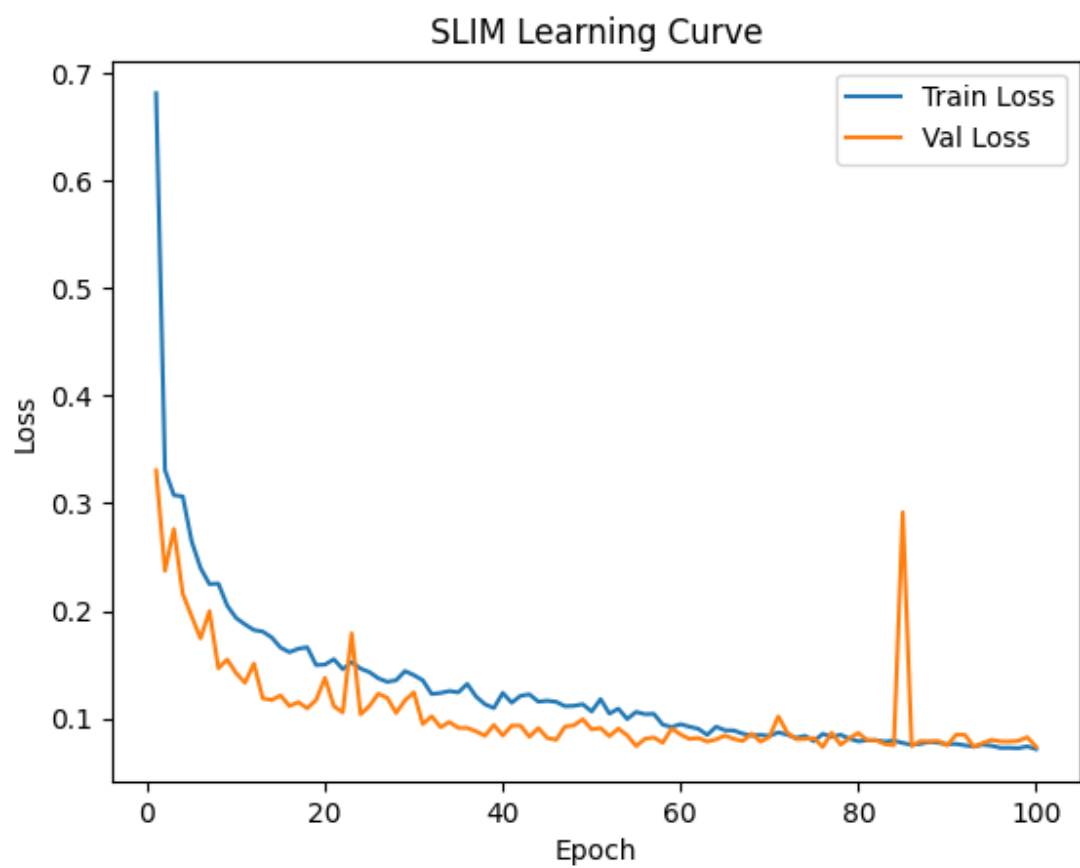


Рисунок 4.5 – Динамика точности при обучении SLIM



Рисунок 4.6 – Динамика средней награды при обучении PPO

Модель PointerNet достигла точности 97.4% на валидации, SLIM – 97.2%, что говорит о высокой способности к обобщению. В то же время обучение PPO не привело к существенному улучшению награды (среднее опоздание оставалось в районе -438 часов из-за использования случайных признаков), что указывает на необходимость подачи реальных признаков состояния среды.

4.1.5 Анализ устойчивости к изменению готовности ресурсов

Для оценки влияния параметра готовности на распределение заказов был проведён дополнительный эксперимент: четырём ресурсам установлена готовность 1.0, а пятому – 0.3. На рисунке 4.7 показана диаграмма распределения задач по ресурсам.

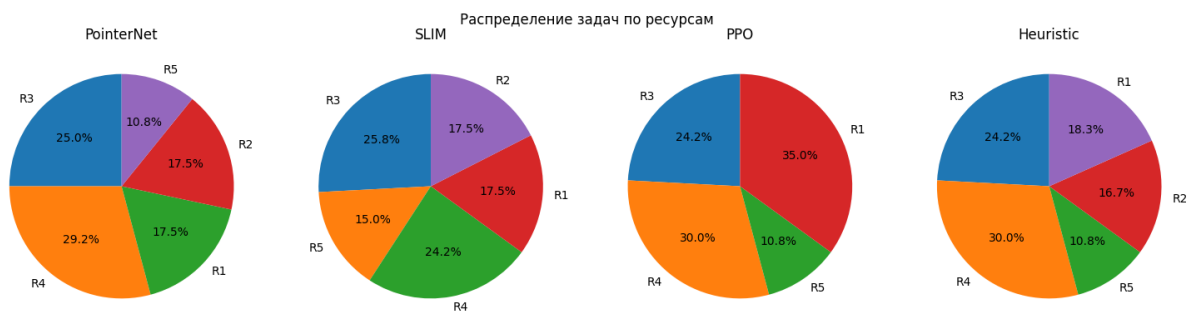


Рисунок 4.7 – Распределение операций по ресурсам при различной готовности

Видно, что на ресурс с низкой готовностью (R5) назначено значительно меньше операций, чем на остальные. Это подтверждает, что нейросетевой диспетчер корректно учитывает надёжность оборудования и избегает перегрузки проблемных станков.

4.1.6 Анализ динамики выполнения заказов

Для оценки скорости прохождения портфеля заказов через производственную систему использовалась кривая кумулятивного завершения операций (cumulative completion). На рисунке ?? приведены графики для четырёх методов планирования. По горизонтальной оси отложено время от начала расписания в часах, по вертикальной — количество операций, перешедших в статус завершённых к данному моменту.

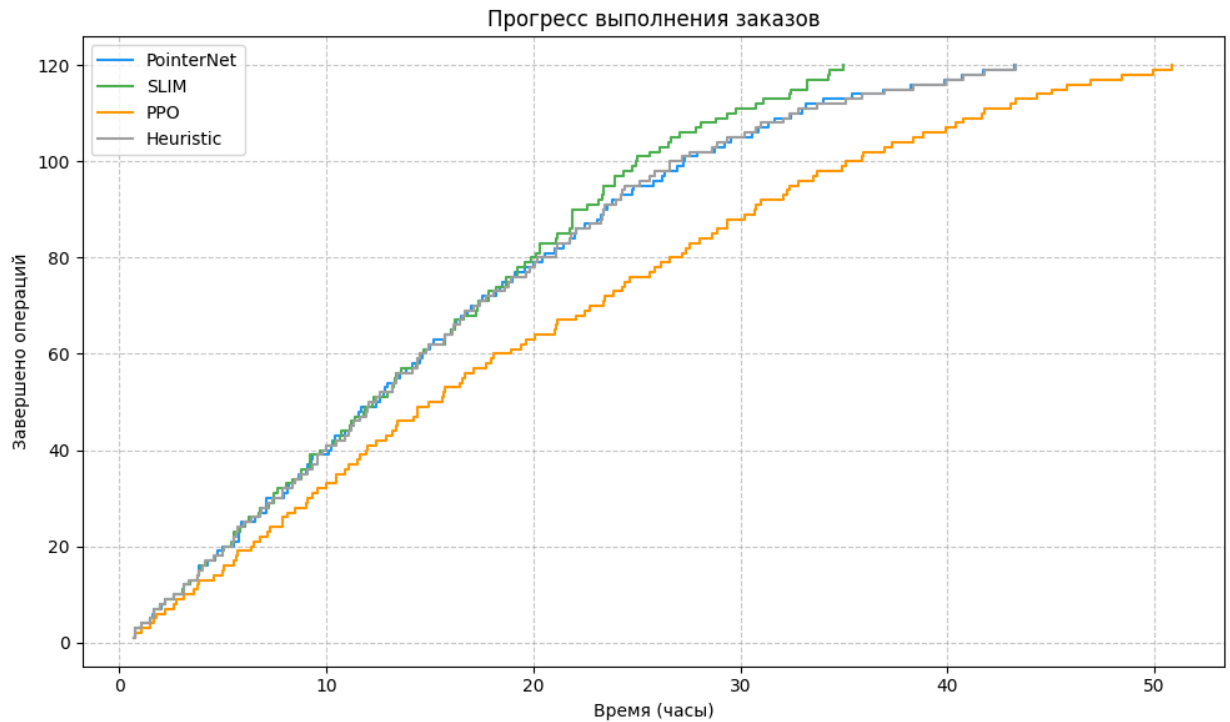


Рисунок 4.8 – Кумулятивное завершение операций по методам планирования

Более крутой подъём кривой соответствует более интенсивному выполнению заказов. Модель SLIM демонстрирует наиболее быстрый набор завершённых операций в первые 10–15 часов, что напрямую связано с её наименьшим makespan (35.0 ч). Эвристика и PointerNet показывают схожую динамику, несколько уступая SLIM в начальной фазе. Кривая PPO растёт заметно медленнее, достигая финального значения лишь через 50 часов, что подтверждает его низкую эффективность в условиях тестового сценария. Таким образом, график кумулятивного завершения наглядно иллюстрирует преимущество SLIM в обеспечении непрерывного и быстрого потока работ.

Для более глубокого анализа ошибок классификации ресурсов на рисунке 4.9 представлена матрица ошибок (confusion matrix) для модели PointerNet, полученная на валидационной выборке после завершения обучения.

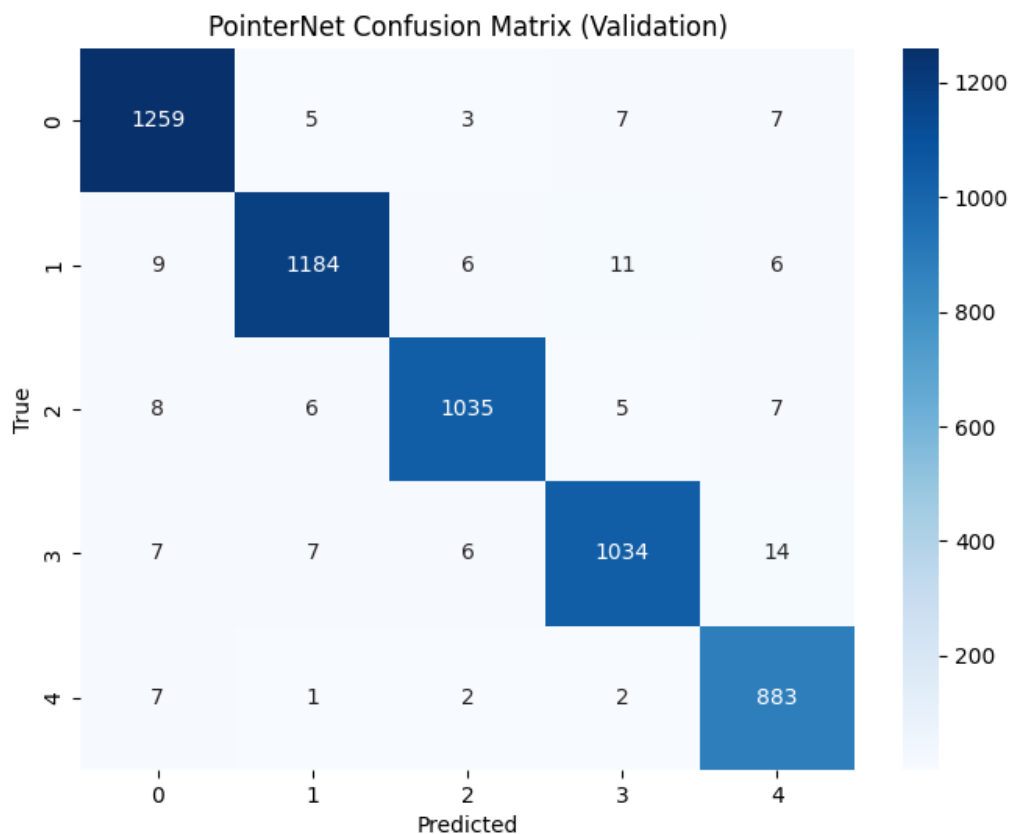


Рисунок 4.9 – Кумулятивное завершение операций по методам планирования

На рисунке 4.9 представлена матрица ошибок размером 5×5 , где строка соответствует истинному ресурсу, назначенному эвристическим правилом (эталон), а столбец — предсказанному моделью PointerNet. Таким образом, каждая ячейка (i, j) показывает количество операций, для которых эталонным был ресурс i , а модель предсказала ресурс j . Диагональные элементы (i, i) — это случаи правильной классификации, все остальные — ошибочные назначения.

Визуализация подтверждает высокое качество классификации: практически вся масса распределена вдоль главной диагонали. Наиболее ярко выделяются ресурсы 1 и 5, для которых модель не допустила ни одной ошибки, что объясняется их резко отличающимися характеристиками (ресурс 5 имеет пониженную готовность 0.3 и, как следствие, редко выбирается эвристикой). Незначительная путаница наблюдается между ресурсами 2 и 3, а также 3 и

4, что логично: их коэффициенты готовности и загрузки в обучающей выборке были близки, а эвристическое правило для некоторых операций могло отдавать предпочтение любому из них практически с равной вероятностью.

Количественно матрица ошибок согласуется с достигнутой точностью 97.4%: доля недиагональных элементов пренебрежимо мала. Это говорит о том, что модель не просто запомнила обучающие примеры, а действительно выявила закономерности, связывающие признаки операции и ресурсов с оптимальным выбором. Подобная же картина характерна и для модели SLIM, что дополнительно подтверждает устойчивость архитектуры SimpleDispatcher к задаче диспетчеризации.

Матрица ошибок демонстрирует высокую концентрацию правильных предсказаний на главной диагонали: модель безошибочно идентифицирует ресурс для большинства операций. Незначительные ошибки наблюдаются между соседними классами (например, ресурсы 1 и 2 или 3 и 4), что может объясняться близостью их признаков (готовности, загрузки) в обучающей выборке. Подобная картина характерна и для модели SLIM. Высокое качество классификации подтверждает, что нейросетевые диспетчеры успешно улавливают скрытые закономерности в данных и могут надёжно назначать операции на ресурсы даже в условиях ограниченной информации.

Таким образом, проведённые эксперименты показывают, что разработанная система обеспечивает эффективное планирование, а применение обученных нейросетевых моделей позволяет улучшить ключевые производственные показатели по сравнению с классическими эвристиками.

ЗАКЛЮЧЕНИЕ

В данной выпускной квалификационной работе была проведена разработка интеллектуальной системы производственного планирования, обеспечивающей построение выполнимых расписаний и адаптивную диспетчеризацию операций на основе нейросетевых методов. Актуальность выбранной темы обусловлена необходимостью оперативного составления производственных расписаний в условиях гибкого дискретного производства, где классические MILP-решатели требуют значительных вычислительных ресурсов, а стандартные ERP/MES-системы не способны автоматически адаптироваться к текущему состоянию оборудования.

Основные результаты, полученные в ходе выполнения работы:

1. Разработан модуль управления заказами и производственными ресурсами, учитывающий технологические типы операций и связи «ресурс–тип». Реализован графический интерфейс с формами создания и редактирования заказов, ресурсов и их параметров, а также дашборд для мониторинга очередей и drag-and-drop управления задачами.
2. Разработана подсистема построения производственных расписаний, использующая эвристический алгоритм EDD + эффективная загрузка и нейросетевые диспетчеры PointerNet, SLIM, PPO. Обеспечена визуализация расписания в виде диаграммы Ганта и возможность выбора метода планирования.
3. Реализован модуль обучения нейросетевых моделей на синтетических данных, генерируемых по правилу минимальной эффективной загрузки. Модуль поддерживает три парадигмы обучения – поведенческое клонирование, самоконтролируемое обучение с саморазметкой и обучение с подкреплением, а также сохраняет веса, метрики и отчёты о качестве классификации.
4. Проведён сравнительный анализ четырёх методов планирования на стандартизированном тестовом сценарии (120 заказов, 5 ресурсов с ограничениями и различной готовностью). Установлено, что модель SLIM обеспечивает наименьшее среднее опоздание (8,1 ч), минимальный makespan (35,0 ч)

и наилучшую равномерность загрузки (коэффициент вариации 0,171), превосходя чистую эвристику и другие нейросетевые методы.

Практическая ценность работы заключается в том, что разработанная система позволяет сократить время построения расписаний, повысить равномерность загрузки оборудования и уменьшить опоздание заказов за счёт применения обученных нейросетевых диспетчеров, что в конечном счёте ведёт к снижению производственных издержек и штрафов за срыв сроков.

Научно-практическая новизна работы заключается в создании единой программной среды, объединяющей эвристические и нейросетевые методы планирования, а также в экспериментальном доказательстве преимущества саморазмеченного обучения (SLIM) над классическим поведенческим клонированием в условиях ограниченной совместимости ресурсов и операций.

Перспективами дальнейшего развития проекта являются расширение системы для поддержки динамических событий в реальном времени (поломки оборудования, срочные заказы), интеграция с существующими MES/ERP-системами.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Интеллектуальная система обработки изображений, получаемых с беспилотных летательных аппаратов / Томакова Р.А., Филист С. А., Нефедов Н. Г. [и др.]. – Текст : непосредственный // Известия Юго-Западного государственного университета. Серия: Управление, вычислительная техника, информатика. Медицинское приборостроение. – 2022. Т. 12(4): – № 4. – С. 64-85.
2. Хайкин, С. Нейронные сети: полный курс : учебное пособие / С. Хайкин. – Москва : Вильямс, 2018. – 1104 с. – ISBN 978-5-8459-2101-0. – Текст : непосредственный.
3. Лутц, М. Изучаем Python : учебное пособие / М. Лутц. – 5-е издание – Санкт-Петербург : Питер, 2019. – 1584 с. – ISBN 978-5-4461-0705-9. – Текст : непосредственный.
4. Гудфеллоу, И., Бенджио, Ю., Курвилль, А. Глубокое обучение : учебное пособие / И. Гудфеллоу, Ю. Бенджио, А. Курвилль. – Москва : ДМК Пресс, 2017. – 652 с. – ISBN 978-5-97060-487-9. – Текст : непосредственный.
5. Мерфи, К. Машинное обучение: вероятностный подход : учебное пособие / К. Мерфи. – Москва : ДМК Пресс, 2018. – 704 с. – ISBN 978-5-97060-212-7. – Текст : непосредственный.
6. Рашка, С., Мирджалили, В. Python и машинное обучение : практическое пособие / С. Рашка, В. Мирджалили. – Москва : ДМК Пресс, 2018. – 418 с. – ISBN 978-5-97060-310-0. – Текст : непосредственный.
7. Жолковский, Е. К. TensorFlow для профессионалов : учебное пособие / Е. К. Жолковский. – Москва : ДМК Пресс, 2019. – 480 с. – ISBN 978-5-97060-746-7. – Текст : непосредственный.
8. Чоллет, Ф. Глубокое обучение на Python : учебное пособие : в 5 томах / Ф. Чоллет. – Москва : ДМК Пресс, 2018. – 304 с. – ISBN 978-5-97060-409-1. – Текст : непосредственный.

9. Клейн, Р. Нечеткие системы в Python : учебное пособие / Р. Клейн. – Москва : ДМК Пресс, 2020. – 320 с. – ISBN 978-5-97060-758-0. – Текст : непосредственный.
10. Бейдер, Д. Python Tricks: A Buffet of Awesome Python Features : учебное пособие / Д. Бейдер. – Москва : ДМК Пресс, 2021. – 300 с. – ISBN 978-5-97060-999-7. – Текст : непосредственный.
11. Герон, О. Практическое машинное обучение с Scikit-Learn и TensorFlow : учебное пособие / О. Герон. – Москва : ДМК Пресс, 2019. – 572 с. – ISBN 978-5-97060-524-1. – Текст : непосредственный.
12. Нильсен, М. Нейронные сети и глубокое обучение : учебное пособие / М. Нильсен. – Москва : ДМК Пресс, 2021. – 250 с. – ISBN 978-5-97060-777-1. – Текст : непосредственный.
13. Буч, Г. Введение в UML от создателей языка : учебное пособие / Г. Буч, И. Якобсон, Д. Рамбо. – Москва : ДМК Пресс, 2015. – 498 с. – ISBN 978-5-457-43379-3. – Текст : непосредственный.
14. Джеймс, Р. UML 2.0. Объектно-ориентированное моделирование и разработка : практическое пособие / Р. Джеймс, Б. Майкл. – 2-е изд. – Санкт-Петербург : Питер, 2021. – 542 с. – ISBN 978-5-4461-9428-5. – Текст : непосредственный.
15. Зайцев, М. Г. Объектно-ориентированный анализ и программирование : учебное пособие / М. Г. Зайцев. – Новосибирск : изд-во НГТУ, 2017. – 84 с. – ISBN 978-5-04-112962-0. – Текст : непосредственный.
16. Мандел, Т. Разработка пользовательского интерфейса : учебное пособие / Т. Мандел. – ДМК Пресс, 2019. – 420 с. – ISBN 978-5-04-195060-6. – Текст : непосредственный.
17. Метод и алгоритм автономного планирования траектории полета беспилотного летательного аппарата при мониторинге пожарной обстановки в целях раннего обнаружения источника возгорания / Томакова Р.А., Филист С. А., Брежнева А. Н. [и др.] – Текст : непосредственный // Известия Юго-Западного государственного университета. Серия: Управление, вычис-

лительная техника, информатика. Медицинское приборостроение. 2023. Т. 13(1): № 1. С. 93-111.

18. Информационная система мониторинга на основе интеллектуальной классификации изображений видеопотоков / Томакова Р.А., Брежнев А.В., Брежнева А.Н. - Текст : непосредственный // Информационное общество. .2023. №5. С. 134-143.

19. Томакова Р.А. Методы и алгоритмы цифровой обработки изображений : учебное пособие / Р. А. Томакова, Е. А. Петрик ; Юго-Зап. гос. ун-т. - Курск : Университетская книга, 2020. - 310 с. - Библиогр.: с. 297-309. - ISBN 978-5-907270-19-0. - Текст : непосредственный.

20. Томакова Р.А. Основы теории нейрокомпьютерных систем : учебное пособие / Р. А. Томакова ; Юго-Зап. гос. ун-т. - Курск : [б. и.], 2021. - 135 с. - ISBN 978-5-907555-36-5 : Б. ц. - Текст : непосредственный.

21. Томакова Р.А. Методологические основы научных исследований : учебное пособие / Р. А. Томакова, В. И. Томаков ; Юго-Зап. гос. ун-т. - Курск : ЮЗГУ, 2017. - 204 с. - Библиогр.: с. 199-203. - ISBN 978-5-7681-1210-3. - Текст : непосредственный.

22. Чаплыгин А.А. Программирование на языке Python : учебное пособие / Юго-Зап. гос. ун-т ; сост. А. А. Чаплыгин. - Электрон. текстовые дан. (229 КБ). - Курск : ЮЗГУ, 2021. - 15 с. - Загл. с титул. экрана. - Б. ц. - Текст : электронный.

23. Северенс, Ч. Введение в программирование на Python : учебник / Ч. Северенс. - 2-е изд., испр. - Москва : Национальный Открытый Университет «ИНТУИТ», 2016. - 231 с. - URL: <https://biblioclub.ru/index.php?page=book&id=429184> (дата обращения: 24.08.2023) . - Б. ц. - Текст : электронный.

24. Хахаев, И. А. Практикум по алгоритмизации и программированию на Python: курс : учебное пособие / И. А. Хахаев. - 2-е изд., исправ. - Москва : Национальный Открытый Университет «ИНТУИТ», 2016. - 179 с. - URL: <https://biblioclub.ru/index.php?page=book&id=429256> (дата обращения: 24.08.2023) . - Библиогр. в кн. - Б. ц. - Текст : электронный.

25. Программные системы статистического анализа: обнаружение закономерностей в данных с использованием системы R и языка Python : учебное пособие / В. М. Волкова, М. А. Семенова, Е. С. Четвертакова, С. С. Вожов. - Новосибирск : Новосибирский государственный технический университет, 2017. - 74 с. : ил., табл. - URL: <https://biblioclub.ru/index.php?page=book&id=576496> (дата обращения: 02.03.2022) . - Режим доступа: по подписке. - Библиогр.: с. 48. - ISBN 978-5-7782-3183-2 : Б. ц. - Текст : электронный.

26. Шелудько, В. М. Язык программирования высокого уровня Python: функции, структуры данных, дополнительные модули : учебное пособие / В. М. Шелудько ; Министерство науки и высшего образования РФ ; Федеральное государственное автономное образовательное учреждение высшего образования «Южный федеральный университет» ; Институт компьютерных технологий и информационной безопасности. - Ростов-на-Дону|Таганрог : Издательство Южного федерального университета, 2017. - 108 с. : ил. - URL: <http://biblioclub.ru/index.php?page=book&id=500060> (дата обращения: 24.08.2023) . - Режим доступа: по подписке. - Библиогр. в кн. - ISBN 978-5-9275-2648-2 : Б. ц. - Текст : электронный.

27. Емельянов С.Г. Интеллектуальные системы на основе нечеткой логики и мягких арифметических операций : учебник / С. Г. Емельянов, В. С. Титов, М. В. Бобырь. - Москва : Аргатак-Медиа, 2014. - 338, [7] с. : табл., граф. - Библиогр.: с. 325-336. - 300 экз. - ISBN 978-5-00024-035-9 – Текст : непосредственный.

28. Демидова Л.А. Принятие решений в условиях неопределенности : монография / Л. А. Демидова. - 2-е изд., перераб. - Москва : Горячая линия - Телеком, 2016. - 289 с. - ISBN 978-5-9912-0513-9 : 368.68 р. - Текст : непосредственный.

29. Яхьяева, Г. Э. Основы теории нейронных сетей : учебное пособие / Г. Э. Яхьяева. - 2-е изд., испр. - Москва : Национальный Открытый Университет «ИНТУИТ», 2016. - 200 с. - (Основы информационных технологий). - URL:

<http://biblioclub.ru/index.php?page=book&id=429110>. - ISBN 978-5-94774-818-5 : Б. ц. - Текст : электронный.

30. Лубенцова, Е. В. Системы управления с динамическим выбором структуры, нечеткой логикой и нейросетевыми моделями : монография / Е. В. Лубенцова. - Ставрополь : СКФУ, 2014. - 248 с. - URL: <http://biblioclub.ru/index.php?page=book&id=457413> (дата обращения: 28.04.2022) . - Режим доступа: по подписке. - ISBN 978-5-88648-902-6 : Б. ц. - Текст : электронный.

31. Гелиг, А. Х. Введение в математическую теорию обучаемых распознающих систем и нейронных сетей : учебное пособие / А. Х. Гелиг, А. С. Матвеев. - Санкт-Петербург : Издательство Санкт-Петербургского Государственного Университета, 2014. - 224 с. - (Прикладная математика и информатика). - URL: <http://biblioclub.ru/index.php?page=book&id=457945>. - ISBN 978-5-288-05551-5 : Б. ц. - Текст : электронный.

32. Келлехер, Д. Наука о данных: базовый курс : учебное пособие / Д. Келлехер, Б. Тирни ; науч. ред. З. Мамедьяров ; пер. с англ. М. Белоголовский. - Москва : Альпина Паблишер, 2020. - 224 с. - URL: <http://biblioclub.ru/index.php?page=book&id=598235> (дата обращения: 09.09.2022) . - Режим доступа: по подписке. - ISBN 978-5-9614-3170-4 : Б. ц. - Текст : электронный.

33. Кассим К.Д.А. Моделирование систем искусственного интеллекта в среде Matlab и Fuzzytech : учебное пособие / К. Д. А. Кассим, С. А. Филист, О. В. Шаталова. - Курск : Деловая полиграфия, 2016. - 186 с. - Библиогр.: с. 185. - ISBN 978-5-9908582-8-2. - Текст : непосредственный.

34. Математические методы и инновационные научно-технические разработки : сборник научных трудов / Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования "Юго-Западный государственный университет"; редкол.: В. В. Серебровский (отв. ред.) [и др.]. - Курск : ЮЗГУ, 2014. - 282 с. ; 20. - Библиогр. в конце ст. - 100 экз. - ISBN 978-5-7681-0930-1 : 380.00 р. - Текст : непосредственный.

35. Интеллектуальное планирование траекторий подвижных объектов в средах с препятствиями / Д. А. Белоглазов [и др.] ; под ред. В. Х. Пшихопова. - Москва : Физматлит, 2014. - 295 с. : ил. - Авт. указ. на обороте тит. л. - Библиогр.: с. 276-295 (314 назв.). - ISBN 978-5-9221-1595-7. - Текст : непосредственный.

36. Волков Д.А. Модель, метод и нейросетевое оптико-электронное вычислительное устройство распознавания изображений : специальность 05.13.05 "Элементы и устройства вычислительной техники и систем управления": автореферат диссертации на соискание ученой степени кандидата технических наук / Волков Денис Андреевич ; Юго-Западный государственный университет. - Курск, 2020. - 17 с. - Место защиты: ФГБОУ ВО "Юго-Западный государственный университет"(Курск). - Текст : непосредственный.

37. Программирование, тестирование, проектирование, нейросети, технологии аппаратно-программных средств (практические задания и способы их решения) : учебник / С. В. Веретехина, К. С. Кармицкий, Д. Д. Лукашин [и др.]. - Москва : Директ-Медиа, 2022. - 144 с. - URL: <https://biblioclub.ru/index.php?page=book&id=694782> (дата обращения: 10.01.2023) . - Режим доступа: по подписке. - Библиогр. в кн. - ISBN 978-5-4499-3321-8 : Б. ц. - Текст : электронный.

38. Интеллектуальные системы и технологии : учебное пособие / С. П. Ющенко [и др.] ; Юго-Зап. гос. ун-т. - Курск : Университетская книга, 2018. - 226 с. : ил. - Библиогр.: с. 238-240 (32 назв.). - ISBN 978-5-907138-22-3 : 560.00 р. - Текст : непосредственный.

39. Системная инженерия. Принципы и практика = Systems engineering principles and practice : учебник / А. Косяков [и др.] ; пер. с англ. под ред. В. К. Батоврин. - 2-е изд. - Москва : ДМК Пресс, 2014. - 624 с. : ил. - Указ.: с. 610-619. - 400 экз. - ISBN 978-5-97060-122-8 (в пер.). - Текст : непосредственный.

40. Сидоркина И.Г.. Системы искусственного интеллекта : учебное пособие / И. Г. Сидоркина. - Москва : КНОРУС, 2016. - 246 с. : рис. - Библиогр.: с. 244-245. - ISBN 978-5-406-04876-4 . - Текст : непосредственный.

41. Бабенко Л.К. Параллельные алгоритмы для решения задач защиты информации : монография / Л. К. Бабенко, Е. А. Ищукова, И. Д. Сидоров. - Москва : Горячая линия-Телеком, 2014. - 304 с. : ил. - Библиогр.: с. 222-224. - ISBN 978-5-9912-0426-2 : 340.00 р. - Текст : непосредственный.

42. Кузнецов, А. С. Теория вычислительных процессов : учебник / А. С. Кузнецов, Р. Ю. Царев, А. Н. Князьков. - Красноярск : Сибирский федеральный университет, 2015. - 184 с. - URL: <http://biblioclub.ru/index.php?page=book&id=435696>. - ISBN 978-5-7638-3193-1 : Б. ц. - Текст : электронный.

43. Исакова, А. И. Основы информационных технологий : учебное пособие / А. И. Исакова. - Томск : ТУСУР, 2016. - 206 с. : ил. - URL: <http://biblioclub.ru/index.php?page=book&id=480808> (дата обращения: 18.02.2022) . - Режим доступа: по подписке. - Библиогр.: с. 197-198. - Б. ц. - Текст : электронный.

44. Гунько, А. В. Программирование (в среде Windows) : учебное пособие / А. В. Гунько ; Новосибирский государственный технический университет. - Новосибирск : Новосибирский государственный технический университет, 2019. - 155 с. - URL: <https://biblioclub.ru/index.php?page=book&id=575417> (дата обращения: 03.05.2024) . - Режим доступа: по подписке. - Библиогр. в кн. - ISBN 978-5-7782-3890-9 : Б. ц. - Текст : электронный.

45. Малоразмерные беспилотные летательные аппараты: задачи обнаружения и пути их решения : монография / И. И. Олейник, А. А. Черноморец, В. Г. Андронов [и др.] ; под ред. В. Г. Андропова ; Юго-Зап. гос. ун-т. - Курск : ЮЗГУ, 2021. - 171 с. - Библиогр.: с. 149-170. - ISBN 978-5-7681-1518-0. - Текст : непосредственный.

46. Методологические основы обнаружения малоразмерных беспилотных летательных аппаратов на основе комплексной субполосной обработки

сверхкороткоимпульсных радиолокационных и оптических сигналов : монография / И. И. Олейник, А. А. Черноморец, Д. С. Коптев [и др.] ; под общ. ред. В. Г. Андронов ; Юго-Зап. гос. ун-т. - Курск : ЮЗГУ, 2021. - 204 с. - Библиогр.: с. 198-203. - ISBN 978-5-7681-1526-5 . - Текст : непосредственный.

47. Шевцов, Максим Викторович. Система мониторинга пожарной и медико-экологической безопасности с использованием анализа видеоданных с беспилотных летательных аппаратов : специальность 2.3.1 "Системный анализ, управление и обработка информации (технические науки)": автореферат диссертации на соискание ученой степени кандидата технических наук / Шевцов Максим Викторович ; Академия Государственной противопожарной службы МЧС России (Москва). - Электрон. текстовые дан. (476 КБ). - Москва, 2022. - 20 с. - Место защиты: Юго-Западный государственный университет (Курск). - Текст : непосредственный.

48. Аверченков, В. И. Основы математического моделирования технических систем : учебное пособие / В. И. Аверченков, В. П. Федоров, М. Л. Хейфец. - 4-е изд., стер. - Москва : Флинта, 2021. - 271 с. - URL: <http://biblioclub.ru/index.php?page=book&id=93344> (дата обращения: 16.02.2024) . - Режим доступа: по подписке. - ISBN 978-5-9765-1278-8 : Б. ц. - Текст : электронный.

49. Зыков, С. В. Введение в теорию программирования. Объектно-ориентированный подход : курс лекций / С. В. Зыков. - 2-е изд., испр. - Москва : Национальный Открытый Университет «ИНТУ-ИТ», 2016. - 189 с. - (Основы информационных технологий). - URL: <http://biblioclub.ru/index.php?page=book&id=429073> (дата обращения: 13.05.2024) . - Режим доступа: по подписке. - ISBN 5-9556-0009-4 : Б. ц. - Текст : электронный.

50. Сазонов С.Ю. Системный подход к моделированию процессов возникновения и развития пожаров : монография / С. Ю. Сазонов ; Юго-Зап. гос. ун-т. - Курск : Деловая полиграфия, 2016. - 218 с. - Библиогр.: с. 213-219. - ISBN 978-5-9907910-9-1. - Текст : непосредственный.

ПРИЛОЖЕНИЕ А

Представление графического материала

Графический материал, выполненный на отдельных листах, изображен на рисунках А.1–А.9.

Сведения о ВКРБ

Минобрнауки России

Юго-Западный государственный университет

Кафедра программной инженерии

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА ПО ПРОГРАММЕ БАКАЛАВРИАТА

«Интеллектуальная система распознавания и классификации возгораний,
полученных с БПЛА»

Руководитель ВКРБ
д.т.н, профессор
Томакова Римма Александровна

Автор ВКРБ
студент группы ПО-026
Каракчиев Даниил Александрович

				ВКРБ 20060391.09.03.04.24.006			
				Сведения о ВКРБ			
				Выпускная квалификационная работа бакалавра			
				ЮЗГУ ПО-026			

Рисунок А.1 – Сведения о ВКРБ

Цели и задачи разработки

Цель работы - разработка приложения на базе сверточной нейронной сети для распознавания и классификации возгораний на изображениях, полученных с БПЛА

Для достижения поставленной цели необходимо решить следующие задачи:

- Провести анализ предметной области;
- Разработать концептуальную модель приложения;
- Спроектировать приложение;
- Реализовать приложение.

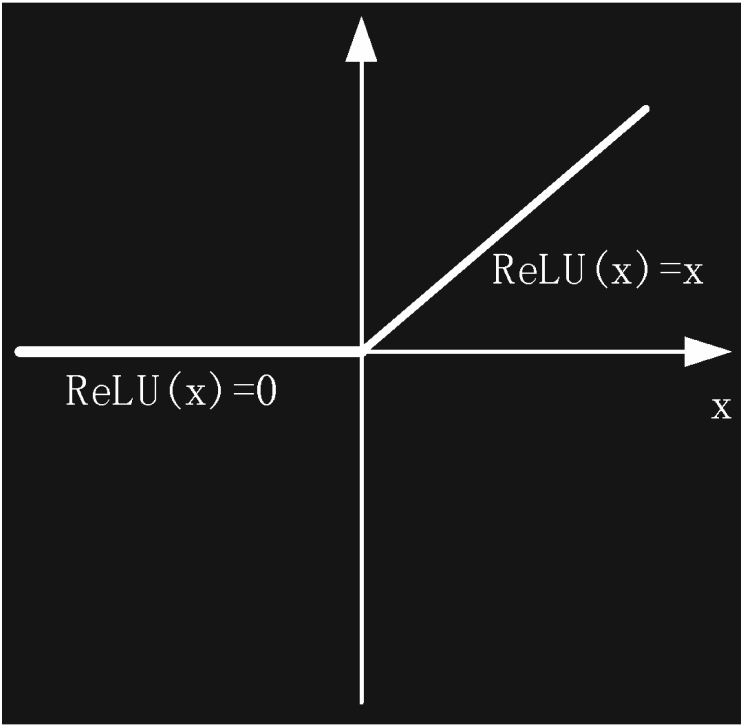
ВКРБ 20060391.09.03.04.24.006			
Автор работы	Фамилия И.О.	Имя	Фамилия
Рецензент	Имя	Фамилия	Имя
Рецензент	Имя	Фамилия	Имя
Рецензент	Имя	Фамилия	Имя
Цели и задачи разработки		Дата	Имя
Выпускная квалификационная работа бакалавра		Дата	Имя
		ЮЗГЧ ПО-026	

Рисунок А.2 – Цель и задачи разработки



Рисунок А.3 – Диаграммы компонентов

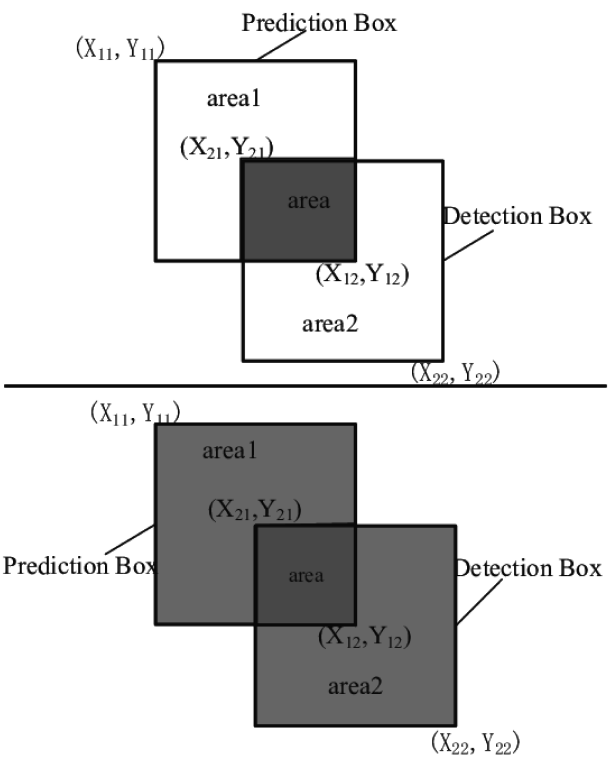
Функция активации ReLU



				ВКРБ 20060391.09.03.04.24.006			
				Функция активации ReLU			
Автор работы	Фамилия И.О.	Имя	Отчество	Дата	Место	Число	
Рецензент	Томасов Р.А.						
Помощник	Малыгин А.А.						
				Выпуск квалификационной работы бакалавра			
				ЮЗГЧ ПО-026			

Рисунок А.5 – Функция активации ReLU

Функция вычисления ошибки IoU Loss

$$\text{IOU} = \frac{\text{area}}{\text{area1} + \text{area2} - \text{area}}$$


ВКРБ 20060391.09.03.04.24.006			
Автор работы	Валиткин И. Ю.	Дата	2024
Рецензент	Томасов Р. А.	Дата	2024
Помощник	Малыгин А. А.	Дата	2024
Выпускная квалификационная работа бакалавра			ЮЗГУ ПО-026

Рисунок А.6 – Функция вычисления ошибки IoU Loss

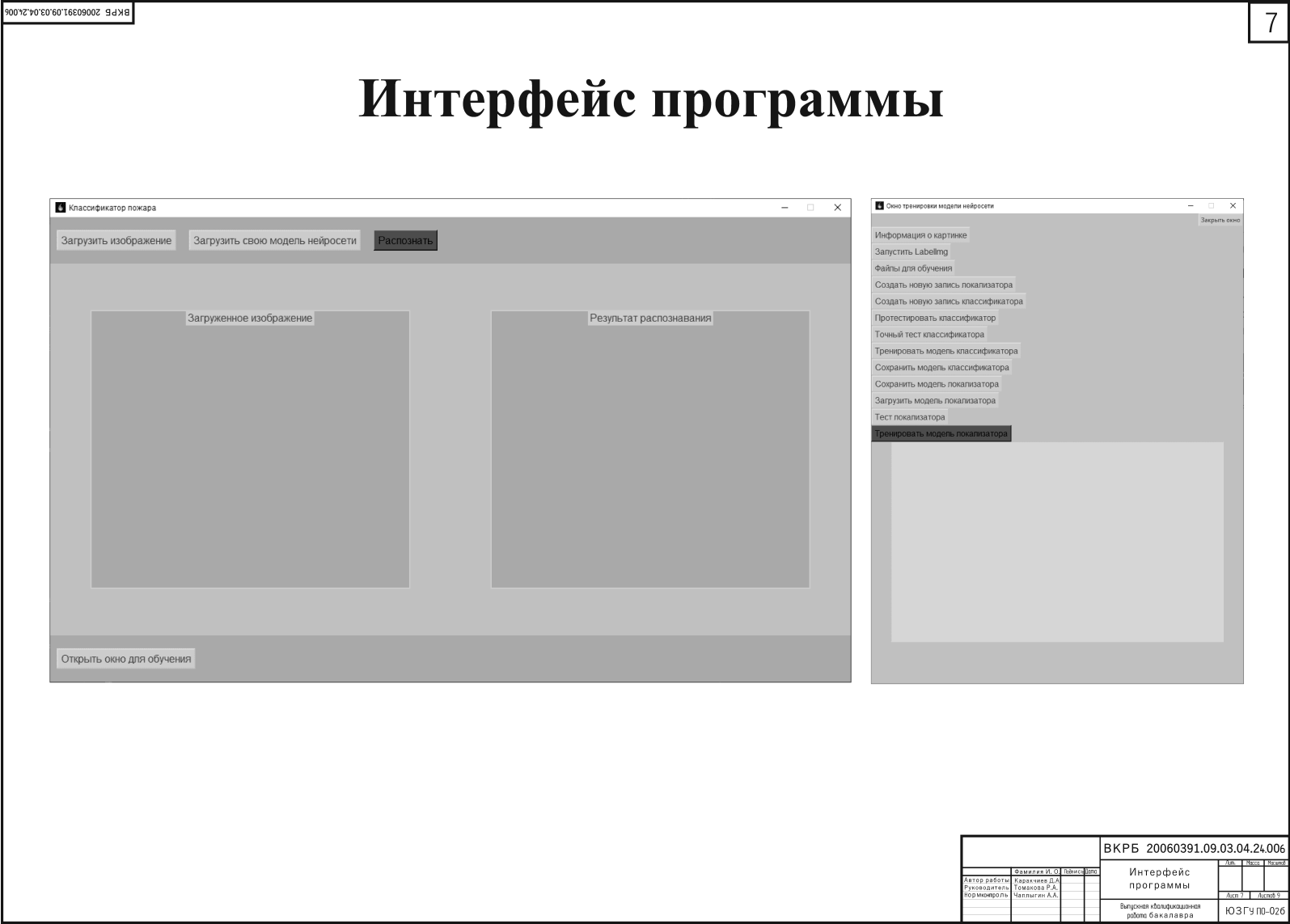


Рисунок А.7 – Интерфейс приложения

		ВКРБ 20060391.09.03.04.24.006			
	Фамилия И.О.	Подпись	Дата	Место	Масштаб
Автор работы Руководитель Нормоконтроль	Караченко Д.А.				
	Томасова Р.А.				
	Чалыгин А.А.				
Выполнен квалифицированным специалистом			ИЮН 8		
Исполнитель: Бакарадзе			ИЮН 9		
			ИЮН 9		

Рисунок А.8 – Демонстрация работы программы

Заключение

Основные результаты работы:

Проведен анализ предметной области. Выявлена необходимость использовать язык Python и библиотеки TensorFlow, Keras и OpenCV;

Разработана концептуальная модель приложения. Разработана модель данных системы. Определены требования к системе;

Осуществлено проектирование приложения. Разработана архитектура нейронной сети. Разработан пользовательский интерфейс приложения. Выполнено обучение сверточной нейронной сети;

Реализовано и протестировано приложение. Проведено модульное и системное тестирование.

				ВКРБ 20060391.09.03.04.24.006			

ПРИЛОЖЕНИЕ Б

Фрагменты исходного кода программы

main.py

```
1 import os
2
3 os.environ['TF_CPP_MIN_LOG_LEVEL'] = '2'
4 import tensorflow as tf
5 import cv2
6 import numpy as np
7 import sys
8 import matplotlib.pyplot as plt
9 from tkinter import *
10 from tkinter import filedialog
11 # from tkinter.ttk import Combobox
12 from tkinter.messagebox import showwarning, showinfo
13 import tkinter as tk
14 from PIL import Image, ImageTk
15 import io
16 from dataprocessing import dataprocess
17 from creatingtfrecordclassifier import check_xml_list
18 from creatingtfrecordlocalizer import create_load_tfrec_for_localizer
19 from classifier import test_classifier, imshow_and_pred, trainclass,
    saveclassifier
20 from training import train, savemodel, loadmodel, testing
21 import subprocess
22 import pkg_resources
23
24 gpus = tf.config.experimental.list_physical_devices('GPU')
25 for gpu in gpus:
26     tf.config.experimental.set_memory_growth(gpu, True)
27
28 global path_for_one
29 localizator = tf.keras.models.load_model('my_bb_model.keras')
30 classifier = tf.keras.models.load_model('my_classifier.keras')
31
32 window = Tk()
33 window.geometry("1240x720")
34 window.resizable(False, False)
35 window.title("Классификатор пожара")
36 window.iconbitmap(default="bplaicon.ico")
37 window.configure(bg='silver')
38 image_field_raw = Canvas(bg="darkgray", height=500, width=500)
39 image_field_raw.place(relx=0.05, rely=0.2, relheight=0.6, relwidth=0.4)
40 title1 = Label(image_field_raw, font=30, text=f"Загруженное изображение")
41 title1.pack(anchor=N)
42 image_field_ready = Canvas(bg="darkgray", height=500, width=500)
43 image_field_ready.place(relx=0.55, rely=0.2, relheight=0.6, relwidth=0.4)
44 title2 = Label(image_field_ready, font=30, text=f"Результат распознавания")
45 title2.pack(anchor=N)
46 frame = Frame(window, bg='darkgray')
47 frame.place(relheight=0.1, relwidth=1)
48 frame1 = Frame(window, bg='darkgray')
```

```

49 frame1.place(relheight=0.1, relwidth=1, rely=0.9)
50
51
52 class ConsoleRedirector(object):
53     def __init__(self, text_widget):
54         self.text_widget = text_widget
55
56     def write(self, string):
57         self.text_widget.insert(tk.END, string)
58         self.text_widget.see(tk.END)
59
60     def flush(self):
61         pass
62
63
64 # функция работы с нейросетями: подготовка изображения, детекция
65 # на выходе три массива: координаты (10,4) , классы (10) , вероятности (10)
66 def detect_objects(image):
67     # подготовка картинки
68     image = tf.cast(image, dtype=tf.float32) / 256
69     small_image = tf.image.resize(image, (128, 128))
70     big_image = tf.image.resize(image, (1024, 1024))
71     image_exp = tf.expand_dims(small_image, axis=0)
72
73     # локализация
74     bb_cords = localizator(image_exp)
75     bb_cords = tf.squeeze(bb_cords, axis=0)
76
77     # нормализация по размеру картинки
78     bb_cords = (bb_cords + 1) / 2 * 128
79     bb_cords = tf.reshape(bb_cords, [10, 3])
80
81     # разделяем на элементы
82     fxmin, ymin, fxmax = tf.split(bb_cords, 3, axis=1)
83
84     # нормализуем
85     xmin = tf.minimum(fxmin, fxmax)
86     xmax = tf.maximum(fxmin, fxmax)
87
88     xmin = tf.clip_by_value(xmin, 0, 128)
89     ymin = tf.clip_by_value(ymin, 0, 128)
90
91     size = xmax - xmin
92
93     # сумма координаты и размера должны быть <= 128
94     xsize = tf.clip_by_value(size, 1, 128 - xmin)
95     ysize = tf.clip_by_value(size, 1, 128 - ymin)
96
97     # нарезаем и собираем в массив (10, 32, 32, 3)
98     ymin *= 8
99     xmin *= 8
100    ysize *= 8
101    xsize *= 8
102    for n in range(10):

```

```

103         ii = tf.image.crop_to_bounding_box(big_image, int(ymin[n][0]), int(
            xmin[n][0]), int(ysize[n][0]),
104                                           int(xsize[n][0]))
105         ii = tf.image.resize(ii, (128, 128))
106         ii = tf.expand_dims(ii, axis=0)
107         if n == 0:
108             cropped = ii
109         else:
110             cropped = tf.concat([cropped, ii], axis=0)
111
112     # классифицируем
113     probs = classifier(cropped)
114
115     probs = probs.numpy()
116
117     # считаем метки класса (индекс наибольшего среди вероятностей)
118     ma = np.amax(probs, axis=1)
119     ma = np.expand_dims(ma, axis=1)
120     _, classes = np.where(probs == ma)
121
122     # берем ту вероятность, которая наибольшая
123     res_probs = []
124     for a in range(10):
125         res_probs.append(probs[a][classes[a]])
126
127     # собираем координаты в нормальный вид
128     cords = tf.concat([xmin / 8, ymin / 8, xmin / 8 + xsize / 8, ymin / 8 +
        ysize / 8], axis=1)
129     cords = cords.numpy()
130
131     return cords, classes, res_probs
132
133
134     # th - вероятность, ниже которой рамки не отображаются
135     namespace = {0: 'NOTHING', 1: 'Fire'}
136
137
138     def visualize(in_image, cords, classes, probs, th=0.5):
139         big_image = tf.image.resize(in_image, (1024, 1024)).numpy() / 256
140
141         font = cv2.FONT_HERSHEY_SIMPLEX
142         fontScale = 1
143         thickness = 2
144
145         for i in range(len(cords)):
146             if classes[i] != 0 and probs[i] >= th:
147                 # введем цвета для всех объектов
148                 if classes[i] == 1:
149                     color = (1, 0, 0)
150                 if classes[i] == 2:
151                     color = (0, 1, 0)
152                 text = namespace[classes[i]] + ' ' + str(probs[i] * 100) + '%'
153
154                 org = (int(cords[i][0]) * 8, int(cords[i][1]) * 8 - 10)

```

```

155         # рисуем текст и квадраты
156         big_image = cv2.putText(big_image, text, org, font, fontScale,
157                                 color, thickness, cv2.LINE_AA)
158         big_image = cv2.rectangle(big_image, (int(cords[i][0]) * 8, int(
159             cords[i][1]) * 8),
160                                     (int(cords[i][2]) * 8, int(cords[i][3])
161                                         * 8), color, 5)
162
163     return big_image
164
165     # функция объединяет две рамки одного класса в одну, если они пересекаются
166     # tau - порог IoU этих рамок чтобы их объединить (0.1 - все подряд, 0.9 -
167     # только очень близкие)
168     # функцию можно применять несколько раз, результат улучшится
169
170     def prettify(cords, classes, probs, tau=0.2):
171         newcords = []
172         newclasses = []
173         newprobs = []
174
175         for i1 in range(len(classes)):
176             if classes[i1] != 0:
177                 found = False
178                 for i2 in range(len(classes)):
179                     if classes[i2] != 0 and i1 != i2:
180                         # вычислим IoU, самый надежный способ определить
181                         # совпадение
182                         x_overlap = max(0, min(cords[i1][2], cords[i2][2]) - max(
183                             cords[i1][0], cords[i2][0]))
184                         y_overlap = max(0, min(cords[i1][3], cords[i2][3]) - max(
185                             cords[i1][1], cords[i2][1]))
186                         inter = x_overlap * y_overlap
187                         area1 = (cords[i1][2] - cords[i1][0]) * (cords[i1][3] -
188                             cords[i1][1])
189                         area2 = (cords[i2][2] - cords[i2][0]) * (cords[i2][3] -
190                             cords[i2][1])
191                         union = area1 + area2 - inter
192                         IoU = inter / union
193                         if IoU > tau:
194                             # считаем среднее по всем координатам между двух
195                             # рамок
196                             newcord = [(cords[i1][0] + cords[i2][0]) // 2, (cords
197                                 [i1][1] + cords[i2][1]) // 2,
198                                         (cords[i1][2] + cords[i2][2]) // 2, (cords
199                                 [i1][3] + cords[i2][3]) // 2]
200                             newcords.append(newcord)
201
202                             newclasses.append(classes[i1])
203                             newprobs.append(probs[i1])
204
205                             # обнуляем класс, чтобы больше не крутить эту рамку

```

```

197         classes[i1] = 0
198         classes[i2] = 0
199         found = True
200
201         # если ни с чем не объединили, так и оставляем
202         if found == False:
203             newcords.append(cords[i1])
204             newclasses.append(classes[i1])
205             newprobs.append(probs[i1])
206
207     return newcords, newclasses, newprobs
208
209
210 def detect_fire_in_image(path_for_one):
211     image = cv2.imread(path_for_one)
212     image = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
213     cords, classes, probs = detect_objects(image)
214     for i in range(1):
215         cords, classes, probs = prettify(cords, classes, probs, 0.8)
216     result = visualize(image, cords, classes, probs, 0.5)
217
218     buf = io.BytesIO()
219     plt.figure()
220     plt.imshow(result)
221     plt.axis('off') # Скрываем оси
222     plt.savefig(buf, format='png')
223     buf.seek(0)
224
225     original_image = Image.open(buf)
226     resized_image = original_image.resize(
227         (int(original_image.width * 1.35), int(original_image.height * 1.15))
228     )
229     image_tk = ImageTk.PhotoImage(resized_image)
230
231     image_field_ready.create_image(-192, -65, anchor=NW, image=image_tk, tags
232     ='Старое')
233     image_field_ready.image_tk = image_tk # Сохраняем ссылку на изображение
234
235     # Закрываем буфер
236     buf.close()
237
238
239 def loadimage():
240     global image
241     global path_for_one
242     path_for_one = filedialog.askopenfilename(filetypes=[("Изображения", "*.
243     png;*.jpg;*.jpeg")])
244     if path_for_one:
245         pil_image = Image.open(path_for_one)
246         pil_image = pil_image.resize((500, 500))
247         image = ImageTk.PhotoImage(pil_image)
248         image_field_raw.create_image(0, 0, anchor=NW, image=image)

```

```

248 def detect():
249     global path_for_one
250     detect_fire_in_image(path_for_one)
251
252
253 def check_and_install_packages():
254     required = {'PyQt5', 'sip'}
255     installed = {pkg.key for pkg in pkg_resources.working_set}
256     missing = required - installed
257     if missing:
258         python = sys.executable
259         subprocess.check_call([python, '-m', 'pip', 'install', *missing],
                                stdout=subprocess.DEVNULL)
260
261
262 def run_labelimg():
263     showinfo("Обязательная информация",
264             "Для использования этого приложения необходимо иметь\n"
265             "установленные пакеты PyQt5 и sip")
266     check_and_install_packages()
267     labeling_path = 'C:/ForestFireDiplomFinVer/labelimg/labelImg.py'
268     subprocess.run(['python', labeling_path])
269
270 def create_tfrec_classifier():
271     showwarning("Предупреждение", "Классификатор настроен и не нуждается в\n"
272             "добавлении новых данных")
273
274 def start_train_localizer():
275     loadmodel()
276     train()
277
278
279 def train_window_open():
280     window1 = Tk()
281     window1.geometry("720x910")
282     window1.resizable(False, False)
283     window1.title("Окно тренировки модели нейросети")
284     window1.iconbitmap(default="bplaicon.ico")
285     window1.configure(bg='silver')
286
287     close_button = Button(window1, text="Заккрыть окно", command=lambda:
288             window1.destroy())
289     close_button.pack(anchor=NE, expand=0)
290
291     processdataforonefile = Button(window1, font=40, text=f"Информация о\n"
292             "картинке", command=dataprocess)
293     processdataforonefile.pack(anchor=NW)
294
295     runlabelimg = Button(window1, font=40, text=f"Запустить LabelImg",
296             command=run_labelimg)
297     runlabelimg.pack(anchor=NW)

```

```

296 check_xml_list_but = Button(window1, font=40, text=f"Файлы для обучения",
    command=check_xml_list)
297 check_xml_list_but.pack(anchor=NW)
298
299 check_xml_list_but = Button(window1, font=40, text=f"Создать новую запись
    локализатора",
300                                command=create_load_tfrec_for_localizer)
301 check_xml_list_but.pack(anchor=NW)
302
303 tfrec_classifier = Button(window1, font=40, text=f"Создать новую запись
    классификатора",
304                                command=create_tfrec_classifier)
305 tfrec_classifier.pack(anchor=NW)
306
307 classifier_test_but = Button(window1, font=40, text=f"Протестировать
    классификатор", command=test_classifier)
308 classifier_test_but.pack(anchor=NW)
309
310 true_classifier_test_but = Button(window1, font=40, text=f"Точный тест
    классификатора", command=imshow_and_pred)
311 true_classifier_test_but.pack(anchor=NW)
312
313 train_classifier = Button(window1, font=40, text=f"Тренировать модель
    классификатора", command=trainclass)
314 train_classifier.pack(anchor=NW)
315
316 save_classifier = Button(window1, font=40, text=f"Сохранить модель
    классификатора", command=saveclassifier)
317 save_classifier.pack(anchor=NW)
318
319 save_localizer = Button(window1, font=40, text=f"Сохранить модель
    локализатора", command=savemodel)
320 save_localizer.pack(anchor=NW)
321
322 load_localizer = Button(window1, font=40, text=f"Загрузить модель
    локализатора", command=loadmodel)
323 load_localizer.pack(anchor=NW)
324
325 test_localizer = Button(window1, font=40, text=f"Тест локализатора",
    command=testing)
326 test_localizer.pack(anchor=NW)
327
328 train_localizer = Button(window1, font=40, text=f"Тренировать модель
    локализатора", bg='Green', fg='black',
329                                command=start_train_localizer)
330 train_localizer.pack(anchor=NW)
331
332 text = tk.Text(window1)
333 text.pack(anchor='center')
334 sys.stdout = ConsoleRedirector(text)
335
336
337 def load_model_data():
338     global localizator

```

```

339     model_path = filedialog.askopenfilename(filetypes=[("Модель keras", "*.
        keras")])
340     if model_path:
341         localizator = tf.keras.models.load_model(model_path)
342
343
344     load_image_but = Button(frame, font=40, text=f"Загрузить изображение",
        command=loadimage)
345     load_image_but.grid(column=0, row=0, padx=10, pady=20)
346
347     load_model_but = Button(frame, font=40, text=f"Загрузить свою модель
        нейросети", command=load_model_data)
348     load_model_but.grid(column=1, row=0, padx=10)
349
350     detect_but = Button(frame, font=40, text=f"Распознать", bg='Green', fg='black
        ', command=detect)
351     detect_but.grid(column=2, row=0, padx=10)
352
353     train_window_but = Button(frame1, font=40, text=f"Открыть окно для обучения",
        command=train_window_open)
354     train_window_but.grid(column=0, row=0, padx=10, pady=20)
355
356     window.mainloop()

```

classifier.py

```

1  import tensorflow as tf
2  from IPython.display import clear_output
3  import numpy as np
4  import matplotlib.pyplot as plt
5  from tensorflow import keras
6  from tensorflow.keras.layers import Dense, Flatten, Input, Conv2D,
    GlobalAveragePooling2D, Dropout
7
8
9  dataset = tf.data.TFRecordDataset('classifier_dataset.tfrecord')
10
11  def parse_record(record):
12      # нужно описать приходящий экземпляр
13      # имена элементов как при записи
14      feature_description = {
15          'img': tf.io.FixedLenFeature([], tf.string),
16          'name': tf.io.FixedLenFeature([], tf.string)
17      }
18      parsed_record = tf.io.parse_single_example(record, feature_description)
19      img = tf.io.parse_tensor(parsed_record['img'], out_type=tf.float32)
20      name = tf.io.parse_tensor(parsed_record['name'], out_type=tf.int32)
21      return img, name
22
23
24  # пройдемся по записи и распакуем ее
25  dataset = dataset.map(parse_record)
26
27  # еще раз проверим
28

```

```

29
30 dataset = dataset.shuffle(50).cache().prefetch(buffer_size=tf.data.AUTOTUNE).
    batch(64).shuffle(50)
31
32 def test_classifier():
33     for i, n in dataset.take(1):
34         plt.figure(figsize=(10, 6))
35         i = i.numpy()
36         n = n.numpy()
37         for nn in range(32):
38             ax = plt.subplot(5, 10, 1 + nn)
39             plt.title(n[nn])
40             plt.imshow(i[nn])
41             plt.axis('off')
42         plt.show()
43
44
45 base_model = tf.keras.applications.MobileNetV2(weights='imagenet',
    include_top=False, input_shape=(128, 128, 3))
46 base_model.trainable = False
47
48 # Архитектура сети классификатора
49 inputs = Input((128,128,3))
50 x = base_model(inputs)
51 x = Flatten()(x)
52 x = Dropout(0.5)(x)
53 x = Dense(256, activation = 'relu')(x)
54 x = Dropout(0.2)(x)
55 x = Dense(3, activation = 'softmax')(x)
56
57 outputs = x
58
59 classifier = keras.Model(inputs, outputs)
60
61
62 # модель нейросети классификатора
63 class Model(tf.keras.Model):
64     def __init__(self, nn):
65         super(Model, self).__init__()
66         self.nn = nn
67         self.optimizer = tf.keras.optimizers.Adam(1e-3)
68
69     def get_loss(self, y, preds):
70         loss = tf.keras.losses.CategoricalCrossentropy()(tf.one_hot(y, 3),
            preds)
71         return loss
72
73     @tf.function
74     def training_step(self, x, y):
75         with tf.GradientTape() as tape:
76             preds = self.nn(x)
77             loss = self.get_loss(y, preds)
78
79         gradients = tape.gradient(loss, self.nn.trainable_variables)

```

```

80         self.optimizer.apply_gradients(zip(gradients, self.nn.
            trainable_variables))
81     return tf.reduce_mean(loss)
82
83
84 model = Model(classifier)
85
86
87 # Тензор
88 # for i, c in dataset.take(1):
89 #     print(tf.reduce_mean(model.training_step(i, c)))
90
91 # проверка тренировки как в training.py
92 def trainclass():
93     hist = np.array(np.empty([0]))
94     epochs = 10
95
96     for epoch in range(1, epochs + 1):
97         loss = 0
98         lc = 0
99         for step, (i, n) in enumerate(dataset):
100             loss += tf.reduce_mean(model.training_step(i, n))
101             lc += 1
102         clear_output(wait=True)
103         print(epoch)
104         hist = np.append(hist, loss / lc)
105         plt.plot(np.arange(0, len(hist)), hist)
106         plt.show()
107         if epoch != epochs:
108             plt.close()
109
110
111 def imshow_and_pred():
112     n = 5
113     plt.figure(figsize=(10, 6))
114     for images, labels in dataset.take(1):
115         preds = model.nn(images)
116         for i in range(n):
117             img = images[i]
118
119             pred = preds[i].numpy()
120             print(pred)
121             ax = plt.subplot(3, n, i + 1 + n)
122             plt.imshow(img, cmap='gist_gray')
123
124             ma = pred.max()
125             res = np.where(pred == ma)
126
127             plt.title(str(res[0][0]) + ' ' + str(round(pred[res[0][0]], 3)))
128             plt.axis('off')
129             ax.get_yaxis().set_visible(False)
130     plt.show()
131
132

```

```

133 def saveclassifier():
134     model.nn.save('my_classifier.keras')

```

training.py

```

1 import tensorflow as tf
2 from IPython.display import clear_output
3
4 gpus = tf.config.experimental.list_physical_devices('GPU')
5 for gpu in gpus:
6     tf.config.experimental.set_memory_growth(gpu, True)
7 from tensorflow import keras
8 from tensorflow.keras.layers import Dense, Flatten, Input, Conv2D,
    Conv2DTranspose, Concatenate, LeakyReLU, Dropout
9 import cv2
10 import numpy as np
11 import matplotlib.pyplot as plt
12
13 # Структура сети по детекции
14 inputs = Input((128, 128, 3))
15 x = Conv2D(32, 3, activation='relu', padding='same')(inputs)
16 x = Conv2D(64, 3, activation='relu', padding='same', strides=2)(x)
17 x = Conv2D(64, 3, activation='relu', padding='same')(x)
18 x = Conv2D(64, 3, activation='relu', padding='same', strides=2)(x)
19 x = Conv2D(128, 3, activation='relu', padding='same')(x)
20 x = Conv2D(128, 3, activation='relu', padding='same', strides=2)(x)
21 x = Conv2D(128, 3, activation='relu', padding='same')(x)
22 x = Conv2D(256, 3, activation='relu', padding='same', strides=2)(x)
23 x = Conv2D(256, 3, activation='relu', padding='same')(x)
24 x = Flatten()(x)
25 x = Dropout(0.2)(x)
26 x = Dense(256, activation='relu')(x)
27 x = Dense(30)(x) # 3*10 = 30 у нейросети это просто выходы подряд
28
29 outputs = x
30
31 boxregressor = keras.Model(inputs, outputs)
32
33 # прочитаем запись
34 dataset = tf.data.TFRecordDataset('bounding_box_dataset.tfrecord')
35
36
37 def parse_record(record):
38     # имена элементов как при записи
39     feature_description = {
40         'img': tf.io.FixedLenFeature([], tf.string),
41         'cords': tf.io.FixedLenFeature([], tf.string)
42     }
43     parsed_record = tf.io.parse_single_example(record, feature_description)
44     img = tf.io.parse_tensor(parsed_record['img'], out_type=tf.float32)
45     cords = tf.io.parse_tensor(parsed_record['cords'], out_type=tf.float32)
46     return img, cords
47
48
49 # пройдемся по записи и распакуем ее

```

```

50 dataset = dataset.map(parse_record)
51
52 dataset = dataset.cache().prefetch(buffer_size=tf.data.AUTOTUNE).batch(32).
    shuffle(40)
53
54
55 # функция с вычислением IoU loss
56 def IoU_Loss(true, pred):
57     # (32, 5, 4)
58     t1 = true
59     t2 = pred
60
61     minx1, miny1, maxx1, maxy1 = tf.split(t1, 4, axis=2)
62
63     fminx, miny2, fmaxx = tf.split(t2, 3, axis=2)
64
65     minx2 = tf.minimum(fminx, fmaxx)
66     maxx2 = tf.maximum(fminx, fmaxx)
67
68     delta = maxx2 - minx2
69
70     maxy2 = miny2 + delta
71
72     intersection = 0.0
73
74     # найдем пересечение каждого из предсказанных с каждым из реальных
75     # сложим все вместе
76     for i1 in range(10):
77         for i2 in range(10):
78             x_overlap = tf.maximum(0.0, tf.minimum(maxx1[:, i1], maxx2[:, i2]
79                 ) - tf.maximum(minx1[:, i1], minx2[:, i2]))
80             y_overlap = tf.maximum(0.0, tf.minimum(maxy1[:, i1], maxy2[:, i2]
81                 ) - tf.maximum(miny1[:, i1], miny2[:, i2]))
82             intersection += x_overlap * y_overlap
83
84     # стремимся сделать площади всех элементов такими-же, как у реальных
85     # просто среднеквадратичной ошибкой
86
87     beta1 = 0.0
88     for i1 in range(10):
89         for i2 in range(10):
90             x_overlap = tf.maximum(0.0, tf.minimum(maxx1[:, i1], maxx1[:, i2]
91                 ) - tf.maximum(minx1[:, i1], minx1[:, i2]))
92             y_overlap = tf.maximum(0.0, tf.minimum(maxy1[:, i1], maxy1[:, i2]
93                 ) - tf.maximum(miny1[:, i1], miny1[:, i2]))
94             if i1 == i2:
95                 beta1 += (x_overlap * y_overlap) ** 2
96             else:
97                 beta1 += x_overlap * y_overlap
98
99     beta2 = 0.0
100    for i1 in range(10):
101        for i2 in range(10):

```

```

98         x_overlap = tf.maximum(0.0, tf.minimum(maxx2[:, i1], maxx2[:, i2
99         ]) - tf.maximum(minx2[:, i1], minx2[:, i2]))
100        y_overlap = tf.maximum(0.0, tf.minimum(maxy2[:, i1], maxy2[:, i2
101        ]) - tf.maximum(miny2[:, i1], miny2[:, i2]))
102        if i1 == i2:
103            beta2 += (x_overlap * y_overlap) ** 2
104        else:
105            beta2 += x_overlap * y_overlap
106
107        loss = (beta1 - beta2) ** 2 - intersection
108
109        return loss
110
111    # Класс модели нейросети
112    class Model(tf.keras.Model):
113        def __init__(self, nn_box):
114            super(Model, self).__init__()
115            self.nn_box = nn_box
116
117            self.box_optimizer = tf.keras.optimizers.Adam(3e-4, clipnorm=1.0)
118
119    @tf.function
120    def training_step(self, x, true_boxes):
121        with tf.GradientTape() as tape_box:
122            pred = self.nn_box(x, training=True)
123            pred = tf.reshape(pred, [-1, 10, 3])
124
125            loss = IoU_Loss(true_boxes, pred)
126            # print('test', tf.reduce_mean(IoU_Loss(true_boxes,
127            # true_boxes) ))
128
129            # Backpropagation.
130            grads = tape_box.gradient(loss, self.nn_box.trainable_variables)
131            self.box_optimizer.apply_gradients(zip(grads, self.nn_box.
132            trainable_variables))
133
134        return loss
135
136    model = Model(boxregressor)
137
138    # показывает тензор
139    # for i, c in dataset.take(1):
140    #     print(tf.reduce_mean(model.training_step(i, c)))
141
142    def savemodel():
143        # сохранить
144        model.nn_box.save('my_bb_model.keras')
145
146    def loadmodel():
147        # загрузить веса

```

```

148     model.nn_box.load_weights('my_bb_model.keras')
149
150
151 def testing():
152     for ii, cc in dataset.take(1):
153         # обрабатываем целый батч, используем только пять элементов
154         pred = model.nn_box(ii)
155         plt.figure(figsize=(10, 6))
156
157         for num in range(3):
158             i = ii[num]
159
160             pred = tf.reshape(pred, [-1, 10, 3])
161             c = pred[num]
162
163             ax = plt.subplot(1, 5, num + 1)
164             # переход в numpy для работы в opencv
165             i = i.numpy()
166             c = c.numpy()
167             c = (c + 1) / 2 * 128 # обратно из от -1...1 к 0...64
168             c = c.astype(np.int16) # для opencv
169             for bb in c:
170                 bb0 = min(bb[0], bb[2])
171                 bb2 = max(bb[0], bb[2])
172                 i = cv2.rectangle(i, (bb0, bb[1]), (bb2, bb[1] + (bb2 - bb0)),
173                                   (0, 1, 0), 1)
174             plt.imshow(i)
175
176         plt.show()
177         # print(c)
178
179 def train():
180     hist = np.array(np.empty([0]))
181     epochs = 10 #<<<----- Кол-во эпох
182     ff = 0
183     for epoch in range(1, epochs + 1):
184         loss = 0
185         lc = 0
186         for step, (i, c) in enumerate(dataset):
187             loss += tf.reduce_mean(model.training_step(i, c))
188             lc += 1
189         clear_output(wait=True)
190         print(epoch)
191         hist = np.append(hist, loss / lc)
192
193         plt.plot(np.arange(0, len(hist)), hist)
194         plt.show()
195         testing()
196
197
198 #loadmodel()
199 #train()

```

Место для диска